

Hardness of KT Characterizes Parallel Cryptography

Hanlin Ren*
University of Oxford

Rahul Santhanam†
University of Oxford

August 25, 2025

Abstract

A recent breakthrough of Liu and Pass (FOCS'20) shows that one-way functions exist if and only if the (polynomial-)time-bounded Kolmogorov complexity, K^t , is bounded-error hard on average to compute. In this paper, we strengthen this result and extend it to other complexity measures:

- We show, perhaps surprisingly, that the KT complexity is bounded-error average-case hard if and only if there exist one-way functions in *constant parallel time* (i.e. NC^0). This result crucially relies on the idea of *randomized encodings*. Previously, a seminal work of Applebaum, Ishai, and Kushilevitz (FOCS'04; SICOMP'06) used the same idea to show that NC^0 -computable one-way functions exist if and only if logspace-computable one-way functions exist.
- Inspired by the above result, we present randomized average-case reductions among the NC^1 -versions and logspace-versions of K^t complexity, and the KT complexity. Our reductions preserve both bounded-error average-case hardness and zero-error average-case hardness. To the best of our knowledge, this is the first reduction between the KT complexity and a variant of K^{poly} complexity.
- We prove tight connections between the hardness of K^t complexity and the hardness of (the hardest) one-way functions. In analogy with the Exponential-Time Hypothesis and its variants, we define and motivate the *Perebor Hypotheses* for complexity measures such as K^t and KT. We show that a Strong Perebor Hypothesis for K^t implies the existence of (weak) one-way functions of near-optimal hardness $2^{n-o(n)}$. To the best of our knowledge, this is the first construction of one-way functions of near-optimal hardness based on a natural complexity assumption about a search problem.
- We show that a Weak Perebor Hypothesis for MCSP implies the existence of one-way functions, and establish a partial converse. This is the first unconditional construction of one-way functions from the hardness of MCSP over a natural distribution.
- Finally, we study the average-case hardness of MKtP. We show that it characterizes cryptographic pseudorandomness in one natural regime of parameters, and complexity-theoretic pseudorandomness in another natural regime.

*h4n1in.r3n@gmail.com

†rahul.santhanam@cs.ox.ac.uk

Contents

1	Introduction	1
1.1	Background and Motivation	1
1.2	Our Contributions	2
1.3	Related Work	6
1.4	Organization	7
2	Intuitions and Techniques	7
2.1	Parallel Cryptography and the Hardness of KT	7
2.2	Applebaum–Ishai–Kushilevitz as a Reduction	9
2.3	Tighter Connections	10
2.4	MCSP-Related Results	11
2.5	Using Hardness of Kt to Capture Cryptographic and Complexity-Theoretic Pseudorandomness	11
3	Preliminaries	12
4	KT Complexity and Parallel Cryptography	18
4.1	One-Way Functions in NC^0 from Hardness of MKTP	19
4.2	Hardness of MKTP from One-Way Functions in $\oplus\text{L}$	20
4.3	Bounded-Error Average-Case Robustness of Meta-Complexity	24
4.4	Zero-Error Average-Case Reductions	24
5	Tighter Connections between Meta-Complexity and One-Way Functions	27
5.1	Technical Building Blocks	28
5.2	CondEP-PRGs from Weak One-Way Functions	29
5.3	Proof of Theorem 5.2	30
5.4	Proof of Theorem 5.3	32
5.5	The Perebor Hypotheses	34
6	MCSP-Related Results	35
6.1	Preliminaries	36
6.2	One-Way Functions from Hardness of MCSP	37
6.3	Hardness of MCSP from DLOGTIME One-Way Functions	39
7	The Average-Case Complexity of MKtP	41
7.1	Characterizing One-Way Functions Using MKtP	41
7.2	A Complexity Theoretic Analogue	42
8	Open Problems	44
	References	45
A	Proof of Theorem 5.9	51
B	Proof of Theorem 5.15	55

1 Introduction

1.1 Background and Motivation

1.1.1 Meta-Complexity

Let μ be a complexity measure, such as the circuit size of a Boolean function or the time-bounded Kolmogorov complexity of a string. Traditional complexity theory studies the complexity measure on fixed functions, e.g. the AC^0 complexity of the Parity function. In contrast, we study the *meta-complexity* problem associated with μ : given an input function, what is its μ value?

Meta-complexity problems are fundamental to theoretical computer science and have been studied since the very beginning of the discipline [Tra84]. They have connections to several areas of theoretical computer science, including circuit lower bounds, learning, meta-mathematics, average-case complexity, and cryptography. However, our knowledge about them is still very limited compared to our knowledge of other fundamental problems such as the Satisfiability problem.

Some of the basic complexity questions about meta-complexity include:

- Is computing a given measure μ complete for some natural complexity class? For example, is the Minimum Circuit Size Problem (MCSP, [KC00]) NP-complete?
- Can we show unconditional circuit lower bounds for computing μ , at least for weak circuit classes? Can we distinguish truth tables with $2^{o(n)}$ -size circuits from random truth tables by a small $AC^0[2]$ circuit?
- Is deciding whether μ is at least some parameter k robust to the choice of the parameter k ? Let $MCSP[s(n)]$ denote the problem of whether an input function (represented as a truth table) has circuit complexity at most $s(n)$; are $MCSP[2^{n/2}]$ and $MCSP[2^{n/3}]$ computationally equivalent?
- How do low-level definitional issues affect the complexity of μ ? Does the complexity of the time-bounded version of Kolmogorov complexity (“ K^t ”) depend on the universal Turing machine that defines it?
- For which pairs of measures μ and μ' can we show that the problem of computing μ reduces to the problem of computing μ' ? Can we reduce computing the time-bounded version of Kolmogorov complexity to computing circuit complexity?

There has been much interest in recent years in these questions. While there has been some progress on answering these questions for specific measures [ABK⁺06, ILO20, All01, AH19, OS17, HS17, GII⁺19, Hir18, Ila20a, Ila20b], there are also barriers to understanding these questions better, such as our inability to prove circuit lower bounds [KC00, MW17] and the magnification phenomenon [OS18, OPS21, MMW19, CJW19]. Many of the above questions such as the NP-completeness of MCSP remain wide open.

1.1.2 Cryptography

A fundamental question in cryptography is whether one-way functions exist. We have been quite successful at basing one-way functions on the hardness of specific problems such as factoring [RSA78], discrete logarithm [DH76], and some lattice problems [Ajt96]. One problem with this approach, however, is that we have little complexity-theoretic evidence for the hardness of these problems (for example, they are unlikely to be NP-hard). The most compelling evidence for their hardness so far is simply that *we have not been able to find efficient algorithms for them*.

Can we base the existence of one-way functions on firm complexity-theoretic assumptions? A “holy grail” in this direction would be to construct one-way functions assuming (only) $NP \not\subseteq BPP$ [DH76]. This goal remains elusive, and there are several obstacles to its resolution:

- Unless PH collapses, non-adaptive “black-box” reductions cannot transform worst-case hardness of NP into average-case hardness of NP [BT06]. As the latter is necessary for one-way functions, this barrier result demonstrates limits of such “black-box” reductions on basing one-way functions from worst-case assumptions such as $\text{NP} \not\subseteq \text{BPP}$. For the task of constructing one-way functions (instead of just a hard-on-average problem in NP), stronger barrier results are known [AGGM06, Nan21].
- Even the seemingly easier task of basing one-way functions from *average-case* hardness of NP remains elusive. Indeed, Impagliazzo [Imp95b] called a world “Pessiland” where NP is hard on average but one-way functions do not exist. It is not hard to construct a relativized Pessiland [Wee06], therefore a relativization barrier exists even for this “easier” task.

1.1.3 The Liu–Pass Result

Very recently, in a breakthrough result, Liu and Pass [LP20] showed an *equivalence* between the existence of one-way functions and the *bounded-error average-case hardness* of computing the K^t complexity (the Kolmogorov complexity of a string with respect to a given polynomial time bound t) over the uniform distribution. This result is significant for several reasons.

- From the perspective of cryptography, it establishes the first equivalence between the existence of one-way functions and the average-case complexity of a natural problem over a natural distribution. Such an *equivalence* result bases cryptography on firmer complexity-theoretic foundations.
- From the perspective of meta-complexity, it enables *robustness* results for the complexity of K^t in the average-case setting. Indeed, [LP20] proved that *approximating* the K^t complexity of a string or *finding* an optimal description for a string are both equivalent to the problem of computing the K^t complexity.
- More generally, such connections suggest the possibility of new and non-trivial *average-case* reductions between natural problems on natural distributions, which is by itself an important goal in average-case complexity theory. Several of the most basic questions in this area remain open: Is random 3-SAT as hard as random 4-SAT (or vice versa)? Is the decision version of Planted Clique as hard as its search version?¹

Given these motivations, it is natural to ask if the main result of [LP20] can be extended to other meta-complexity problems. For example, is the average-case hardness of MCSP also equivalent to the existence of one-way functions? There is a “Kolmogorov-version” of circuit complexity, named KT, which is more “fine-grained” than circuit complexity [All01, ABK⁺06]. Maybe this problem is also closely related to the existence of one-way functions? What about Levin’s K_t complexity [Lev84]?²

1.2 Our Contributions

We give strong positive answers to the above questions. We show somewhat surprisingly that the average-case hardness of KT complexity is also equivalent to a natural cryptographic primitive, namely the existence of one-way functions computable in *fast parallel time*.³ For MCSP, we

¹The decision version of Planted Clique is to distinguish Erdős–Rényi random graphs from graphs with a planted clique; the search version is to find the planted clique. Three years after the conference version of our paper (CCC’21), Hirahara and Shimizu [HS24] showed that the two versions are equivalent.

²See Definition 3.3 for the precise definitions of K^t , KT, and K_t .

³Due to a result in [AIK06], the “fast parallel time” here can be interpreted as either NC^0 or NC^1 . We also point the reader to Benny Applebaum’s book *Cryptography in Constant Parallel Time* [App14], which inspired the title of the current paper.

obtain weaker results: exponential hardness of computing circuit size over the uniform distribution implies the existence of one-way functions, and there is a partial converse. Bounded-error average-case complexity of K^t complexity turns out to be equivalent to the existence of one-way functions in one natural setting of parameters (despite the fact that computing K^t in the worst case is EXP-hard [ABK⁺06]), and equivalent to the existence of complexity-theoretic pseudo-random generators in another natural setting of parameters.

We also extend the connection between the hardness of K^t complexity and one-way functions to the high end of the parametric regime — this yields one-way functions of almost optimal hardness from plausible assumptions about the hardness of K^t complexity. We define and motivate the Perebor Hypotheses⁴, which are average-case analogues of the Exponential-Time Hypothesis and its variants for meta-complexity problems, stating that there is no better way to solve meta-complexity problems than brute force search. This is a conceptual contribution of this work, and we expect these hypotheses to have further applications to cryptography, average-case complexity, and fine-grained complexity.

We now describe our results in more detail.

1.2.1 Connections between Meta-Complexity and One-Way Functions

Our main result is an equivalence between “parallel cryptography” and the average-case hardness of MKTP:

Theorem 1.1 (Main Result; Informal). *There is a one-way function computable in uniform NC^1 if and only if KT is bounded-error hard on average.*

The class “uniform NC^1 ” in the above theorem is somewhat arbitrary and can be replaced by any class between NC^0 and $\oplus\text{L}$.⁵ This is because [AIK06] proved that the existence of one-way functions in $\oplus\text{L}$ implies the existence of one-way functions in NC^0 .

For comparison, Liu and Pass [LP20] showed an equivalence between (“sequential”) cryptography and the average-case hardness of time-bounded Kolmogorov complexity (K^t).

Theorem 1.2 (Main Result of [LP20]). *There is a one-way function if and only if for some polynomial t , K^t is bounded-error hard on average.*

Theorem 1.2 shows that the one-way function defined based on hardness of K^t is a natural *universal* one-way function.⁶ Similarly, Theorem 1.1 shows that the one-way function we define based on the hardness of KT is a natural universal one-way function in NC^1 .

As a corollary, the classical open question of whether polynomial-time computable one-way functions imply one-way functions in NC^0 is equivalent to the question of whether average-case hardness of K^t implies average-case hardness of KT .

Results for MCSP. The KT complexity was defined as a variant of Kolmogorov-complexity that resembles circuit complexity [All01, ABK⁺06]. Therefore, it is natural to ask whether our equivalence also holds for circuit complexity.

It turns out that circuit complexity is less convenient to deal with. Nevertheless, we still proved non-trivial analogues of Theorem 1.1 as follows:

Theorem 1.3 (Informal). *The following are true:*

⁴Our terminology is inspired by Trakhtenbrot’s survey [Tra84] on work in the former Soviet Union aiming to show that various meta-complexity problems require brute force search to solve. “Perebor” roughly means “by exhaustive search” in Russian.

⁵ $\oplus\text{L}$ is the class of problems solvable by a *parity* Turing machine with $O(\log n)$ space. This class contains both NC^1 and L (log-space).

⁶A *universal one-way function* is a polynomial-time computable function f such that if any one-way function exists, then f is a one-way function. An artificial universal one-way function can be defined by enumerating uniform algorithms and concatenating their outputs [Lev03, Gol01].

- If MCSP is exponentially⁷ hard on average, then there is a (super-polynomially hard) one-way function.
- If there is an exponentially hard weak one-way function in NC^0 , then MCSP is (exponentially) hard on average.

For the technical difficulties of handling circuit complexity, the reader is referred to Section 6 (and in particular Remark 6.9).

Results for MKtP. We also observe that the existence of (polynomial-time computable) one-way functions can be characterized by the bounded-error average-case complexity of Kt.

Theorem 1.4. *There is a one-way function if and only if Kt is bounded-error hard on average.*

This result may seem surprising as computing Kt is EXP-hard under polynomial-size reductions [ABK⁺06]. This is true even for any oracle that is a *zero-error* heuristic for computing Kt. In contrast, we show that the *bounded-error* average-case complexity of Kt is captured by one-way functions, a notion that seems much “easier” than EXP.

The harder direction in Theorem 1.4 is to construct a one-way function from the hardness of Kt. How could we construct a one-way function from merely a hard problem *in exponential time*? The crucial insight is as follows: For *most* strings $x \in \{0, 1\}^n$ whose optimal Kt complexity is witnessed by a machine d and a time bound t where $\text{Kt}(x) = |d| + \log t$, we have $t \leq \text{poly}(n)$. We refer the reader to Section 2.1.2 and Section 7 for more details.

Note that Theorem 1.4 can also be seen as a characterization of cryptographic pseudorandomness, by the known equivalence between one-way functions and cryptographic pseudorandomness [HILL99]. In a different regime of parameters, average-case hardness of Kt turns out to capture the existence of *complexity-theoretic* pseudorandom generators, which are pseudorandom generators with non-trivial seed length computable in *exponential* time. Thus the average-case complexity of a single problem (Kt) can be used to capture both cryptographic pseudorandomness and complexity-theoretic pseudorandomness!

The result below is essentially a corollary of known results from [ABK⁺06, NW94].

Theorem 1.5 (Informal). *For each $\epsilon > 0$, there is a pseudo-random generator from n^ϵ bits to n bits computable in time $2^{n^\epsilon} \text{poly}(n)$ secure against $\text{poly}(n)$ size circuits iff for each $c > 1/2$ there are no polynomial-size circuits solving Kt on more than $1 - 2^{-cn}$ fraction of inputs of length n .*

1.2.2 Application in Meta-Complexity: Robustness Theorems

We exploit the connection between MKTP and parallel cryptography to establish more robustness results for meta-complexity. It is known that parallel cryptography is *extremely* robust: L-computable one-way functions exist, if and only if NC^1 -computable one-way functions exist, if and only if NC^0 -computable one-way functions exist [AIK06]. We define L- and NC^1 -variants of K^t complexity, and translate the result in [AIK06] to the following robustness theorem:

Theorem 1.6 (Bounded-Error Robustness of Meta-Complexity; Informal). *The following statements are equivalent:*

- For $t_1(n) := n^{10}$, the search version of $\text{NC}^1\text{-K}^{t_1}$ is bounded-error hard on average.
- For $t_2(n) := 5n$, L- K^{t_2} is bounded-error hard on average to approximate, within an additive error of $100 \log n$.
- KT is bounded-error hard on average.

⁷In this paper, *exponential* hardness always means hardness $2^{\Omega(n)}$.

It turns out that the above theorem can be interpreted as a *reduction*! In particular, we discover the following average-case reduction from L-K^t to MKTP:

Theorem 1.7 (Informal). *Let n, t be parameters, $m := \text{poly}(n, t)$. There is a randomized reduction $\text{Red}(x)$ running in $\text{poly}(m)$ time that maps a length- n input to a length- m input, and satisfies the following property:*

- *Given a uniform random input x of length n , $\text{Red}(x)$ produces a uniform random string of length m .*
- *Given a string x such that $\text{L-K}^t(x)$ is small, for every possible randomness used in Red , the KT complexity of $\text{Red}(x)$ is also small.*

To the best of our knowledge, this is the first reduction from a variant of K^t complexity to a variant of KT complexity. The only special property of L that we use is that L -computable functions have *perfect randomized encodings* [AIK06]. If polynomial-time computable functions have such perfect randomized encodings, then our techniques imply an average-case reduction from the (standard) K^t complexity to the KT complexity.

We have focused on the *bounded-error* average-case complexity of meta-complexity problems so far. However, Theorem 1.7 also implies robustness in the *zero-error* regime. Here, let $\text{MKTP}[s]$ be the problem of determining whether the input x satisfies $\text{KT}(x) \leq s(|x|)$, and let $\text{MINK}^t[s]$ be the problem of determining whether the input x satisfies $\text{K}^t(x) \leq s(|x|)$.

Theorem 1.8 (Zero-Error Robustness of Meta-Complexity; Informal). *Let $t_1(n) := n^{10}$ and $t_2(n) := 5n$. Among the following items, we have (1) $\iff$ (2), and both items are implied by (3).*

1. *There is a constant $c > 0$ such that $\text{NC}^1\text{-MINK}^{t_1}[n - c \log n]$ is zero-error easy on average.*
2. *There is a constant $c > 0$ such that $\text{L-MINK}^{t_2}[n - c \log n]$ is zero-error easy on average.*
3. *There is a constant $c > 0$ such that $\text{MKTP}[n - c \log n]$ is zero-error easy on average.*

1.2.3 Application in Cryptography: Maximally Hard One-Way Functions

How hard can a one-way function be? The standard definition of one-way functions only requires that no polynomial-time adversary inverts a random output except with negligible probability. However, it is conceivable that some one-way function requires $2^n/\text{poly}(n)$ time to invert (say, on a constant fraction of inputs)!

When studying the fine-grained complexity of inverting one-way functions, it is useful to draw an analogy to the theory of Satisfiability, where the Exponential Time Hypothesis (ETH) and Strong Exponential Time Hypothesis (SETH) [IP01, IPZ01, CIP09] have gained prominence recently. ETH states that CNF-SAT requires time $2^{\Omega(n)}$ to solve, and SETH states that CNF-SAT requires time $2^{n-o(n)}$ to solve. Analogously, we define the *Weak* and *Strong* Perebor Hypotheses for any given complexity measure μ . The Weak Perebor Hypothesis for μ states that μ requires time $2^{\Omega(n)}$ to compute on average, and the Strong Perebor Hypothesis for μ states that μ requires time $2^{n-o(n)}$ to compute on average. Note that the Strong Perebor Hypothesis for μ essentially states that brute-force search is close to optimal for computing μ on average. While the Perebor Hypotheses refer by default to hardness against uniform algorithms, we can define non-uniform analogues in a natural way. (Unfortunately, due to the lack of a uniform hardcore lemma with good enough parameters, the results here only hold for the non-uniform Perebor Hypotheses.)

We suggest that the Perebor Hypotheses might find similar applicability to ETH and SETH in the theory of fine-grained complexity.

In this work, we tighten the connection between *weak* one-way functions (for which it is hard to invert a random instance w.p. $1 - 1/\text{poly}(n)$) and the hardness of K^t complexity. We managed to show a very tight result:

Theorem 1.9 (Informal). *For every constant $\alpha > 0$, there exists a weak one-way function with non-uniform hardness $2^{(1-o(1))\alpha n}$ if and only if K^t complexity is hard on average for non-uniform algorithms of size $2^{(1-o(1))\alpha n}$.*

Note that the two α 's in the exponent $(1 - o(1))\alpha n$ are the *same*. That is, we essentially construct the best (weak) one-way functions from the hardness of K^t complexity. In particular, almost maximally hard one-way functions exist if the (non-uniform version of the) Strong Pseudorandomness Hypothesis holds for K^t .

We also attempted to strengthen the relationship between one-way functions in NC^0 and the hardness of KT complexity. Our result is that exponentially-hard weak one-way functions in NC^0 imply exponential hardness of KT.

Theorem 1.10 (Informal). *If there is a weak one-way function in NC^0 with non-uniform hardness $2^{\Omega(n)}$, then KT requires $2^{\Omega(n)}$ circuit size to solve on average.*

In other words, the existence of an exponentially hard weak one-way function in NC^0 implies the non-uniform version of the Weak Pseudorandomness Hypothesis for KT.

1.3 Related Work

There have been several previous works connecting meta-complexity to cryptography. Impagliazzo and Levin [IL90] show that the existence of one-way functions is equivalent to the hardness of a certain learning task related to time-bounded Kolmogorov complexity, and follow-up results in this direction are shown by Blum, Furst, Kearns, and Lipton [BFKL93]. Oliveira and Santhanam [OS17] show a dichotomy between learnability and cryptographic pseudorandomness in the non-uniform setting: there is a non-trivial non-uniform learner for polynomial-size Boolean circuits iff there is no exponentially secure distribution on functions computable by polynomial-size circuits. Santhanam [San20] proves an equivalence between the existence of one-way functions and the non-existence of natural proofs under a certain universality assumption about succinct pseudorandom distributions. We note here that the non-existence of natural proofs is equivalent to the zero-error average-case hardness of MCSP [HS17].

None of the above results gives an unconditional equivalence between the average-case hardness of a natural decision problem over a natural distribution and the existence of one-way functions. This was finally achieved by Liu and Pass [LP20], who showed that the weak hardness of K^{poly} over the uniform distribution is equivalent to the existence of one-way functions. [LP20] leaves open whether there are similar connections between one-way functions and the hardness of other meta-complexity problems such as KT and MCSP over the uniform distribution. In this work, we show such connections to *parallel* cryptography, i.e., to the existence of one-way functions in NC^1 , which by [AIK06] is equivalent to the existence of one-way functions in NC^0 .

There is an extensive literature on parallel cryptography, beginning with the work of [AIK06]. We refer to [App14] and [App16] for further information.

Our work also relates to average-case meta-complexity, which was studied in [RR97, HS17]. [HS17] essentially observe that the identity reduction trivially reduces μ to μ' over the uniform distribution in a zero-error average-case sense, where μ and μ' are any two meta-complexity measures such that $\mu'(x) \leq \mu(x) \leq |x| + O(\log(|x|))$ for all x . In this work (particularly Sections 4.3 and 4.4), we give several *non-trivial* examples of zero-error and bounded-error average-case reductions between meta-complexity problems.

Concurrent works of [LP21] and [ACM⁺21]. We now discuss the relationship of our work with the concurrent works of [LP21] and [ACM⁺21], which overlap in some respects with ours.

Liu and Pass [LP21] show an equivalence between the bounded-error weak average-case hardness of K^t over the uniform distribution and the existence of one-way functions — this

is essentially the same as our Theorem 1.4. They also show that the *zero-error* average-case hardness of K^t over the uniform distribution is equivalent to $\text{EXP} \neq \text{BPP}$. In contrast, our Theorem 1.5 gives an equivalence between the bounded-error average-case hardness of K^t over the uniform distribution in a different parametric regime and the worst-case hardness of EXP , where the hardness in each case is with respect to non-uniform adversaries. The somewhat surprising message of both sets of results is the same: a minor variation on an average-case complexity assumption that is equivalent to the worst-case hardness of EXP implies the existence of one-way functions.

[LP21] also give characterizations of parallel cryptography but they do this using space-bounded Kolmogorov complexity and the conditional version thereof. Their work does not contain any results relating to the hardness of K^t or MCSP.

Allender, Cheraghchi, Myrasiotis, Tirumala, and Volkovich [ACM⁺21] relate the average-case hardness of the conditional version of K^t complexity (McKTP) over the uniform distribution to the existence of one-way functions. They show that if McKTP is hard on a polynomial fraction of instances then one-way functions exist. They also give a weak converse: if one-way functions exist, then McKTP is hard on an exponential fraction of instances. Finally, using our techniques, they show that the average-case hardness of McKTP characterizes parallel cryptography as well.

Further work on the Perebor Hypotheses. There are several works on the Perebor Hypotheses after our paper becomes available online. Carmosino, Hoover, Impagliazzo, Kabanets, and Kolokolova [CHI⁺21] demonstrated barriers against obtaining “natural” reductions between different variants of AC^0 -MCSP, under a weak form of Perebor Hypothesis for AC^0 -MCSP. More recently, Mazon and Pass [MP24], and independently Hirahara, Ilango, and Williams [HIW24], disproved the non-uniform version of the Strong Perebor Hypothesis for K^{poly} (Hypothesis 5.19), thereby confirming the speculations we raised in Section 5.5.

1.4 Organization

Section 2 presents some of our main ideas and techniques. Section 3 provides basic definitions and preliminaries.

The equivalence between the existence of NC^0 -computable one-way functions and the hardness of K^t complexity is proved in Section 4. We prove our robustness results in Section 4.3 and 4.4. In Section 5, we present the tight connection between the hardness of K^t complexity and maximally hard one-way functions. To motivate future study, we put forward a few Perebor Hypotheses in Section 5.5, which are closely related to the existence of maximally-hard one-way functions. The results related to MCSP are proved in Section 6, and the results related to MKtP are proved in Section 7. Finally, we leave a few open questions in Section 8.

2 Intuitions and Techniques

For strings $s_1, s_2, \dots, s_n$, we use $s_1 \circ s_2 \circ \dots \circ s_n$ to denote their concatenation.

2.1 Parallel Cryptography and the Hardness of K^t

Our proof of Theorem 1.1 builds on [LP20]. However, it turns out that we need new ideas for both directions of the equivalence.

2.1.1 Hardness of K^t from One-Way Functions in NC^0

We first review how Liu and Pass [LP20] proved that one-way functions imply average-case hardness of K^t .

Any cryptographically-secure PRG G implies *zero-error* hardness of K^t [RR97, KC00, ABK⁺06]. Roughly speaking, the outputs of G have “non-trivial” K^t complexity, but random strings are likely to have “trivial” K^t complexity.⁸ If there is a polynomial-time (zero-error) heuristic for K^t , this heuristic will recognize most random strings as “trivial”, but recognize every output of G as “non-trivial”. Thus, we can use it as a distinguisher for G , contradicting the security of G .

It is crucial in the above argument that our heuristic does not make mistakes. If the outputs of G are “sparse” and our heuristic has two-sided error, our heuristic could also recognize the outputs of G as “non-trivial”. (Here, a PRG G with output length n is sparse if the number of possible outputs of G is significantly smaller than 2^n .) In this case, the heuristic may still be correct on most length- n strings but fail to distinguish the outputs of G from true random strings.

Why not *make G dense*? This is the core idea of Liu and Pass. In particular, from an arbitrary one-way function f , they constructed a *dense* PRG G ,⁹ and used G to argue that K^t is bounded-error average-case hard. Roughly speaking, if the outputs of G occupy a $1/\text{poly}(n)$ fraction of $\{0,1\}^n$, then any bounded-error heuristic for K^t with error probability $1/n^{\omega(1)}$ is a distinguisher for G . It follows from the security of G that K^t is bounded-error hard on average.

What about KT? Recall that the KT complexity of a string x is the minimum of $|d| + t$ over programs d and integers t such that x can be generated *implicitly* from d in at most t steps, i.e., the universal machine computes the i -th bit x_i of x correctly in at most t steps with oracle access to d .

When we use the above framework to analyze the hardness of KT complexity, there is a problem: the outputs of G might have “trivial” KT complexity. Indeed, if G is a *dense* PRG, then it has at least $2^n/\text{poly}(n)$ many different outputs and hence its seed length is at least $n - O(\log n)$. Furthermore, if G runs in $t(n)$ time, then we can only upper bound the KT complexity of its outputs by $n - O(\log n) + t(n)$. For most PRGs G , $t(n)$ would be a large polynomial, hence this upper bound is even worse than the trivial upper bound (which is $n + O(\log n)$). However, if G is computable in *constant parallel time*¹⁰, then our upper bound on the KT complexity is still non-trivial. In other words, a dense PRG computable in NC^0 still suffices for the bounded-error average-case hardness of KT.

We can construct an NC^0 -computable PRG from NC^0 -computable one-way functions as follows.¹¹ We first use [LP20] to construct a dense PRG G . This PRG is not necessarily in NC^0 , as [LP20] needs some more complex primitives (e.g. extractors). Nevertheless, we can apply the randomized encodings in [AIK06] to compile G into a PRG in NC^0 .

2.1.2 One-Way Functions in NC^0 from the Hardness of KT

It is straightforward to construct a one-way function from the hardness of KT using techniques of [LP20, Section 4]. Roughly speaking, the one-way function f receives two inputs d, t , where d is the description of a machine and t is a time bound. Let x be the string such that for each $i \in [n]$, x_i is equal to the output bit of $d(i)$ for t steps. We define $f(d, t) := (|d| + t, x)$. An inverter, on input (ℓ, x) , is required to find a description of x with complexity at most ℓ ,

⁸Here, the K^t (or KT) complexity of a length- n string is “non-trivial”, if it is at most $n - \Omega(\log n)$. Most length- n strings have complexity at least $n - \Omega(\log n)$; every length- n string has complexity at most $n + O(\log n)$ (justifying the word “trivial”).

⁹The input distribution of their PRG is not the uniform distribution, which is different from standard PRGs; see Definition 3.13. We ignore this difference in the informal exposition.

¹⁰Due to low-level issues in the computational models, the “constant time” in [App14] actually corresponds to $O(\log n)$ time in this paper. See Section 3.1 for details.

¹¹Note that this is different from [HILL99, HRV13]. The PRG we construct is dense, but its input distribution is not uniform. In contrast, [HILL99, HRV13] constructs a PRG (on uniformly random inputs) from an arbitrary one-way function, but the PRG is not necessarily dense.

thus it needs to solve MKTP. (All one-way functions in this section are *weak*, meaning that no adversary can efficiently invert them on a $1 - 1/\text{poly}(n)$ fraction of inputs.)

There is one problem: f is not in NC^0 . By [AIK06], it suffices to construct a one-way function in $\oplus\text{L}$, but f is also not in $\oplus\text{L}$ (unless $\oplus\text{L} = \text{P}$).

Our idea is to only consider *typical* inputs and throw away the atypical ones. In particular, for most strings x , the values of t in the optimal description of $\text{KT}(x) = |d| + t$ are small. (We have $t = O(\log n)$ for every string x with Kolmogorov complexity at least $n - O(\log n)$.) We call an input *typical* if its value of t is at most $O(\log n)$. If KT is (bounded-error) hard on average, then it is also hard on average conditioned on the input being typical.

Therefore, we place the restriction that $t \leq c \log n$ in our one-way function f , where c is a constant depending on the hardness of KT . We can still base the hardness of f on the hardness of KT . More importantly, f is computable in space complexity $O(c \log n)$ and we obtain a one-way function in NC^0 by [AIK06].

2.2 Applebaum–Ishai–Kushilevitz as a Reduction

For any “reasonable” circuit class $\mathcal{C}$, we can use [LP20] to show that the existence of one-way functions computable in $\mathcal{C}$ is equivalent to the hardness of $\mathcal{C}\text{-K}^t$. (The precise definition of $\mathcal{C}\text{-K}^t$ is beyond the scope of this paper, but $\text{NC}^1\text{-K}^t$ and L-K^t are defined in Definition 3.4.) Now, let us review the main results of [AIK06]: $\oplus\text{L}$ -computable one-way functions exist if and only if NC^0 -computable one-way functions exist. In other words, $\oplus\text{L-K}^t$ is hard on average if and only if $\text{NC}^0\text{-K}^t$ is hard on average!¹²

It is natural to ask whether there is a reduction between $\oplus\text{L-K}^t$ and $\text{NC}^0\text{-K}^t$. It turns out that the answer is *yes*! In this section, we describe this reduction without using the language of one-way functions. This reduction is randomized, reduces any string with non-trivial $\oplus\text{L-K}^t$ complexity to a string with non-trivial $\text{NC}^0\text{-K}^t$ complexity, and reduces a random string to a random string. Although it may not be a worst-case reduction, it establishes non-trivial equivalence results between average-case complexities of $\oplus\text{L-K}^t$ and $\text{NC}^0\text{-K}^t$.

The property that enables our reduction is *resamplability* [FSUV13]. For now, think of “easy” as being NC^0 -computable and “hard” as otherwise. A hard function f is *resamplable* if given an input x and random coins r , there is an easy procedure (the “resampler”) that produces a uniformly random input of f whose answer is the same as x .

Example 2.1. The parity function $\text{PARITY}(x) = x_1 \oplus x_2 \oplus \dots \oplus x_n$ is hard (i.e., not computable in NC^0). Given n input bits $x_1, x_2, \dots, x_n$ and $n-1$ random bits $r_1, r_2, \dots, r_{n-1}$, we can produce a uniformly random input whose answer is the same as x , as follows:

$$(x_1 \oplus r_1, x_2 \oplus r_1 \oplus r_2, x_3 \oplus r_2 \oplus r_3, x_4 \oplus r_3 \oplus r_4, \dots, x_{n-1} \oplus r_{n-2} \oplus r_{n-1}, x_n \oplus r_{n-1}).$$

Note that the resampler is easy (i.e., in NC^0), thus parity is resamplable.

The reduction. We will use a $\oplus\text{L}$ -complete problem named DCMD that is resamplable (see Section 3.7). Our reduction is very simple: given an input $x \in \{0, 1\}^n$, we choose a large enough $N = \text{poly}(n)$ and replace every bit x_i by a random length- N instance of DCMD whose answer is x_i . Our reduction outputs the concatenation of these n instances.

Since DCMD is balanced (i.e., the number of 0-instances and 1-instances are the same), our reduction maps a random instance to a random instance.

Now assume that $\oplus\text{L-K}^t(x) = n - \gamma$ is non-trivial, and d is a $\oplus\text{L}$ machine of description length $n - \gamma$ that “computes” x . Since DCMD is $\oplus\text{L}$ -complete (under NC^0 -reductions), for each i , the computation of x_i can be reduced to a DCMD-instance s_i of length N such that $\text{DCMD}(s_i) = x_i$. Moreover, given the description d , we can produce $s_1 \circ s_2 \circ \dots \circ s_n$ in NC^0 .

¹² NC^0 may not be “reasonable” in the above sense, but the reduction we present in this section is correct.

We use the *resamplability* of DCMD. The resampler for DCMD only uses $N - 1$ random bits (which is optimal). Consider the following NC^0 circuit. It receives d and $r_1, r_2, \dots, r_n$ as inputs where each r_i is a random string of length $N - 1$. It computes $s_1, s_2, \dots, s_n$ from d , and for each i , feeds s_i and r_i to the resampler to obtain a uniformly random DCMD instance whose answer is the same as s_i . When r_i are random bits, the output distribution of this NC^0 circuit is identical to the distribution of $\text{NC}^0\text{-K}^t$ instances we reduced x to. Moreover, the $\text{NC}^0\text{-K}^t$ complexity of *every* string in this distribution is at most $(n - \gamma) + (N - 1)n + O(\log n) = Nn - \gamma + O(\log n)$, which is non-trivial.¹³

As a consequence, we also obtain an (average-case) reduction from $\oplus\text{L-K}^t$ to KT .

2.3 Tighter Connections

To obtain a tight relationship between the hardness of K^t and the hardness of weak one-way functions, we optimize the construction from one-way functions to PRGs in [LP20]. Suppose that given a one-way function f with input length n , we could construct a PRG with output length m' . Then solving K^t on length m' is (roughly) as hard as inverting f on length n . Therefore, we need m' to be as close to n as possible. As the PRG is dense, its output length m' is close to its input length m , thus we only need m to be close to n .

It turns out that the input of the PRG consists of the input of f and the seeds of a few pseudorandom objects.

- One object is an *extractor* $\text{Ext}(\mathcal{X}, r)$ [NZ93, Nis96], which given a “somewhat random” distribution $\mathcal{X}$ and a truly random seed r , outputs a distribution that is statistically close to the uniform distribution.

We use the near-optimal explicit extractors with $O(\log^2 n)$ seed length [GUV09].

- Another object is a *hardcore function* $\text{HC}(x, r)$ [GL89]. Let f be a one-way function, x be a random input, and r be a random seed. Given $f(x) \circ r$, it should be infeasible to distinguish between $\text{HC}(x, r)$ and a uniformly random string. Note that $\text{HC}(x, r)$ needs to have multiple output bits; in contrast, a *hardcore predicate* (also defined in [GL89]) only has one output bit.

We use the observation, implicit in [Tre01, TZ04], that any seed-extending “black-box” pseudorandom generator is a good hardcore function. We use the *direct product* generator [Hir20b, Hir20a] as our hardcore function, which has $O(\log^2 n)$ seed length, and very small “advice complexity.” The advice complexity turns out to be related to the overhead of our reduction.

There is another problem: [LP20] needs a *strong* one-way function to start with, but we only have a *weak* one-way function. (A strong one-way function is infeasible to invert on *almost every* input, but a weak one-way function is only infeasible to invert on a *non-trivial fraction* of inputs.) Yao [Yao82] showed how to “amplify” a weak one-way function to a strong one-way function, but the overhead of this procedure is too large. In particular, Yao’s hardness amplification does not preserve *exponential* hardness, and it is open whether exponentially-hard weak one-way functions imply exponentially-hard strong one-way functions.

Our idea is to use *Impagliazzo’s hardcore lemma* [Imp95a] instead. The hardcore lemma states that for any weak one-way function f , there is a “hardcore” distribution on which f becomes a strong one-way function. We (and [LP20]; see Footnote 9) allow the input distribution of our PRG to be *arbitrary*, as long as the output distribution is pseudorandom. Such “PRGs” still imply hardness of K^t . The hardcore lemma only has a small complexity overhead, which allows us to prove tight results.

¹³The additive factor here is $O(\log n)$ since in our computational model, each memory access requires $\Theta(\log n)$ time. See Section 3.1 for details.

Now, from a weak one-way function of input length n , we can construct a PRG with output length $n + O(\log^2 n)$. This construction allows us to transform the hardness of one-way functions to the hardness of K^t at almost no cost.

Tighter connections between MKTP and one-way functions in NC^0 . Here, the problem becomes to construct NC^0 -computable PRGs from NC^0 -computable one-way functions. We use a construction of universal hash functions in NC^0 with linear seed length by Applebaum [App17]. Such hash functions are both good extractors (by the leftover hash lemma) and good hardcore functions (proved in [BIO14, HMS04]). As the hash functions require linear seed length, from an NC^0 -computable one-way function with input length n , we obtain an NC^0 -computable PRG with output length $O(n)$. It follows that if the one-way function is hard against $2^{\Omega(n)}$ -size adversaries, then MKTP is also hard against $2^{\Omega(n)}$ -size algorithms.

2.4 MCSP-Related Results

One-way functions from hardness of MCSP. We use the straightforward construction: our one-way function receives a circuit C and outputs $|C|$ and $tt(C)$, where $|C|$ is the size of C and $tt(C)$ is the truth table of C . The inverter, on input (s, tt) , is required to find a size- s circuit whose truth table is tt , thus needs to solve MCSP.

One problem with this construction is that if we sample a uniform circuit (according to some distribution), the induced distribution over truth tables may not be uniform. In the case of K^t (and KT), we can show that for every string of length n , its optimal description is sampled (in the one-way function experiment) w.p. at least $2^{-n}/\text{poly}(n)$, therefore we can “transfer” the hardness of K^t over a random truth table to the hardness of inverting the one-way function over a random description. Using the best bounds on the maximum circuit complexity of n -bit Boolean functions [FM05], we can still prove that for every truth table of length N , its optimal circuit is sampled w.p. at least $2^{-N}/2^\eta$, where $\eta < o(N)$. This means that starting from *exponential* hardness of MCSP, we can still obtain non-trivial one-way functions.

We conjecture that hardness of MCSP actually implies one-way functions in NC^0 ; see Remark 6.9 for details.

Hardness of MCSP from one-way functions in NC^0 . To argue about the hardness of MCSP, we need a PRG whose outputs have non-trivial circuit complexity. As before, we use the hash functions in [App17] to construct an exponentially-hard PRG. We would like to argue that all outputs of the PRG have non-trivial circuit complexity. In order to do this, we use the mass production theorem of Uhlig [Uhl74, Uhl84] to generate a circuit of size $(1 + o(1))2^n/n$ that evaluates a given function on *multiple* inputs. (If our PRG has locality d , i.e., each output bit depends on d input bits, then we need a size- $(1 + o(1))2^n/n$ circuit that evaluates d inputs in parallel.) However, Uhlig’s theorem only gives us non-trivial circuit size if our PRG has *linear* stretch, i.e., stretch ϵn for some constant $\epsilon > 0$. This is why we need the hardness of the one-way function in our assumption to be at least $\text{poly}(2^{\epsilon n})$.

2.5 Using Hardness of K_t to Capture Cryptographic and Complexity-Theoretic Pseudorandomness

To show Theorem 1.4, we use ideas similar to those in Section 2.1.2. Suppose we try to define a one-way function by computing the string corresponding to an optimal description with respect to K_t complexity. An obvious issue is that such strings might require exponential time to compute, while the one-way function needs to be evaluated efficiently. However, we observe that *typical* inputs only require polynomial time to generate from their optimal descriptions. Here, the typical inputs are those with Kolmogorov complexity $n - O(\log n)$. In their optimal descriptions $K_t(x) = |d| + \log t$, we have $t \leq \text{poly}(n)$. Our one-way function receives two inputs

d, t , where d is the description of a machine, and $t \leq \text{poly}(n)$ is a time bound. We simply simulate the machine d for t steps and output what it outputs. The proof that this gives a one-way function is closely analogous to the proof of the reverse implication in Theorem 1.1. The proof that one-way functions imply the average-case hardness of Kt complexity mimics the proof of the corresponding implication in Theorem 1.2, since the outputs of a cryptographic PRG with stretch $\lambda \log n$ have non-trivial Kt complexity when λ is large enough compared to the time required to compute the PRG.

To show Theorem 1.5, we use the Nisan–Wigderson generator [NW94] in a way similar to how it is used by [ABK⁺06] to show that Kt is complete for exponential time under polynomial-size reductions. The interesting direction is to show that the Nisan–Wigderson generator implies the average-case hardness of Kt for the range of parameters in the statement of Theorem 1.5. We use the fact that the Nisan–Wigderson generator can be made seed-extending without loss of generality. We truncate the output of the generator so that the stretch is $(1 + \epsilon)n$ for some small $\epsilon > 0$ — this implies that the outputs of the generator on all seeds have non-trivial Kt complexity. Since the generator is seed-extending, the output has high entropy, hence a strong enough average-case algorithm for Kt can distinguish random strings (which have trivial Kt complexity) from the outputs of the PRG. Here we take advantage of the stretch being *small* rather than *large*: this gives us better parameters for our average-case hardness result.

3 Preliminaries

We use $\mathcal{U}_n$ to denote the uniform distribution over length- n binary strings. For a distribution $\mathcal{D}$, we use $\mathbf{x} \leftarrow \mathcal{D}$ to denote that $\mathbf{x}$ is a random variable drawn from $\mathcal{D}$. A function $\text{negl} : \mathbb{N} \rightarrow [0, 1]$ is *negligible* if for every constant c , $\text{negl}(n) \leq 1/n^c$ for large enough integers n .

Let $D : \{0, 1\}^n \rightarrow \{0, 1\}$ be a function and X, Y be two random variables over $\{0, 1\}^n$. For $\epsilon > 0$, we say D ϵ -*distinguishes* X from Y if

$$|\Pr[D(X) = 1] - \Pr[D(Y) = 1]| \geq \epsilon.$$

Otherwise we say X and Y are ϵ -*indistinguishable* by D .

We often consider ensembles of functions in this paper. For example, a function $f : \{0, 1\}^* \rightarrow \{0, 1\}^*$ can be interpreted as an ensemble $f = \{f_n : \{0, 1\}^n \rightarrow \{0, 1\}^*\}$; we call each f_n the n -th *slice* of f . Similarly, we also consider ensemble of distributions $\mathcal{D} = \{\mathcal{D}_n\}$ as input distributions for a function f where each $\mathcal{D}_n$ is a distribution over $\{0, 1\}^n$.

3.1 Computational Model and Uniformity

We need a computational model with random access to inputs. We consider a Turing machine that accesses the length- n input x via an “address” tape and a length- $O(1)$ “answer” tape. Whenever the machine enters a particular “address” state, let i be the binary number written in the address tape. After one step, the content of the answer tape becomes x_i , and the address tape is cleared. (In other words, the Turing machine treats x as the truth table of an *oracle*.)

We also assume that the address tape has length $\lceil \log n \rceil$. In particular, there are two special markers at the address tape, and there are $\lceil \log n \rceil$ cells strictly between them. The machine can only modify this portion of $\lceil \log n \rceil$ cells; the rest of the address tape is read-only. For sub-linear time Turing machines, this can be viewed as a mechanism to provide information about n (i.e., the length of x ; up to a factor of 2). We also require that whenever the machine enters the “address” state, all the $\lceil \log n \rceil$ cells between the two markers are non-empty, hence we can interpret the concatenation of these cells as a (binary) address.

Every bit operation takes one step. Therefore, it takes $\Theta(\log n)$ time to write down an address. Note that we clear the address tape after each access, which means when we access another input bit, we have to spend another $\Theta(\log n)$ time to write down the address from

scratch. This definition ensures that in $O(\log n)$ time we can only access a constant number of input bits, so **DLOGTIME** becomes a natural uniform analog of NC^0 .

In addition to the address tape and the answer tape, we also have a constant number of work tapes. In the case that our Turing machine computes a multi-output function f , we also provide an input tape that contains an index i (note that our *real* input is the “oracle” x), which means our Turing machine outputs the i -th bit of $f(x)$. We use $M^x(i)$ to denote the output of the machine M on input i given oracle access to the string x . To measure the space complexity of our Turing machine, we assume the input tape is read-only and we only count the total length of work tapes.

Definition 3.1. Let $c > 0$ be a constant, $p(\cdot)$ be a polynomial, and $F = \{F_n : \{0, 1\}^n \rightarrow \{0, 1\}^{p(n)}\}$ be an ensemble of functions. We say $F \in \text{TIME}[c \log n]$ if there is a Turing machine M with running time $c \log n$ such that for every sufficiently large n , every $x \in \{0, 1\}^n$, and every $1 \leq i \leq p(n)$, $M^x(n, i)$ outputs the i -th bit of $F_n(x)$.¹⁴

Let $\text{DLOGTIME} = \bigcup_{c \geq 1} \text{TIME}[c \log n]$.

Definition 3.2. Let $c > 0$ be a constant, $p(\cdot)$ be a polynomial, and $F = \{F_n : \{0, 1\}^n \rightarrow \{0, 1\}^{p(n)}\}$ be an ensemble of functions.

We say F is in $\text{ATIME}[c \log n]$, if there is an *alternating* Turing machine M of $c \log n$ running time such that for every large enough n , every $x \in \{0, 1\}^n$, every $1 \leq i \leq p(n) + 1$, and every $b \in \{0, 1, \star\}$, $M^x(n, i, b) = 1$ if the i -th bit of $F_n(x)$ is b .

We say F is in $\text{SPACE}[c \log n]$, if there is a Turing machine M of space complexity $c \log n$ that satisfies the above requirement. We say F is in uniform $\oplus \text{SPACE}[c \log n]$, if there is a *parity* Turing machine M of space complexity $c \log n$ that satisfies the above requirement.

Let $\text{ALOGTIME} = \text{NC}^1 = \bigcup_{c \geq 1} \text{ATIME}[c \log n]$. (That is, in this paper, we use **ALOGTIME** and NC^1 interchangeably.) We also define $\text{L} = \bigcup_{c \geq 1} \text{SPACE}[c \log n]$ and $\oplus \text{L} = \bigcup_{c \geq 1} \oplus \text{SPACE}[c \log n]$.

For the readers not familiar with parity Turing machines, we provide an alternative definition of $\oplus \text{L}$ by its complete problems; see Section 3.7.

3.2 Resource-Bounded Kolmogorov Complexity

We define some variants of resource-bounded Kolmogorov complexity. In particular, we define the plain Kolmogorov complexity K , the KT complexity [All01], the time-bounded Kolmogorov complexity K^t [Ko91], and Levin’s K_t complexity [Lev84]. Then we define the NC^1 - and L -versions of K^t .

Definition 3.3. Let U be a Turing machine and x be a string. Artificially let $x_{|x|+1} = \star$.

- $K_U(x)$ is the minimum $|d|$ over the description $d \in \{0, 1\}^*$ such that for every $1 \leq i \leq |x|+1$ and $b \in \{0, 1, \star\}$, $U^d(i, b)$ accepts if and only if $x_i = b$.
- $\text{KT}_U(x)$ is the minimum value of $|d|+t$ over the pairs (d, t) such that for every $1 \leq i \leq |x|+1$ and $b \in \{0, 1, \star\}$, $U^d(i, b)$ accepts in t steps if and only if $x_i = b$.
- $\text{Kt}_U(x)$ is the minimum value of $|d| + \log t$ over the pairs (d, t) such that for every $1 \leq i \leq |x| + 1$ and $b \in \{0, 1, \star\}$, $U^d(i, b)$ accepts in t steps if and only if $x_i = b$.
- Let $t : \mathbb{N} \rightarrow \mathbb{N}$ be a resource bound. $\text{K}_U^t(x)$ is the minimum value of $|d|$ such that for every $1 \leq i \leq |x| + 1$ and $b \in \{0, 1, \star\}$, $U^d(i, b)$ accepts in $t(|x|)$ steps if and only if $x_i = b$.

Definition 3.4. Let U_a be an alternating Turing machine, and U_s be a (space-bounded) Turing machine. Let x be a string and artificially let $x_{|x|+1} = \star$.

¹⁴Note that n and i are encoded in binary using $O(\log n)$ bits here.

- Let $t : \mathbb{N} \rightarrow \mathbb{N}$ be a resource bound. $\text{NC}^1\text{-K}_{U_a}^t(x)$ is the minimum value of $|d|$ such that for every $1 \leq i \leq |x| + 1$ and $b \in \{0, 1, \star\}$, $U_a^d(i, b)$ accepts in alternating time $\log t(|x|)$ if and only if $x_i = b$.
- Let $t : \mathbb{N} \rightarrow \mathbb{N}$ be a resource bound. $\text{L-K}_{U_s}^t(x)$ is the minimum value of $|d|$ such that for every $1 \leq i \leq |x| + 1$ and $b \in \{0, 1, \star\}$, $U_s^d(i, b)$ accepts in space $\log t(|x|)$ if and only if $x_i = b$.

Fix the number of tapes $k \geq 2$, then there is a universal k -tape Turing machine U that simulates every k -tape Turing machine with only a constant factor blow-up in both time complexity and space complexity [Für82]. Our results hold for this efficient universal Turing machine, therefore in what follows, we drop the subscript U and simply write KT , K^t , etc.

We also define the *circuit complexity* of a truth table:

Definition 3.5. Let $N = 2^n$, $tt \in \{0, 1\}^N$ be the truth table that corresponds to a function $f : \{0, 1\}^n \rightarrow \{0, 1\}$. We define $\text{Size}(tt)$ as the size (number of gates) of the smallest circuit that computes f .

Given a complexity measure μ , the Minimum μ Problem is the language $\{(x, 1^k) : \mu(x) \leq k\}$. In particular:

Definition 3.6. We define the following problems:

- (Minimum KT Problem) $\text{MKTP} := \{(x, 1^k) : \text{KT}(x) \leq k\}$.
- (Minimum Time-Bounded Kolmogorov Complexity Problem) $\text{MINKT} := \{(x, 1^t, 1^s) : \text{K}^t(x) \leq s\}$.
- (Minimum Circuit Size Problem) $\text{MCSP} := \{(tt, 1^s) : \text{Size}(tt) \leq s\}$.

There are natural search versions for the problems above. The search version for MKTP is to find a description d such that x can be generated from d implicitly in time at most $k - |d|$. The search version for MINKT is to find a description d of size at most s for x such that x can be generated from d in time at most t . The search version for MCSP is to find a circuit of size at most s for the Boolean function whose truth table is tt .

We need a “trivial” upper bound on these complexity measures. We only state the upper bound for KT complexity.

Fact 3.7 ([ABK⁺06, Proposition 13]). *There is an absolute constant $c' > 0$ such that $\text{KT}(x) \leq |x| + c' \log |x|$ for every string x .*

We need the fact that most strings have large Kolmogorov complexity.

Fact 3.8. *Let n be an integer, $s \leq n - 1$, then*

$$\Pr_{\mathbf{x} \leftarrow \mathcal{U}_n} [\text{K}(\mathbf{x}) \leq s] \leq 2^{-(n-s-1)}.$$

Proof Sketch. The number of strings x such that $\text{K}(x) \leq s$ is at most $\sum_{i=0}^s 2^i = 2^{s+1} - 1$. $\square$

3.3 Basic Information Theory

We also need some basic concepts in information theory. The *Shannon entropy* of a random variable X , denoted as $\text{H}(X)$, is defined as

$$\text{H}(X) := \mathbb{E}_{\mathbf{x} \leftarrow X} [-\log \Pr[X = \mathbf{x}]].$$

The *min-entropy* of a random variable X , denoted as $H_\infty(X)$, is the largest real number k such that for every x in the support of X ,

$$\Pr[X = x] \leq 2^{-k}.$$

Let X, Y be two random variables defined over a set $\mathcal{S}$. The *statistical distance* between X and Y , denoted as $\text{SD}(X, Y)$, is defined as

$$\text{SD}(X, Y) := \frac{1}{2} \sum_{s \in \mathcal{S}} |\Pr[X = s] - \Pr[Y = s]|.$$

An equivalent definition is as follows: $\text{SD}(X, Y)$ is the maximum value of ϵ such that there is a (possibly unbounded) distinguisher $\mathcal{D}$ that ϵ -distinguishes X from Y :

$$\text{SD}(X, Y) := \max_{\mathcal{D}: \mathcal{S} \rightarrow \{0,1\}} |\Pr[\mathcal{D}(X) = 1] - \Pr[\mathcal{D}(Y) = 1]|.$$

3.4 Bounded-Error Average-Case Hardness

We define the (bounded-error) average-case hardness of a function f . (Think of $f = \text{KT}$ or K^t .) In the cryptographic setting, we require that any algorithm with an *arbitrary polynomial* run time fails to solve a *fixed-polynomial* fraction of inputs.

Definition 3.9. Let $f : \{0, 1\}^* \rightarrow \mathbb{N}$ be a function.

- We say that f is (*bounded-error*) *hard on average* if the following is true. There is a constant $c > 0$ such that for every PPT¹⁵ machine $\mathcal{A}$ and every large enough input length n ,

$$\Pr_{\mathbf{x} \leftarrow \mathcal{U}_n} [\mathcal{A}(\mathbf{x}) = f(\mathbf{x})] \leq 1 - \frac{1}{n^c}.$$

- Let d be a constant. We say that f is (bounded-error) *hard on average to $(d \log n)$ -approximate* if the following is true. There is a constant $c > 0$ such that for every PPT machine $\mathcal{A}$ and every large enough input length n ,

$$\Pr_{\mathbf{x} \leftarrow \mathcal{U}_n} [f(\mathbf{x}) \leq \mathcal{A}(\mathbf{x}) \leq f(\mathbf{x}) + d \log n] \leq 1 - \frac{1}{n^c}.$$

3.5 One-Way Functions

We recall the standard definition of one-way functions and weak one-way functions. Note that by a standard padding trick (see e.g., [Gol01]), every (weak or strong) one-way function can be made *length-preserving*, i.e. for every input $x \in \{0, 1\}^*$, $|f(x)| = |x|$. In this paper, we will implicitly assume that *every candidate one-way function we consider is length-preserving*.

Definition 3.10 (One-Way Functions). Let $f : \{0, 1\}^* \rightarrow \{0, 1\}^*$ be a polynomial-time computable function. We say f is a *one-way function* if for every PPT adversary $\mathcal{A}$, it inverts a random output of f with negligible probability. That is, for every $n \in \mathbb{N}$,

$$\Pr_{\mathbf{x} \leftarrow \mathcal{U}_n} [\mathcal{A}(f(\mathbf{x})) \in f^{-1}(f(\mathbf{x}))] \leq \text{negl}(n).$$

One-way functions are also called *strong* one-way functions, as no PPT adversary could invert it non-trivially. We also consider *weak* one-way functions, where no PPT adversary could invert it on a $1 - 1/p(n)$ fraction of inputs for some *fixed* polynomial $p(\cdot)$.

¹⁵PPT stands for probabilistic polynomial-time.

Definition 3.11 (Weak One-Way Functions). Let $f : \{0, 1\}^* \rightarrow \{0, 1\}^*$ be a polynomial-time computable function. We say f is a *weak one-way function* if there is a polynomial $p(\cdot)$ such that the following holds. For every PPT adversary $\mathcal{A}$, it inverts a random output of f with probability at most $1 - 1/p(n)$. That is, for every $n \in \mathbb{N}$,

$$\Pr_{\mathbf{x} \leftarrow \mathcal{U}_n} [\mathcal{A}(f(\mathbf{x})) \in f^{-1}(f(\mathbf{x}))] \leq 1 - \frac{1}{p(n)}.$$

Yao showed that every weak one-way function can be *amplified* into a strong one-way function.

Theorem 3.12 ([Yao82, Gol01]). *If there exists a weak one-way function, then there exists a strong one-way function.*

In particular, let f be a weak one-way function. Then there is a polynomial $k(\cdot)$ such that the following function f^k is a strong one-way function.

$$f^k(x_1, x_2, \dots, x_{k(n)}) = f(x_1) \circ f(x_2) \circ \dots \circ f(x_{k(n)}),$$

where $x_1, x_2, \dots, x_{k(n)}$ are length- n inputs.

3.6 Conditionally Secure Entropy-Preserving PRGs

Here we define conditionally secure entropy-preserving PRGs (condEP-PRGs), introduced in [LP20].

A *pseudorandom generator*, according to the standard definition, is a polynomial-time computable function $G : \{0, 1\}^n \rightarrow \{0, 1\}^m$ (where $m > n$) such that $G(\mathcal{U}_n)$ and $\mathcal{U}_m$ are computationally indistinguishable. Compared with standard PRGs, a *condEP-PRG* $G : \{0, 1\}^n \rightarrow \{0, 1\}^m$ has three differences:

- The input distribution of G is not $\mathcal{U}_n$. Instead, it is the uniform distribution over a subset of inputs $\mathcal{E}_n$, called the *condition*. (We will use $\mathcal{E}_n$ to denote both the subset and the uniform distribution over this subset.)
- G is *entropy-preserving*, meaning that $G(\mathcal{E}_n)$ has large (information-theoretic) entropy. (Note that $\log |\mathcal{E}_n| \leq n \leq m$. As a consequence, $\log |\mathcal{E}_n|$ cannot be too small compared to m .)
- Finally, G only $(1/p(n))$ -fools PPT adversaries for a fixed polynomial $p(\cdot)$. For comparison, a standard PRG is required to $(1/p(n))$ -fool PPT adversaries *for every polynomial* $p(\cdot)$. This difference is mostly technical.

Definition 3.13 (Conditionally Secure Entropy-Preserving PRG, abbr. condEP-PRG, [LP20]). Let $\gamma > 0$ be a constant and $p(\cdot)$ be a polynomial. Consider a polynomial-time computable ensemble of functions $G = \{G_n : \{0, 1\}^n \rightarrow \{0, 1\}^{n+\gamma \log n}\}$. We say G is a *condEP-PRG*, if there is a family of subsets $\mathcal{E} = \{\mathcal{E}_n \subseteq \{0, 1\}^n\}$ (called the “events” or “conditions”) such that the following are true.

(Pseudorandomness) $G_n(\mathcal{E}_n)$ is $(1/p(n))$ -indistinguishable from $\mathcal{U}_{n+\gamma \log n}$ by PPT adversaries.

That is, for every PPT $\mathcal{A}$ and every integer n ,

$$\left| \Pr_{\mathbf{x} \leftarrow \mathcal{U}_{n+\gamma \log n}} [\mathcal{A}(\mathbf{x}) = 1] - \Pr_{\mathbf{x} \leftarrow G_n(\mathcal{E}_n)} [\mathcal{A}(\mathbf{x}) = 1] \right| < 1/p(n).$$

(Entropy-Preservation) There is a constant d such that for every large enough n , $H(G_n(\mathcal{E}_n)) \geq n - d \log n$.

We say the *stretch* of G is $\gamma \log n$ and the *security* of G is $1/p(n)$.

Theorem 3.14 ([LP20]). *There is a function EP-PRG computable in ALOGTIME such that the following holds. For any one-way function $f : \{0,1\}^* \rightarrow \{0,1\}^*$ and any constant $\gamma > 0$, let $G(x, z) = \text{EP-PRG}(\gamma, x, f(x), z)$, then G is a condEP-PRG with stretch $\gamma \log n$ and security $1/n^\gamma$.*

Remark 3.15. It is important that the machine EP-PRG is fixed and does not depend on the constant γ . Suppose there is an absolute constant $c > 0$ such that for every $\gamma > 0$, there is a PRG G_γ that runs in $\text{TIME}[c \log n]$ and stretches n bits into $n + \gamma \log n$ bits. The outputs of G_γ will always have KT complexity at most $n + c \log n + O(1) < n + \gamma \log n$, hence a heuristic for MKTP can always distinguish the outputs of G_γ from truly random strings. It follows that we can use such G_γ to argue about the hardness of MKTP. On the other hand, if the time complexity of G_γ depends on γ , it does not necessarily imply any hardness of MKTP.

3.7 Complete Problems for $\oplus\text{L}$

We introduce the $\oplus\text{L}$ -complete problems, called Connected Matrix Determinant (CMD) and Decomposed Connected Matrix Determinant (DCMD), which will be crucial to us. Originally motivated by secure multi-party computation [IK00, IK02], these problems have found surprisingly many applications in cryptography and complexity theory [AIK06, GGH⁺07, FSUV13, HS17, CR20].

Let n be any integer, define $\ell_{\text{CMD}}(n) := n(n+1)/2$ and $\ell_{\text{DCMD}}(n) := n^3(n+1)/2$.

Definition 3.16 (See e.g., [CR20]). An instance of CMD is an $n \times n$ matrix over $\text{GF}(2)$ where the main diagonal and above may contain either 0 or 1, the second diagonal (i.e., the one below the main diagonal) contains 1, and other entries are 0. In other words, the matrix is of the following form (where $*$ represents any element in $\text{GF}(2)$):

$$\begin{pmatrix} * & * & * & \cdots & * & * \\ 1 & * & * & \cdots & * & * \\ 0 & 1 & * & \cdots & * & * \\ 0 & 0 & 1 & \cdots & * & * \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & \cdots & 1 & * \end{pmatrix}.$$

The instance is an $(n(n+1)/2)$ -bit string specifying elements on and above the main diagonal. We define $x \in \text{CMD}$ if and only if the determinant (over $\text{GF}(2)$) of the matrix corresponding to x is 1.

An instance of DCMD is a string of length $n^3(n+1)/2$. For an input x , $\text{DCMD}(x)$ is computed as follows: we partition x into blocks of length n^2 , let $y_i (1 \leq i \leq n(n+1)/2)$ be the parity of the i -th block, and define $\text{DCMD}(x) := \text{CMD}(y_1 \circ y_2 \circ \cdots \circ y_{n(n+1)/2})$.

In fact, the precise definitions of CMD and DCMD are not important here, but we need the following important facts about them.

Theorem 3.17 ([AIK06]). *Let n be an integer. There is a function $P_{\text{CMD}} : \{0,1\}^{\ell_{\text{CMD}}(n)} \times \{0,1\}^{\ell_{\text{DCMD}}(n)-1} \rightarrow \{0,1\}^{\ell_{\text{DCMD}}(n)}$ computable in DLOGTIME such that the following holds. For any input $x \in \{0,1\}^{\ell_{\text{CMD}}(n)}$, the distribution of $P_{\text{CMD}}(x, \mathcal{U}_{\ell_{\text{DCMD}}(n)-1})$ is equal to the uniform distribution over $\{y \in \{0,1\}^{\ell_{\text{DCMD}}(n)} : \text{DCMD}(y) = \text{CMD}(x)\}$.*

Note that P_{CMD} only uses $\ell_{\text{DCMD}}(n) - 1$ random bits, which is optimal. It also implies:

Corollary 3.18. *DCMD is balanced. In other words, for every integer n , the number of Yes instances and No instances of DCMD on input length $\ell_{\text{DCMD}}(n)$ are the same.*

Proof. Fix any Yes instance $x \in \{0, 1\}^n$ of CMD, then $\{P_{\text{CMD}}(x, r) : r \in \{0, 1\}^{\ell_{\text{DCMD}}(n)-1}\}$ contains every Yes instance of DCMD. It follows that there are at most $2^{\ell_{\text{DCMD}}(n)-1}$ Yes instances of DCMD on input length $\ell_{\text{DCMD}}(n)$. The same upper bound can also be obtained for No instances. Since there are $2^{\ell_{\text{DCMD}}(n)}$ strings of length $\ell_{\text{DCMD}}(n)$, there must be exactly $2^{\ell_{\text{DCMD}}(n)-1}$ Yes instances and exactly $2^{\ell_{\text{DCMD}}(n)-1}$ No instances of length $\ell_{\text{DCMD}}(n)$. $\square$

Theorem 3.19 ([IK00, IK02]). *CMD is $\oplus\text{L}$ -complete under projections.*¹⁶

In other words, a language L is in $\oplus\text{L}$ if and only if there is a polynomial $t(\cdot)$ and a DLOGTIME-computable projection $p : \{0, 1\}^n \rightarrow \{0, 1\}^{\ell_{\text{CMD}}(t(n))}$ such that for every input $x \in \{0, 1\}^n$, $x \in L$ if and only if $\text{CMD}(p(x)) = 1$.

Remark 3.20. A proof of Theorem 3.19 can be found in [CR20, Section B.1]. However, the proof in [CR20] does not show that the projections are DLOGTIME-uniform. In particular, the reduction needs to calculate the topological order of the underlying (parity) branching program $(\sigma_1, \sigma_2, \dots, \sigma_m)$ in [CR20, Section B.1], which may not be computable in DLOGTIME.

We can fix this issue by adding a clock to the log-space Turing machine; a state of the Turing machine appears earlier in the topological order if its clock value is smaller. Equivalently, let $G = (V, E)$ be the old branching program. The new branching program G^{new} has a vertex (i, v) for every $0 \leq i \leq |V|$ and $v \in V$, and has edges from (i, u) to $(i+1, v)$ for every edge $(u, v) \in G$ and every $0 \leq i < |V|$. Let $V = \{v_1, \dots, v_n\}$, then

$$(0, v_1), \dots, (0, v_n), (1, v_1), \dots, (1, v_n), \dots, (|V|, v_1), \dots, (|V|, v_n)$$

is a valid topological ordering of G^{new} . Now we can use [CR20, Section B.1] to reduce the computation of G^{new} to CMD by a DLOGTIME-uniform projection.

We thank Yanyi Liu for pointing out this issue.

Theorem 3.17 and 3.19 implies the following beautiful result in [AIK06].

Theorem 3.21 ([AIK06]). *Suppose there is a one-way function computable in $\oplus\text{L}$. Then there is a one-way function computable in DLOGTIME.*

Proof Sketch. Let f be a one-way function in $\oplus\text{L}$. There is a DLOGTIME-computable function $p(\cdot, i)$ that maps n input bits to $\text{poly}(n)$ output bits such that for every integer i and every string x , the i -th output bit of $f(x)$ is $\text{CMD}(p(x, i))$. Consider the following function:

$$g(x, y, i) := P_{\text{CMD}}(p(x, i), y).$$

It turns out that the function $g(x, y_1, \dots, y_n) = g(x, y_1, 1) \circ \dots \circ g(x, y_n, n)$ is still one-way. $\square$

4 KT Complexity and Parallel Cryptography

In this section, we characterize the existence of one-way functions in DLOGTIME by the average-case hardness of MKTP. Recall that the seminal work of [AIK06] showed that the existence of one-way functions in DLOGTIME is also equivalent to the existence of one-way functions in uniform NC^1 , L , or $\oplus\text{L}$.

Theorem 1.1 (Main Result; Informal). *There is a one-way function computable in uniform NC^1 if and only if KT is bounded-error hard on average.*

¹⁶A *projection* is a (multi-output) function where each output bit either is a constant or only depends on one input bit.

4.1 One-Way Functions in NC^0 from Hardness of MKTP

Theorem 4.1. *Suppose that the search version of KT is bounded-error hard on average. Then there is a one-way function computable in DLOGTIME.*

Proof. We show that there is a weak one-way function computable in logarithmic space. Then by Theorem 3.12, there is a one-way function in logarithmic space, and by Theorem 3.21, there is a one-way function in DLOGTIME.

Suppose KT is bounded-error hard on average. By Definition 3.9, there is a constant $c > 0$ such that for every PPT algorithm $\mathcal{A}$ and every large enough n , the probability that $\mathcal{A}$ solves the search version of KT on a random length- n input is at most $1 - 1/n^c$.

For a string x , we define $t(x)$ to be the parameter t in the definition of $\text{KT}(x)$ (Definition 3.3). Formally, $t(x)$ is the smallest integer t such that there is a description d of length $\text{KT}(x) - t$ such that for every $1 \leq i \leq |x|$ and $b \in \{0, 1, \star\}$, $U^d(i, b)$ accepts in t steps if and only if $x_i = b$. We can see that most strings have small $t(x)$. In what follows, let c_1 be the absolute constant in Fact 3.7 such that for every $x \in \{0, 1\}^n$, $\text{KT}(x) \leq |x| + c_1 \log |x|$.

Claim 4.2. *For all but an $1/n^{c+1}$ fraction of strings $x \in \{0, 1\}^n$, we have $t(x) \leq (c + c_1 + 2) \log n$.*

Proof of Claim 4.2. By Fact 3.7, for every $x \in \{0, 1\}^n$, $\text{KT}(x) \leq n + c_1 \log n$. By Fact 3.8, all but a $1/n^{c+1}$ fraction of strings $x \in \{0, 1\}^n$ satisfies that $K(x) > n - (c + 1) \log n - 1$. For such strings x , we have $t(x) \leq \text{KT}(x) - K(x) \leq (c + c_1 + 2) \log n$. $\diamond$

For convenience, we say a pair (d, t) *outputs* the string x , if (d, t) is a valid “witness” for $\text{KT}(x)$, i.e. for every $1 \leq i \leq |x| + 1$ and $b \in \{0, 1, \star\}$, $U^d(i, b)$ accepts in time t if and only if $x_i = b$. Let $\text{Output}(d, t)$ be the (unique) string that (d, t) outputs; if (d, t) does not output any (finite) string, let $\text{Output}(d, t) = \perp$.

We define a weak one-way function f as in Algorithm 1.

Algorithm 1 Weak OWF in L from Average-Case Hardness of MKTP

```

1: function  $f(\ell, t, M)$ 
2:   The input consists of integers  $\ell \in [n + c_1 \log n]$ ,  $t \in [(c + c_1 + 2) \log n]$ , and a string
    $M \in \{0, 1\}^{n + c_1 \log n}$ .
3:    $M' \leftarrow$  the first  $\ell$  bits of  $M$ 
4:    $\text{out} \leftarrow \text{Output}(M', t)$ 
5:   if  $|\text{out}| = n$  then
6:     return the concatenation of  $\ell$ ,  $t$ , and  $\text{out}$ 
7:   else
8:     return  $\perp$ 

```

Since $t \leq O(\log n)$, we can always compute $\text{Output}(M', t)$ in logarithmic space. It follows that f is computable in logarithmic space.

Let $\mathcal{D}_{\text{owf}}$ be the output distribution of f on uniform inputs. In other words, to sample from $\mathcal{D}_{\text{owf}}$, we sample two integers $\ell \leftarrow [n + c_1 \log n]$, $t \leftarrow [(c + c_1 + 2) \log n]$ and a string M of length $n + c_1 \log n$, and output $f(\ell, t, M)$. We prove that $\mathcal{D}_{\text{owf}}$ *almost dominates* the uniform distribution over $\{0, 1\}^n$, in the following sense.

Claim 4.3. *Let n be a large enough integer. For every string x such that $t(x) \leq (c + c_1 + 2) \log n$, the probability that a random sample from $\mathcal{D}_{\text{owf}}$ is equal to $(\text{KT}(x) - t(x), t(x), x)$ is at least $\frac{1}{2^{n n^{2+c_1}}}$.*

Proof of Claim 4.3. For a large enough n , with probability at least $\frac{1}{n^2}$, the sampler for $\mathcal{D}_{\text{owf}}$ samples $t = t(x)$ and $\ell = \text{KT}(x) - t(x)$. Then, with probability $\frac{1}{2^\ell} \geq \frac{1}{2^{n n^{c_1}}}$, the sampler samples a description M' such that $\text{Output}(M', t) = x$. It follows that w.p. at least $\frac{1}{2^{n n^{2+c_1}}}$ the sampler outputs $(\text{KT}(x) - t(x), t(x), x)$. $\diamond$

Now, we can prove the security of the weak OWF f . Let $\mathcal{A}_{\text{owf}}$ be a candidate PPT adversary trying to invert f . We construct a polynomial-time algorithm $\mathcal{A}_{\text{KT}}$ that attempts to solve (the search version of) MKTP in Algorithm 2.

Algorithm 2 Bounded-Error Heuristic $\mathcal{A}_{\text{KT}}$ for MKTP from Inverter $\mathcal{A}_{\text{owf}}$ for f

```

1: function  $\mathcal{A}_{\text{KT}}(x)$ 
2:    $n \leftarrow |x|$ ;  $\text{OPT} \leftarrow +\infty$ ;  $\text{WITNESS} \leftarrow \perp$ 
3:   for  $\ell \in [n + c_1 \log n]$  and  $t \in [(c + c_1) \log n]$  do
4:      $(\ell', t', M) \leftarrow \mathcal{A}_{\text{owf}}(\ell, t, x)$ 
5:      $M' \leftarrow$  the first  $\ell'$  bits of  $M$ 
6:     if  $\text{Output}(M', t') = x$  and  $\text{OPT} > |M'| + t'$  then
7:        $\text{OPT} \leftarrow |M'| + t'$ 
8:        $\text{WITNESS} \leftarrow (M', t')$ 
9:   return  $(\text{OPT}, \text{WITNESS})$ 

```

Note that for a fixed input x , $\mathcal{A}_{\text{KT}}(x)$ fails to output a valid witness for $\text{KT}(x)$ only if $\mathcal{A}_{\text{owf}}$ fails to invert the output $(\text{KT}(x) - t(x), t(x), x)$. Let $p_{\text{fail}}(x)$ be the probability (over the internal randomness of $\mathcal{A}_{\text{owf}}$) that $\mathcal{A}_{\text{owf}}$ fails to invert $(\text{KT}(x) - t(x), t(x), x)$, then we have

$$\mathbb{E}_{\mathbf{x} \leftarrow \mathcal{U}_n} [p_{\text{fail}}(\mathbf{x})] \geq \Pr_{\mathbf{x} \leftarrow \mathcal{U}_n} [\mathcal{A}_{\text{KT}}(\mathbf{x}) \text{ fails on input } \mathbf{x}] \geq \frac{1}{n^c}. \quad (1)$$

Let p be the probability that $\mathcal{A}_{\text{owf}}$ fails to invert a random input of f . Then

$$\begin{aligned}
p &\geq \sum_{\substack{x \in \{0,1\}^n \\ t(x) \leq (c+c_1+2) \log n}} \Pr_{\mathbf{y} \leftarrow \mathcal{D}_{\text{owf}}} [\mathbf{y} = (\text{KT}(x) - t(x), t(x), x)] \cdot p_{\text{fail}}(x) \\
&\geq \sum_{\substack{x \in \{0,1\}^n \\ t(x) \leq (c+c_1+2) \log n}} \frac{1}{2^n n^{2+c_1}} \cdot p_{\text{fail}}(x) && (\text{Claim 4.3}) \\
&\geq \frac{1}{2^n n^{2+c_1}} \left(\left(\sum_{x \in \{0,1\}^n} p_{\text{fail}}(x) \right) - \frac{2^n}{n^{c+1}} \right) && (\text{Claim 4.2}) \\
&\geq \frac{1}{n^{2+c_1}} \left(\mathbb{E}_{\mathbf{x} \leftarrow \mathcal{U}_n} [p_{\text{fail}}(\mathbf{x})] \right) - \frac{1}{n^{c+c_1+3}} \\
&\geq \frac{1}{n^{2+c_1+c}} - \frac{1}{n^{c+c_1+3}}. && \text{By (1)}
\end{aligned}$$

Let $c' = c + c_1 + 4$, then every PPT adversary $\mathcal{A}_{\text{owf}}$ fails to invert a random input of f w.p. at least $\frac{1}{n^{c'}}$. It follows that f is a weak OWF. $\square$

4.2 Hardness of MKTP from One-Way Functions in $\oplus \mathsf{L}$

In this section, we prove the following theorem.

Theorem 4.4. *Suppose there is a one-way function computable in $\oplus \mathsf{L}$. Then for every constant $\lambda > 0$, KT is bounded-error hard on average to approximate within an additive factor of $\lambda \log n$.*

Let f be a one-way function in $\oplus \mathsf{L}$. The proof consists of three steps:

- First, we use f to build a condEP-PRG G . If f is computable in $\oplus \mathsf{L}$, then G is also computable in $\oplus \mathsf{L}$. This step is the same as in [LP20] and follows directly from Theorem 3.14.

- Second, we construct the randomized encoding $\tilde{G}$ of G . We argue that $\tilde{G}$ is also a condEP-PRG. Moreover, $\tilde{G}$ is computable in DLOGTIME. This step is implemented in Lemma 4.5.
- Last, as every output of $\tilde{G}$ has small KT complexity, we use the security of $\tilde{G}$ to show that MKTP is bounded-error hard on average. This step is implemented in Lemma 4.7.

4.2.1 CondEP-PRG in DLOGTIME

In this section, we prove the following lemma that constructs a DLOGTIME-computable condEP-PRG from a $\oplus\text{L}$ -computable condEP-PRG.

Lemma 4.5. *Suppose there is a constant $c > 0$ such that for every constant $\lambda > 0$, there is a condEP-PRG G with stretch $\lambda \log n$ and security $1/n^\lambda$ that is computable in $\oplus\text{SPACE}[c \log n]$.*

Then there is a constant $c' > 0$ such that for every constant $\lambda > 0$, there is a condEP-PRG $\tilde{G}$ with stretch $\lambda \log n$ and security $1/n^\lambda$ that is computable in $\text{TIME}[c' \log n]$.

Proof. Fix an input length n . Let λ' be a constant that depends on λ , we will fix λ' later. Let G be a condEP-PRG with stretch $\lambda' \log n$ and security $1/n^{\lambda'}$ that is computable in $\oplus\text{SPACE}[c \log n]$. We denote the n -th slice of G as $G_n : \{0, 1\}^n \rightarrow \{0, 1\}^\ell$, where $\ell := n + \lambda' \log n$.

Let $N := n^c$. Since $G \in \oplus\text{SPACE}[\log N]$, there are projections

$$G_1^{\text{proj}}, G_2^{\text{proj}}, \dots, G_\ell^{\text{proj}} : \{0, 1\}^n \rightarrow \{0, 1\}^{\ell_{\text{CMD}}(N)},$$

such that the i -th output bit of $G(x)$ is equal to $\text{CMD}(G_i^{\text{proj}}(x))$. Let $r_1, r_2, \dots, r_\ell$ be random strings of length $\ell_{\text{DCMD}}(N) - 1$. Let P_{CMD} be the DLOGTIME-computable function defined in Theorem 3.17, then the i -th output bit of $G(x)$ is equal to $\text{DCMD}(P_{\text{CMD}}(G_i^{\text{proj}}(x), r_i))$. We define

$$\tilde{G}(x, r_1, \dots, r_\ell) = P_{\text{CMD}}(G_1^{\text{proj}}(x), r_1) \circ P_{\text{CMD}}(G_2^{\text{proj}}(x), r_2) \circ \dots \circ P_{\text{CMD}}(G_\ell^{\text{proj}}(x), r_\ell).$$

($\tilde{G}$ is the “randomized encoding” of G in the sense of [AIK06].)

It is easy to see that there is a constant c_g depending only on c such that $\tilde{G} \in \text{TIME}[c_g \log n]$. Note that the input length of $\tilde{G}$ is $n_{\text{in}} := n + \ell \cdot (\ell_{\text{DCMD}}(N) - 1)$, the output length of $\tilde{G}$ is $n_{\text{out}} := \ell \cdot \ell_{\text{DCMD}}(N)$, and $n_{\text{out}} = n_{\text{in}} + (\ell - n) = n_{\text{in}} + \lambda' \log n$. Here, we fix λ' large enough such that $\lambda' \geq \lambda \frac{\log n_{\text{in}}}{\log n}$.

Claim 4.6. *$\tilde{G}$ is a condEP-PRG with stretch $\lambda \log n_{\text{in}}$ and security $1/(n_{\text{in}})^\lambda$.*

Proof. Clearly, the stretch of $\tilde{G}$ is $\lambda' \log n \geq \lambda \log n_{\text{in}}$.

Suppose $\mathcal{E} = \{\mathcal{E}_n \subseteq \{0, 1\}^n\}$ is a sequence of events such that G and $\mathcal{E}$ satisfy Definition 3.13. Let $\tilde{\mathcal{E}} := \{\tilde{\mathcal{E}}_{n_{\text{in}}}\}$ where $\tilde{\mathcal{E}}_{n_{\text{in}}} := \mathcal{E}_n \times \{0, 1\}^{\ell \cdot (\ell_{\text{DCMD}}(N) - 1)}$. We verify that $\tilde{G}$ and $\tilde{\mathcal{E}}$ satisfy Definition 3.13.

Pseudorandomness. Suppose, for the sake of contradiction, that there is a PPT adversary $\mathcal{A}'$ such that

$$\Pr[\mathcal{A}'(\tilde{G}(\tilde{\mathcal{E}}_{n_{\text{in}}}))] - \Pr[\mathcal{A}'(\mathcal{U}_{n_{\text{out}}})] \geq 1/(n_{\text{in}})^\lambda.$$

Consider an adversary $\mathcal{A}$ that distinguishes $G(\mathcal{E}_n)$ from $\mathcal{U}_\ell$ as follows. On input y , for every $1 \leq i \leq \ell$, we sample $\mathbf{r}_i \leftarrow P_{\text{CMD}}(y_i, \mathcal{U}_{\ell_{\text{DCMD}}(N) - 1})$, then $\mathbf{r}_i$ is a uniformly random length- $\ell_{\text{DCMD}}(N)$ input of DCMD such that $\text{DCMD}(\mathbf{r}_i) = y_i$. We concatenate them as $\mathbf{r} = \mathbf{r}_1 \circ \mathbf{r}_2 \circ \dots \circ \mathbf{r}_\ell$ and let $\mathcal{A}(y) = \mathcal{A}'(\mathbf{r})$.

Suppose $\mathbf{y} \leftarrow G(\mathcal{E}_n)$, then the distribution of $\mathbf{r}$ is exactly $\tilde{G}(\tilde{\mathcal{E}}_{n_{\text{in}}})$. On the other hand, suppose $\mathbf{y} \leftarrow \mathcal{U}_\ell$, then the distribution of $\mathbf{r}$ is exactly $\mathcal{U}_{n_{\text{out}}}$. As $\mathcal{A}'$ distinguishes $\tilde{G}(\tilde{\mathcal{E}}_{n_{\text{in}}})$ from $\mathcal{U}_{n_{\text{out}}}$ with advantage $\geq 1/(n_{\text{in}})^\lambda$, we can see that $\mathcal{A}$ also distinguishes $G(\mathcal{E}_n)$ from $\mathcal{U}_\ell$ with advantage $\geq 1/(n_{\text{in}})^\lambda \geq 1/n^{\lambda'}$, contradicting the security of G .

Entropy-preservation. Consider the above experiment, where we first sample $\mathbf{y} \leftarrow G(\mathcal{E}_n)$, then sample a uniform string $\mathbf{r}_i$ of length $\ell_{\text{DCMD}}(N)$ such that $\text{DCMD}(\mathbf{r}_i) = \mathbf{y}_i$ for every $1 \leq i \leq \ell$, and finally concatenate them as $\mathbf{r} = \mathbf{r}_1 \circ \mathbf{r}_2 \circ \dots \circ \mathbf{r}_\ell$. The distribution of $\mathbf{r}$ is exactly $\tilde{G}(\tilde{\mathcal{E}}_{n_{\text{in}}})$. Therefore,

$$\begin{aligned} H(\tilde{G}(\tilde{\mathcal{E}}_{n_{\text{in}}})) &= H(G(\mathcal{E}_n)) + \ell \cdot (\ell_{\text{DCMD}}(N) - 1) \\ &\geq n - \Omega(\log n) + \ell \cdot (\ell_{\text{DCMD}}(N) - 1) \\ &\geq n_{\text{in}} - \Omega(\log n_{\text{in}}). \end{aligned} \quad \diamond$$

We have only defined $\tilde{G}$ and $\tilde{\mathcal{E}}$ on input lengths of the form $n_{\text{in}}(n) = n + (n + \lambda' \log n)(\ell_{\text{DCMD}}(n^c) + 1)$. However, it is straightforward to define $\tilde{G}$ and $\tilde{\mathcal{E}}$ on every input length. Let m be an input length, $m' = n_{\text{in}}(n)$ be the largest number of the form $n_{\text{in}}(n)$ such that $m' \leq m$. On input $x \in \{0, 1\}^m$, let x_1 be the length- m' prefix of x and x_2 be the rest of x (i.e., $x = x_1 \circ x_2$), and we can define $\tilde{G}(x) = \tilde{G}(x_1) \circ x_2$. Similarly, we could define $\tilde{\mathcal{E}}_m = \tilde{\mathcal{E}}_{m'} \times \{0, 1\}^{m-m'}$. $\square$

4.2.2 Hardness of MKTP

Lemma 4.7. *Suppose there is a constant $c > 0$ such that for every constant $\lambda > 0$, there is a condEP-PRG G with stretch $\lambda \log n$ and security $1/n^\lambda$ that is computable in $\text{TIME}[c \log n]$.*

Then for every constant $\lambda > 0$, KT is bounded-error hard on average to approximate within an additive error of $\lambda \log n$.

Proof. Let $\lambda' := \lambda + c_1 + 2$ for a constant c_1 defined later, and G be a condEP-PRG with stretch $\lambda' \log n$ and security $1/n^{\lambda'}$ that is computable in $\text{TIME}[c \log n]$. Fix an input length n , and let $\ell := n + \lambda' \log n$.

We note that the KT complexity of every output of G is nontrivial. Let c_1 be a large enough constant that only depends on c . Since $G \in \text{TIME}[c \log n]$, there is a description d of constant length such that the following holds: For every input $x \in \{0, 1\}^n$, every $1 \leq i \leq \ell + 1$, and every $b \in \{0, 1, \star\}$, $U^{d,x}(i, b)$ accepts in $(c_1 - 1) \log n$ time if and only if the i -th bit of $G(x)$ is equal to b . It follows that for every $x \in \{0, 1\}^n$,

$$\text{KT}(G(x)) \leq n + (c_1 - 1) \log n + O(1) < \ell - (\lambda' - c_1) \log n.$$

Suppose, for the sake of contradiction, that KT is bounded-error easy on average to approximate within an additive factor of $\lambda \log n$. For a large constant c_{kt} that we fix later, there is a PPT machine $\mathcal{A}$ such that

$$\Pr_{\mathbf{y} \leftarrow \mathcal{U}_\ell} [\text{KT}(\mathbf{y}) \leq \mathcal{A}(\mathbf{y}) \leq \text{KT}(\mathbf{y}) + \lambda \log n] \geq 1 - \frac{1}{n^{c_{\text{kt}}}}. \quad (2)$$

It is natural to consider the following adversary $\mathcal{A}'$: On input $y \in \{0, 1\}^\ell$, $\mathcal{A}'$ outputs 1 if $\mathcal{A}(y) \geq \ell - 2 \log n$, and outputs 0 otherwise. We will prove the following two lemmas, showing that $\mathcal{A}'$ distinguishes $G(\mathcal{E}_n)$ from $\mathcal{U}_\ell$ with good advantage.

Lemma 4.8. $\Pr_{\mathbf{y} \leftarrow \mathcal{U}_\ell} [\mathcal{A}'(\mathbf{y}) = 1] \geq 1 - \frac{1}{n^2} - \frac{1}{n^{c_{\text{kt}}}}.$

Proof. By Fact 3.8, all but a $\frac{1}{n^2}$ fraction of strings $y \in \{0, 1\}^\ell$ satisfies that $\text{K}(y) \geq \ell - 2 \log n$. Therefore, for all but a $(\frac{1}{n^2} + \frac{1}{n^{c_{\text{kt}}}})$ fraction of strings $y \in \{0, 1\}^\ell$, we have $\mathcal{A}(y) \geq \text{KT}(y) \geq \text{K}(y) \geq \ell - 2 \log n$. On these strings y we have $\mathcal{A}'(y) = 1$. $\diamond$

Lemma 4.9. $\Pr_{\mathbf{y} \leftarrow G(\mathcal{E}_n)} [\mathcal{A}'(\mathbf{y}) = 1] \leq 1 - \frac{1}{n}.$

Proof. Let $H := H(G(\mathcal{E}_n))$. There is a constant d such that $\ell - H \leq d \log n$. Note that d does not depend on c_{kt} , which means we can set $c_{\text{kt}} := d + 15$.

Consider the set of outputs of G that is outputted with probability at most 2^{1-H} . We say these inputs are *good*. Let **Good** be the set of good inputs, i.e.,

$$\mathbf{Good} := \left\{ y \in \{0, 1\}^\ell : 0 < \Pr[G(\mathcal{E}_n) = y] \leq 2^{1-H} \right\}.$$

We can see that there are many good strings. Actually, let $p := \Pr_{\mathbf{y} \leftarrow G(\mathcal{E}_n)}[\mathbf{y} \in \mathbf{Good}]$, then

$$H = H(G(\mathcal{E}_n)) \leq p \cdot n + (1 - p) \cdot (H - 1),$$

which implies that $p \geq \frac{1}{n-H+1}$.

Let **Err** be the subset of **Good** on which $\mathcal{A}$ fails to produce a good approximation of KT. (In case that $\mathcal{A}$ is a randomized algorithm, it fails w.p. at least $1/n^4$.) That is,

$$\mathbf{Err} := \{ y \in \mathbf{Good} : \Pr[\text{KT}(y) \leq \mathcal{A}(y) \leq \text{KT}(y) + \lambda \log n] \leq 1 - 1/n^4 \}.$$

By Eq. (2), $|\mathbf{Err}| \leq 2^\ell / n^{c_{\text{kt}}-4}$. Therefore,

$$\Pr_{\mathbf{y} \leftarrow G(\mathcal{E}_n)}[\mathbf{y} \in \mathbf{Err}] \leq (2^\ell / n^{c_{\text{kt}}-4}) \cdot 2^{1-H} \leq 2 \cdot n^{d+4-c_{\text{kt}}} \leq 1/n^4.$$

Note that for every y in the range of $G(\mathcal{E}_n)$, if $\mathcal{A}$ is correct on y , we have $\mathcal{A}(y) < \ell - (\lambda' - c_1) \log n + \lambda \log n = \ell - 2 \log n$. Therefore for every $y \in \mathbf{Good} \setminus \mathbf{Err}$, we have $\mathcal{A}'(y) = 0$ w.p. at least $1 - 1/n^4$ over the internal randomness of $\mathcal{A}'$. It follows that

$$\begin{aligned} \Pr_{\mathbf{y} \leftarrow G(\mathcal{E}_n)}[\mathcal{A}'(\mathbf{y}) = 1] &\leq (1 - p) + \Pr_{\mathbf{y} \leftarrow G(\mathcal{E}_n)}[\mathbf{y} \in \mathbf{Err}] + \frac{1}{n^4} \\ &\leq 1 - \frac{1}{n - H + 1} + \frac{1}{n^4} + \frac{1}{n^4} \\ &\leq 1 - \frac{1}{n}. \end{aligned} \quad \diamond$$

From the pseudorandomness of the condEP-PRG G , we conclude that KT is hard on average to approximate within an additive error of $\lambda \log n$.

Note that we have only proved the hardness of MKTP on input lengths of the form $n + \lambda' \log n$, but it is straightforward to extend the argument to every input length m . Let m' be the largest number of the form $m' = n + \lambda' \log n$ such that $m' \leq m$, then $m - m' \leq O(1)$. For every $x \in \{0, 1\}^m$, let x_1 be the length- m' prefix of x . There is an absolute constant d such that $\text{KT}(x_1) - d \log m \leq \text{KT}(x) \leq \text{KT}(x_1) + d \log m$. It follows that if we can approximate MKTP on input length m' , then we can also approximate MKTP on input length m . $\square$

4.2.3 Proof of Theorem 4.4

Theorem 4.4. *Suppose there is a one-way function computable in $\oplus \mathbf{L}$. Then for every constant $\lambda > 0$, KT is bounded-error hard on average to approximate within an additive factor of $\lambda \log n$.*

Proof. Let c be a constant such that there is a one-way function f computable in $\oplus \mathbf{SPACE}[c \log n]$. Let EP-PRG be the Turing machine guaranteed in Theorem 3.14. For every constant $\lambda > 0$, let $G(x, z) = \text{EP-PRG}(\lambda, x, f(x), z)$. Then there is a constant c_1 only depending on c (not on λ) such that G is computable in $\oplus \mathbf{SPACE}[c_1 \log n]$. Moreover, G is a condEP-PRG with stretch $\lambda \log n$ and security $1/n^\lambda$.

By Lemma 4.5, there is a constant c_2 only depending on c such that for every constant $\lambda > 0$, there is a condEP-PRG with stretch $\lambda \log n$ and security $1/n^\lambda$ that is computable in $\mathbf{TIME}[c_2 \log n]$. By Lemma 4.7, for every constant $\lambda > 0$, KT is bounded-error hard on average to approximate within an additive error of $\lambda \log n$. $\square$

4.3 Bounded-Error Average-Case Robustness of Meta-Complexity

Our techniques also show that the meta-complexity of (resource-bounded) Kolmogorov complexity is “robust”, i.e. a slight change in the underlying computation model has little effect on their hardness. Actually, for many resource-bounded variants of Kolmogorov complexity, such as KT , $\text{NC}^1\text{-K}^t$, and L-K^t , either all of them admit bounded-error polynomial-time heuristics, or none of them do. (See Section 3.2 for their definition.)

Theorem 4.10. *The following are equivalent:*

1. *There is a one-way function computable in $\oplus\text{L}$.*
2. *There is a one-way function computable in DLOGTIME .*
3. *The search version of KT is hard on average.*
4. *For every constant $\lambda > 0$, KT is hard on average to approximate within an additive error of $\lambda \log n$.*
5. *There is a polynomial $t(\cdot)$ such that the search version of $\text{NC}^1\text{-K}^t$ is hard on average.*
6. *For every constant $\lambda > 0$ and polynomial $t(\cdot)$ such that $t(n) > 2n$, $\text{NC}^1\text{-K}^t$ is hard on average to approximate within an additive error of $\lambda \log n$.*
7. *There is a polynomial $t(\cdot)$ such that the search version of L-K^t is hard on average.*
8. *For every constant $\lambda > 0$ and polynomial $t(\cdot)$ such that $t(n) > 2n$, L-K^t is hard on average to approximate within an additive error of $\lambda \log n$.*

Proof Sketch. (2) $\implies$ (1), (4) $\implies$ (3), (6) $\implies$ (5), and (8) $\implies$ (7) are trivial.

(3) $\implies$ (2): Directly from Theorem 4.1.

(5) $\implies$ (2) and (7) $\implies$ (2): The construction from [LP20, Section 4] gives a one-way function computable in ALOGTIME (i.e., uniform NC^1) based on the hardness of $\text{NC}^1\text{-K}^t$. By Theorem 3.21, there is a one-way function computable in DLOGTIME . The same argument works for L-K^t .

(1) $\implies$ (4): Directly from Theorem 4.4.

(1) $\implies$ (6): Consider the condEP-PRG G computable in $\text{TIME}[c \log n]$ that we constructed in the proof of Theorem 4.4, where c is some constant. Let $t'(n) := n^{O(c)}$, then for every $x \in \{0, 1\}^n$ that in the range of G , $\text{NC}^1\text{-K}^{t'}(x) \leq n - \Theta(\log n)$. It follows that there is a polynomial t' such that $\text{NC}^1\text{-K}^{t'}$ is hard on average to approximate.

To prove that $\text{NC}^1\text{-K}^t$ is hard on average to approximate for every polynomial t , we use a padding trick. (See also [LP20, Theorem 5.6].) Let $\epsilon > 0$ be a small enough constant, and $n_1 = n^\epsilon$. Consider the generator $G'(x, r) = G(x) \circ r$, where $|x| = n_1$ and $|r| = n - n_1$. It is easy to see that if G is a condEP-PRG, then G' is also a condEP-PRG. For every $x \in \{0, 1\}^n$ that is in the range of G' , if we take ϵ to be a small enough constant, we have $\text{NC}^1\text{-K}^t(x) \leq n - \Theta(\log n)$. Since G' is pseudorandom, $\text{NC}^1\text{-K}^t$ is hard on average to approximate.

(1) $\implies$ (8): The same argument as in (1) $\implies$ (6) also works for L-K^t . $\square$

4.4 Zero-Error Average-Case Reductions

Our techniques actually imply reductions among MKTP , $\text{NC}^1\text{-MINKT}$, and L-MINKT . A closer look at these reductions reveals that they are not only *two-sided error* average-case reductions, but also *zero-error* ones! This allows us to prove new relations between the *zero-error* average-case complexity of variants of MINKT and MKTP .

The standard definition of an average-case complexity class, such as AvgZPP , is a class of pairs $(L, \mathcal{D})$ where L is a language, and $\mathcal{D}$ is a distribution ensemble over inputs. (See, e.g.,

[AB09, Chapter 18].) In this section, we only deal with the uniform distribution as the input distribution. Therefore, for simplicity, we define **AvgZPP** as a class of languages rather than (language, distribution) pairs.

Definition 4.11. Let L be a language and $\delta < 1$ be a constant. We say $L \in \text{Avg}_\delta\text{ZPP}$ if there is a zero-error PPT heuristic $\mathcal{H}$ such that the following are true: (To emphasize that $\mathcal{H}$ is a randomized heuristic, we use $\mathcal{H}(x; r)$ to denote the output of $\mathcal{H}$ on input x and randomness r .)

- For every input $x \in \{0, 1\}^*$ and $r \in \{0, 1\}^{\text{poly}(|x|)}$, $\mathcal{H}(x; r) \in \{L(x), \perp\}$.
- For every integer n , $\Pr_{\mathbf{x} \leftarrow \mathcal{U}_n, \mathbf{r} \leftarrow \mathcal{U}_{\text{poly}(n)}}[\mathcal{H}(\mathbf{x}; \mathbf{r}) = \perp] \leq \delta$.

Let $\text{AvgZPP} := \bigcup_{\delta < 1} \text{Avg}_\delta\text{ZPP}$.

We consider the parameterized versions of MKTP and MINKT in this section. Let $t(n) \leq \text{poly}(n)$ be a time bound and $s(n) \leq n$ be a size parameter. We define $\text{MKTP}[s] := \{x : \text{KT}(x) \leq s(|x|)\}$, and $\text{MINK}^t[s] := \{x : \text{K}^{t(|x|)}(x) \leq s(|x|)\}$. The problems $\text{NC}^1\text{-MINK}^t[s]$ and $\text{L-MINK}^t[s]$ are defined similarly.

A language L is ϵ -sparse if for every integer n , $\Pr_{\mathbf{x} \leftarrow \mathcal{U}_n}[\mathbf{x} \in L] \leq \epsilon$. From Fact 3.8, for every unbounded function $f(n) = \omega(1)$, $\text{MKTP}[n - f(n)]$ and $\text{MINK}^{\text{poly}(n)}[n - f(n)]$ are $o(1)$ -sparse. In general, to solve an $o(1)$ -sparse problem L on average, it suffices to design a heuristic that distinguishes every instance in L from the random instances. Therefore, the following notion of reductions will be convenient for studying the zero-error average-case complexity of sparse problems:

Definition 4.12. Let L_1, L_2 be two problems. We say there is a *one-sided mapping reduction* from L_1 to L_2 , if there are polynomials $p(\cdot)$, $m(\cdot)$ and a randomized polynomial-time mapping $\text{Red} : \{0, 1\}^n \times \{0, 1\}^{p(n)} \rightarrow \{0, 1\}^{m(n)}$ such that the following holds for every large enough n .

- For every $x \in L_1 \cap \{0, 1\}^n$ and $r \in \{0, 1\}^{p(n)}$, it holds that $\text{Red}(x; r) \in L_2$.
- The distribution of $\text{Red}(\mathcal{U}_n; \mathcal{U}_{p(n)})$ is equal to $\mathcal{U}_{m(n)}$.

Remark 4.13. Here we require that the reduction maps the uniform distribution to the uniform distribution exactly. In some cases, this requirement is too strong, and we only need that $\mathcal{U}_{m(n)}$ dominates $\text{Red}(\mathcal{U}_n; \mathcal{U}_{p(n)})$. (See [AB09, Definition 18.6].) Nevertheless, thanks to the perfect randomized encodings [AIK06], we are able to design reductions as strong as Definition 4.12.

In short, a one-sided mapping reduction (among sparse problems) maps a Yes instance to a Yes instance and maps a random instance to a random instance. It is easy to see that such reductions preserve the property of being in **AvgZPP**.

Fact 4.14. Let L_1, L_2 be two $o(1)$ -sparse problems. Suppose that there is a one-sided mapping reduction Red from L_1 to L_2 . If there is a constant $\delta_2 < 1$ such that $L_2 \in \text{Avg}_{\delta_2}\text{ZPP}$, then there is a constant $\delta_1 < 1$ such that $L_1 \in \text{Avg}_{\delta_1}\text{ZPP}$.

For every $s_1(n) \leq s_2(n)$, there is a one-sided mapping reduction from $\text{MKTP}[s_1(n)]$ to $\text{MKTP}[s_2(n)]$. (The identity mapping is a valid reduction [HS17].) Similarly, for every s_1, t_1, s_2, t_2 such that an alternating machine of description length s_1 and (alternating) time $\log t_1$ can be compiled into a deterministic machine of description length s_2 and space $\log t_2$, there is a one-sided mapping reduction from $\text{NC}^1\text{-MINK}^{t_1}[s_1]$ to $\text{L-MINK}^{t_2}[s_2]$. (Again, the identity mapping is a valid reduction.)

Now we present a one-sided mapping reduction from **L-MINKT** to **MKTP**. Actually, the reduction we present is from **L-MINKT** to $\text{MINK}^{t'}$, where $t'(n) = \lambda \log n$ for some absolute constant $\lambda > 0$.

Theorem 4.15. *For every polynomial $t(\cdot)$ and integer $c > 0$, there is a constant $c' > 0$ such that there is a one-sided mapping reduction Red from $\text{L-MINK}^t[n - c' \log n]$ to $\text{MINK}^{t'}[n - c \log n]$.*

Proof. For convenience, denote $s(n) := n - c' \log n$. Let $x \in \{0, 1\}^n$ be an input to $\text{L-MINK}^t[s]$.

The reduction is simple. It fixes $N := \text{poly}(t(n))$ and reduces a length- n input to a length- $\tilde{N}$ input, where $\tilde{N} := n \cdot \ell_{\text{DCMD}}(N)$. For every bit x_i ($1 \leq i \leq n$), it samples a uniformly random string $s_i \in \{0, 1\}^{\ell_{\text{DCMD}}(N)}$ conditioned on that $\text{DCMD}(s_i) = x_i$. Finally, it outputs the concatenation of $s_1, s_2, \dots, s_n$.

Since DCMD is balanced (Corollary 3.18), the reduction maps a random instance to a random instance. Now it remains to show that it maps a Yes instance to a Yes instance.

Suppose x is a Yes instance. Denote $\text{Red}(x; r) := s_1 \circ s_2 \circ \dots \circ s_n$, where r is the random coins that our reduction uses. For $t'(n) = \lambda \log n$, we want to prove that $K^{t'}(\text{Red}(x; r)) \leq \tilde{N} - c \log \tilde{N}$.

Let U be the universal Turing machine we consider. Then there is a description d of length at most $s(n)$ such that for every $1 \leq i \leq n + 1$ and every $b \in \{0, 1, \star\}$, $U^d(i, b)$ accepts in space $\log t(n)$ if and only if $x_i = b$. Since CMD is L-hard under projections (Theorem 3.19), for $N = \text{poly}(t(n))$, there is a DLOGTIME -computable projection

$$p_x : \{0, 1\}^{s(n)} \times [n + 1] \times \{0, 1, \star\} \rightarrow \{0, 1\}^{\ell_{\text{CMD}}(N)}$$

such that for every $1 \leq i \leq n + 1$ and $b \in \{0, 1, \star\}$, $x_i = b$ if and only if $\text{CMD}(p_x(d, i, b)) = 1$.

The description of $\text{Red}(x; r)$ contains the string d and n strings $s'_1, s'_2, \dots, s'_n$ of length $\ell_{\text{DCMD}}(N) - 1$ each. Let P_{CMD} be the DLOGTIME -computable projection in Theorem 3.17. The string s'_i is chosen such that $P_{\text{CMD}}(p_x(d, i, 1), s'_i) = s_i$. (Note that $\text{CMD}(p_x(d, i, 1)) = \text{DCMD}(s_i)$, so each s'_i exists and is unique.)

Let $1 \leq i \leq |\text{Red}(x; r)|$. To compute the i -th bit of $\text{Red}(x; r)$, we first “locate” i by computing $k := \lfloor \frac{i-1}{\ell_{\text{DCMD}}(N)} \rfloor + 1$ and $j := i - \ell_{\text{DCMD}}(N)(k - 1)$. Now, the i -th bit of $\text{Red}(x; r)$ is the j -th bit of s_k . We can simply calculate the j -th bit of $P_{\text{CMD}}(p(d, i, 1), s'_i)$, which takes $\lambda \log \tilde{N}$ time for some absolute constant $\lambda > 0$.

It follows that whenever $\text{L-K}^t(x) \leq n - c' \log n$, regardless of the random bits r we choose, there is a description that allows us to quickly retrieve each bit of $\text{Red}(x; r)$. Moreover, the description has length $n - c' \log n + n(\ell_{\text{DCMD}}(N) - 1) = \tilde{N} - c' \log n$. If the constant c' is big enough compared with c , then $\text{Red}(x; r)$ is a Yes instance of $\text{MINK}^{t'}[\tilde{N} - c \log \tilde{N}]$. $\square$

Note that for $c > \lambda$, $\text{MINK}^{t'}[n - c \log n]$ reduces to $\text{MKTP}[n - (c - \lambda) \log n]$ via the identity mapping. (See the proof of Theorem 4.16.) Therefore, Theorem 4.15 shows a one-sided mapping reduction from some version of MINKT to some version of MKTP . To the best of our knowledge, this reduction is the first result of its kind.

Moreover, Theorem 4.15 demonstrates the *robustness* of meta-complexity w.r.t. the zero-error average-case complexity. In particular:

Theorem 4.16. *Let $t(\cdot)$ be a fixed polynomial such that $t(n) > 2n$. The following are equivalent:*

1. *There is a constant $c > 0$ such that $\text{NC}^1\text{-MINK}^t[n - c \log n] \in \text{AvgZPP}$.*
2. *There is a constant $c > 0$ such that $\text{L-MINK}^t[n - c \log n] \in \text{AvgZPP}$.*
3. *There is a constant $c > 0$ such that $\text{MINK}^{t'}[n - c \log n] \in \text{AvgZPP}$, where $t'(n) = \lambda \log n$ is defined above.*

Moreover, the above items are implied by the following items:

4. *There is a constant $c > 0$ such that $\text{MKTP}[n - c \log n] \in \text{AvgZPP}$.*

Proof. (4) $\implies$ (3): It suffices to show that for every $c > 0$, the identity mapping reduces $\text{MINK}^{t'}[n - c' \log n]$ to $\text{MKTP}[n - c \log n]$ where $c' = c + \lambda$. Let $x \in \text{MINK}^{t'}[n - c' \log n] \cap \{0, 1\}^n$, and d be a description of length $n - c' \log n$ witnessing the fact that $K^{t'}(x) \leq n - c' \log n$. Since $(d, t'(n))$ is also a witness that $\text{KT}(x) \leq n - c \log n$, we have $x \in \text{MKTP}[n - c \log n]$.

(3) $\implies$ (2): By Theorem 4.15 and Fact 4.14.

(3) $\implies$ (1): Note that the only property of L used in the proof of Theorem 4.15 is that CMD is hard for L . (In other words, $\mathsf{L} \subseteq \oplus \mathsf{L}$.) As CMD is also hard for NC^1 , the proof of Theorem 4.15 is also true for L - MINKT replaced by NC^1 - MINKT .

(2) $\implies$ (3): Every machine that runs in $t'(n)$ time also runs in $t'(n)$ space. Therefore, for every $c > 0$, the identity mapping reduces $\text{MINK}^{t'}[n - c' \log n]$ to L - $\text{MINK}^{t_1}[n - c \log n]$, where $t_1(n) = 2^{O(t'(n))}$. We can use a padding trick [LP20, Theorem 5.6] to reduce L - MINK^{t_1} to L - MINK^t .

(1) $\implies$ (3): The same argument as (2) $\implies$ (3) also works for NC^1 - K^t . $\square$

5 Tighter Connections between Meta-Complexity and One-Way Functions

In this section, we present a tighter connection between the hardness of MINKT (or MKTP) and the maximum security of *weak* one-way functions. We first define the *security* of weak one-way functions.

Definition 5.1. Let $f : \{0, 1\}^n \rightarrow \{0, 1\}^n$ be a polynomial-time computable function. We say f is a weak one-way function *with security* $S(n)$ if there is a polynomial $p(\cdot)$ such that for every circuit C of size $S(n)$,

$$\Pr_{\mathbf{x} \leftarrow \mathcal{U}_n} [C(f(\mathbf{x})) \in f^{-1}(f(\mathbf{x}))] \leq 1 - \frac{1}{p(n)}.$$

Our main results are as follows.

Theorem 5.2. Let $S(n)$ be any monotone function such that $S(n + O(\log^2 n)) \leq S(n) \cdot n^{O(\log n)}$. The following are equivalent:

- (a) There is a weak one-way function with security $S(n) \cdot n^{\Theta(\log n)}$.
- (b) There are polynomials p, t such that the search version of K^t requires $S(n) \cdot n^{\Theta(\log n)}$ size to compute on a $1 - 1/p(n)$ fraction of inputs.
- (c) For every constant $\lambda > 0$, there are polynomials p, t such that K^t requires $S(n) \cdot n^{\Theta(\log n)}$ size to $(\lambda \log n)$ -approximate on a $1 - 1/p(n)$ fraction of inputs.

Theorem 5.3. Suppose there is a weak one-way function f with security $2^{\Omega(n)}$ computable in DLOGTIME . Then there is a polynomial p such that KT requires $2^{\Omega(n)}$ size to compute on a $1 - 1/p(n)$ fraction of inputs.

Remark 5.4. A few remarks are in order.

- We only consider non-uniform adversaries in this section. The reason is that we will use Impagliazzo's hardcore lemma (Lemma A.1), which only works for non-uniform adversaries. We remark that there are hardcore lemmas that also work for uniform adversaries: if there is no time- t' algorithm that inverts a weak one-way function on a $1 - o(1)$ fraction of inputs, then there is no time- t algorithm that non-trivially inverts every hardcore of the same one-way function. However, we do not know whether the dependence of t' on t is tight. Theorem 4.5 of [VZ13] achieves $t' = \text{poly}(t)$, but we need $t' = t \cdot \text{polylog}(t)$. We leave this issue for future work.

- Our equivalence only holds for *weak* one-way functions. Indeed, it is an open problem whether the existence of *exponentially*-hard weak one-way functions is equivalent to the existence of *exponentially*-hard strong one-way functions [GIL⁺90]. Yao’s hardness amplification theorem (Theorem 3.12) blows up the input length by a polynomial factor. Therefore given a $2^{\Omega(n)}$ -hard weak one-way function, it only produces a $2^{n^{\Omega(1)}}$ -hard strong one-way function.
- Our result for KT (Theorem 5.3) is weaker than our result for K^t . In particular, suppose the one-way function has security $2^{\alpha n}$, we can only show that KT requires $2^{\beta n}$ size on average, for some constant β that is much smaller than α .
- The $n^{\Theta(\log n)}$ overhead in Theorem 5.2 arises from the use of *extractors* and *hardcore functions* in our construction. Specifically, we rely on explicit extractors that extract all min-entropy, and the best known such construction has seed length $O(\log^2 n)$ [GUV09]. In addition, the hardcore function used in Theorem 5.11 requires a seed length of $O(k \log(n/\epsilon))$, which is also $O(\log^2 n)$ under our parameter setting. These result in the $n^{\Theta(\log n)}$ factor in our main theorem.

We rely on the construction of condEP-PRGs from weak one-way functions in [YLW15, LP20], thus we structure this section as follows. In Section 5.1, we define *extractors* and *hardcore functions*, which are technical building blocks of the construction. In Section 5.2, we describe the construction in [YLW15, LP20]. (The correctness of this construction is proved in Appendix A.) The proofs of Theorem 5.2 and 5.3 appear in Section 5.3 and 5.4 respectively.

5.1 Technical Building Blocks

5.1.1 Extractors

Definition 5.5. A function $\text{Ext} : \{0, 1\}^n \times \{0, 1\}^d \rightarrow \{0, 1\}^m$ is a (k, ϵ) -extractor if for every random variable X over $\{0, 1\}^n$ such that $H_\infty(X) \geq k$, the statistical distance between $\text{Ext}(X, \mathcal{U}_d)$ and $\mathcal{U}_m$ is at most ϵ .

Moreover, Ext is a *strong* (k, ϵ) -extractor if for every random variable X as above, the statistical distance between $\text{Ext}(X, \mathcal{U}_d)$ and $\mathcal{U}_m$ is at most ϵ , even *conditioned on the seed*. That is, the statistical distance between the following two distributions is at most ϵ :

$$\mathcal{D}_1 := (\mathbf{r} \circ \text{Ext}(\mathbf{x}, \mathbf{r}) \mid \mathbf{r} \leftarrow \mathcal{U}_d, \mathbf{x} \leftarrow X), \text{ and } \mathcal{D}_2 := \mathcal{U}_{d+m}.$$

5.1.2 Hardcore Functions

Definition 5.6. Let $\epsilon = \epsilon(n) > 0$, $L = L(n) \leq \text{poly}(n)$, $\text{HC} : \{0, 1\}^n \times \{0, 1\}^d \rightarrow \{0, 1\}^m$ be a function, and R be a probabilistic oracle algorithm. We say HC is a *hardcore function* with reconstruction algorithm R , distinguishing probability ϵ , and list size L , if the following holds.

- On every oracle $\mathcal{O}$, $R^{\mathcal{O}}$ outputs a list of L strings of length n .
- For every string x and every oracle $\mathcal{O}$ that ϵ -distinguishes $\mathcal{U}_d \circ \text{HC}(x, \mathcal{U}_d)$ from $\mathcal{U}_{d+m}$, x is in the list outputted by $R^{\mathcal{O}}$ w.p. $\geq 1/2$.

Our definition of hardcore functions indeed implies the standard definition in [GL89]:

Fact 5.7. Let $\text{HC} : \{0, 1\}^n \times \{0, 1\}^d \rightarrow \{0, 1\}^m$ be a hardcore function with a $\text{poly}(n)$ -time reconstruction algorithm, distinguishing probability $\epsilon = 1/\text{poly}(n)$, and list size $L \leq \text{poly}(n)$.

Let f be any one-way function, $\mathbf{x} \leftarrow \mathcal{U}_n$, and $\mathbf{r} \leftarrow \mathcal{U}_d$. No polynomial-size adversary can 2ϵ -distinguish the distribution $f(\mathbf{x}) \circ \mathbf{r} \circ \text{HC}(\mathbf{x}, \mathbf{r})$ from the distribution $f(\mathbf{x}) \circ \mathbf{r} \circ \mathcal{U}_m$.

Proof. Let $\mathcal{A}$ be an adversary of size $\text{poly}(n)$ that 2ϵ -distinguishes the distribution $f(\mathbf{x}) \circ \mathbf{r} \circ \text{HC}(\mathbf{x}, \mathbf{r})$ from $f(\mathbf{x}) \circ \mathbf{r} \circ \mathcal{U}_m$. Say $x \in \{0, 1\}^n$ is *good* if $\mathcal{A}$ can ϵ -distinguish $f(x) \circ \mathbf{r} \circ \text{HC}(x, \mathbf{r})$ from $f(x) \circ \mathbf{r} \circ \mathcal{U}_m$. Then by a Markov bound, at least an ϵ fraction of inputs x are good. We will use $\mathcal{A}$ to invert $f(x)$ on every good input x in probabilistic polynomial time. Our inversion algorithm will have success probability $1/2$ on a good x ; as $(\epsilon/2) > 1/\text{poly}(n)$, this contradicts the one-wayness of f .

On input $y = f(x)$ where x is good, define the oracle

$$\mathcal{O}(z) := \mathcal{A}(y, z).$$

Then $\mathcal{O}$ can ϵ -distinguish $\mathcal{U}_d \circ \text{HC}(x, \mathcal{U}_d)$ from $\mathcal{U}_{d+m}$. The reconstruction algorithm $R^{\mathcal{O}}$ outputs a list of size $\text{poly}(n)$ which contains x . We could easily find and output any element x' in this list such that $f(x') = y$. With probability $1/2$ over the internal randomness of R , we invert y successfully. $\square$

5.2 CondEP-PRGs from Weak One-Way Functions

In this section, we present the following construction from weak one-way functions to condEP-PRGs.

Construction 5.8 ([YLW15, LP20]). Let $0 < \epsilon < \frac{1}{10n^2}$ be the desired security parameter of the condEP-PRG (i.e., it should be $O(\epsilon)$ -indistinguishable from uniformly random strings). Let $\delta > 0$ and f be a weak one-way function that is hard to invert on a $(1 - \delta)$ fraction of inputs. Let $\alpha > 0$ be the desired stretch of our condEP-PRG. Suppose we have the following objects:

- For every k , a strong (k, ϵ) -extractor $\text{Ext} : \{0, 1\}^n \times \{0, 1\}^d \rightarrow \{0, 1\}^m$ with optimal output length, where $d := d_{\text{Ext}}(n, \epsilon)$ and $m := k - 2\log(1/\epsilon) - O(1)$. We write the extractor as $\text{Ext}^{(k)}$ if we need to emphasize the min-entropy parameter k .
- For $k_{\text{HC}} := \alpha + \log(n/\delta) + 4\log(1/\epsilon) + O(1)$, a hardcore function $\text{HC} : \{0, 1\}^n \times \{0, 1\}^{d'} \rightarrow \{0, 1\}^{k_{\text{HC}}}$ with $\text{poly}(n/\epsilon)$ -time reconstruction algorithm R , distinguishing probability ϵ , and list size $L \leq \text{poly}(n/\epsilon)$, where $d' := d_{\text{HC}}(n, k_{\text{HC}}, \epsilon)$.

Let $G_{n,r} : \{0, 1\}^n \times \{0, 1\}^d \times \{0, 1\}^d \times \{0, 1\}^{d'} \rightarrow \{0, 1\}^{n+2d+d'+\alpha}$ be the following construction:

$$G_{n,r}(x, z_1, z_2, z_3) := z_1 \circ \text{Ext}^{(r-1)}(x, z_1) \circ z_2 \circ \text{Ext}^{(\lfloor n-r-\log(2n/\delta) \rfloor)}(f(x), z_2) \circ z_3 \circ \text{HC}(x, z_3).$$

Theorem 5.9. Let $\epsilon, \delta, \alpha, f$ be defined as in Construction 5.8. If $\epsilon \geq 1/\text{poly}(n)$ and $L \leq \text{poly}(n)$, then there is a function $r : \mathbb{N} \rightarrow \mathbb{N}$ such that $G = \{G_{n,r(n)}\}_{n \in \mathbb{N}}$ is a condEP-PRG with stretch α and security 4ϵ .

More precisely, let $\tilde{n} = n + 2d + d'$. Suppose that for every subset $\mathcal{D} \subseteq \{0, 1\}^{\tilde{n}}$ such that $H(G(\mathcal{D})) \geq \tilde{n} - \Omega(\log(\frac{n}{\delta\epsilon}))$ and every k , there is an adversary of size s that 4ϵ -distinguishes $G_{n,k}(\mathcal{D})$ from the uniform random distribution. Then there is an adversary of size $s \cdot \text{poly}(nL/\epsilon)$ that inverts f on a $1 - \delta$ fraction of inputs.

The proof basically follows from [LP20], and we present a self-contained proof in Appendix A. However, there are two major differences between our proof and the proof in [LP20]:

- We replace the extractors and hardcore functions with better constructions. In particular, our extractors and hardcore functions in Section 5.3 only require $O(\log^2 n)$ random bits.

- More importantly, in the very beginning, we need to transform the weak one-way function into a strong one. [LP20] uses hardness amplification (Theorem 3.12) to implement this step. However, Theorem 3.12 does not preserve *exponential* security, therefore we use *Impagliazzo's hardcore lemma* [Imp95a] instead. We only obtain a strong one-way function on a “hardcore” distribution of inputs (instead of the uniform distribution), but this already suffices for our purpose.

5.2.1 Warm-Up: Proof of Theorem 3.14

Theorem 5.9 immediately implies Theorem 3.14.

Theorem 3.14 ([LP20]). *There is a function EP-PRG computable in ALOGTIME such that the following holds. For any one-way function $f : \{0,1\}^* \rightarrow \{0,1\}^*$ and any constant $\gamma > 0$, let $G(x, z) = \text{EP-PRG}(\gamma, x, f(x), z)$, then G is a condEP-PRG with stretch $\gamma \log n$ and security $1/n^\gamma$.*

We first introduce the (very simple) extractors and hardcore functions used in [YLW15, LP20].

- The extractors are derived from the *leftover hash lemma* [HILL99]. (See also [Vad12, Theorem 6.18].) Let $h : \{0,1\}^n \times \{0,1\}^d \rightarrow \{0,1\}^m$ be a pairwise independent family of hash functions, where $d = O(n + m)$, then for every k, ϵ such that $m = k - 2 \log(1/\epsilon)$, h is also a strong (k, ϵ) -extractor.

We instantiate the pairwise independent hash family by Toeplitz matrices.¹⁷ More precisely, our keys will have length $d := n + m - 1$, and every $key \in \{0,1\}^{n+m-1}$ corresponds to a Toeplitz matrix. For every $1 \leq i \leq m$ and every input $x \in \{0,1\}^n$, the i -th output of $H(x, key)$ is the inner product of x and $key_{i \sim (i+n-1)}$ (the substring of key from the i -th bit to the $(i + n - 1)$ -th bit) in $\text{GF}(2)$. In other words, $\text{Ext}(x, key)$ is the concatenation of $\langle x, key_{i \sim (i+n-1)} \rangle$ for each i , where $\langle \cdot, \cdot \rangle$ denotes inner product.

- Let $\text{GL} : \{0,1\}^n \times \{0,1\}^d \rightarrow \{0,1\}^k$ be the Goldreich–Levin hardcore function.

In [GL89], GL is defined in terms of Toeplitz matrices (again). Let $d := n + k - 1$. For every $x \in \{0,1\}^n$, $r \in \{0,1\}^d$ and $1 \leq i \leq k$, the i -th output bit of $\text{GL}(x, r)$ is the inner product of x and $r_{i \sim (i+n-1)}$ in $\text{GF}(2)$. Also, it is shown in [GL89] that for every $\epsilon > 0$, GL is a hardcore function with distinguishing probability ϵ and list size $\text{poly}(n \cdot 2^k/\epsilon)$.

Proof Sketch of Theorem 3.14. We can plug the parameters $\epsilon := \frac{1}{4n^\gamma}$, $\alpha := \gamma \log n$, $\delta := 1/2$ into Theorem 5.9. The list size of GL is $L \leq \text{poly}(n)$. Theorem 5.9 gives us a function $r : \mathbb{N} \rightarrow \mathbb{N}$ such that $\{G_{n,r(n)}\}$ is a condEP-PRG with stretch $\gamma \log n$ and security $1/n^\gamma$. We can easily construct a uniform condEP-PRG with essentially the same stretch and security: We parse the input as an integer $r \leq n$, a string x of length n , and some garbage w . Then we output $G_{n,r}(x) \circ w$.

Now we implement EP-PRG in alternating time $c \log n$, for some absolute constant $c > 0$ independent of γ . On input $(\gamma, x, f(x), z, i)$, we want to compute the i -th output bit of our condEP-PRG. This bit is either equal to some input bit, or the inner product of two length- n sub-strings of the input. It is easy to implement either case in alternating $O(\log n)$ time. $\square$

5.3 Proof of Theorem 5.2

To prove Theorem 5.2, we replace the leftover hash lemma and GL by extractors and hardcore functions with very short seed length:

¹⁷An $n \times m$ matrix M is *Toeplitz* if $M_{i,j} = M_{i+1,j+1}$ holds for every $1 \leq i < n$, $1 \leq j < m$. We can represent a Toeplitz matrix by $n + m - 1$ elements, namely the elements in the first row and the first column.

Theorem 5.10 ([GUV09, Theorem 5.14]). Let $d_{\text{Ext}}(n, \epsilon) := O(\log n \cdot \log(n/\epsilon))$, then for every $1 \leq k \leq n$ and $\epsilon > 0$, there is a strong (k, ϵ) -extractor $\text{Ext} : \{0, 1\}^n \times \{0, 1\}^{d_{\text{Ext}}(n, \epsilon)} \rightarrow \{0, 1\}^m$, where $m = k - 2\log(1/\epsilon) - O(1)$ is optimal.

We observe that the “ k -wise direct product generator” used in [Hir20b, Hir20a] is a good hardcore function:

Theorem 5.11. Let $d_{\text{HC}}(n, k, \epsilon) := O(k \log(n/\epsilon))$, then there is a hardcore function $\text{HC} : \{0, 1\}^n \times \{0, 1\}^{d_{\text{HC}}(n, k, \epsilon)} \rightarrow \{0, 1\}^k$ with a $\text{poly}(n2^k/\epsilon)$ -time reconstruction algorithm R , distinguishing probability ϵ , and list size $L \leq 2^k \cdot \text{poly}(k/\epsilon)$.

Proof Sketch. Consider the function $\text{DP} : \{0, 1\}^n \times \{0, 1\}^d \rightarrow \{0, 1\}^{d+k}$ defined in [Hir20a, Theorem 7.1]. The first d bits of $\text{DP}(x, z)$ is always equal to z , and we let $\text{HC}(x, z)$ be the remaining k bits of $\text{DP}(x, z)$.

In [Hir20a], the reconstruction algorithm is stated as $R^\mathcal{O} : \{0, 1\}^a \times \{0, 1\}^r \rightarrow \{0, 1\}^n$. Here, $a \leq k + O(\log(k/\epsilon))$ is the “advice complexity” of DP , the first a input bits correspond to the advice, and the remaining $r = \text{poly}(n/\epsilon)$ input bits are random coins used by R . For every $x \in \{0, 1\}^n$ and every oracle $\mathcal{O}$ that ϵ -distinguishes $\text{DP}(x, \mathcal{U}_d)$ from $\mathcal{U}_{d+k}$, we have

$$\Pr_{\mathbf{w} \leftarrow \mathcal{U}_r} [\exists \alpha \in \{0, 1\}^a, R^\mathcal{O}(\alpha, \mathbf{w}) = x] \geq 3/4.$$

Our reconstruction algorithm simply samples a random $\mathbf{w} \leftarrow \mathcal{U}_r$ and outputs $R^\mathcal{O}(\alpha, \mathbf{w})$ for every $\alpha \in \{0, 1\}^a$. It follows that the list size is $L(n, k, \epsilon) \leq 2^a \leq 2^k \text{poly}(k/\epsilon)$. $\square$

Now we use Construction 5.8 to prove Theorem 5.2.

Theorem 5.2. Let $S(n)$ be any monotone function such that $S(n + O(\log^2 n)) \leq S(n) \cdot n^{O(\log n)}$. The following are equivalent:

- (a) There is a weak one-way function with security $S(n) \cdot n^{\Theta(\log n)}$.
- (b) There are polynomials p, t such that the search version of K^t requires $S(n) \cdot n^{\Theta(\log n)}$ size to compute on a $1 - 1/p(n)$ fraction of inputs.
- (c) For every constant $\lambda > 0$, there are polynomials p, t such that K^t requires $S(n) \cdot n^{\Theta(\log n)}$ size to $(\lambda \log n)$ -approximate on a $1 - 1/p(n)$ fraction of inputs.

Proof Sketch. (c) $\implies$ (b) is trivial.

(b) $\implies$ (a): Suppose that the search version of K^t requires $S(n) \cdot n^{\Theta(\log n)}$ size to solve on a $1/p(n)$ fraction of inputs, where p is a polynomial. The construction in [LP20, Section 4] shows that there is a weak one-way function f such that every adversary of size $S(n) \cdot n^{\Theta(\log n)}$ only inverts an $1 - 1/q(n)$ fraction of inputs, where $q(n) := O(n \cdot p(n)^2)$.

(a) $\implies$ (c): Suppose there is a constant $\lambda > 0$ such that, for every polynomial p , there is an algorithm of size $S(n) \cdot n^{\Theta(\log n)}$ that approximates K^t on a $1 - 1/p(n)$ fraction of inputs, within an additive error of $\lambda \log n$. Let f be a candidate weak one-way function, $\delta := 1/q(n)$ for any polynomial q , and $\epsilon := 1/n^2$. Let $\alpha := (\lambda + C) \log n$ be the stretch of the condEP-PRG we construct, where C is a large absolute constant. Let $r : \mathbb{N} \rightarrow \mathbb{N}$ be any function. Consider the function $G_{n, r(n)}$ in Construction 5.8 where the input length of $G_{n, r(n)}$ is

$$\tilde{n} := n + 2d_{\text{Ext}}(n, \epsilon) + d_{\text{HC}}(n, O(\log(n/(\delta\epsilon))), \epsilon) = n + O(\log^2 n).$$

Suppose G runs in $t(\tilde{n})$ time. Then every output y of G satisfies $K^{t(|y|)}(y) \leq \tilde{n} - (\lambda + 2) \log \tilde{n}$.

Consider any sequence of subsets $\mathcal{E} = \{\mathcal{E}_n \subseteq \{0, 1\}^{\tilde{n}}\}$ such that $H(G_{n, r(n)}(\mathcal{E}_n)) \geq \tilde{n} - \Omega(\log n)$. The same argument as in Lemma 4.7 shows that there is an adversary of size

$$S(\tilde{n}) \cdot \tilde{n}^{\Theta(\log \tilde{n})} \leq S(n) \cdot n^{\Theta(\log n)}$$

that 4ϵ -distinguishes $G_{n, r(n)}(\mathcal{E}_n)$ from the uniform distribution. It follows from Theorem 5.9 that there is an adversary of size $S(n) \cdot n^{\Theta(\log n)}$ that inverts f on a $1 - \delta$ fraction of inputs. Therefore, there is no weak one-way function with hardness $S(n) \cdot n^{\Theta(\log n)}$. $\square$

5.4 Proof of Theorem 5.3

To prove Theorem 5.3, we need a family of universal hash functions that admit very efficient randomized encodings, as constructed in [IKOS08, App17]. In [App17], it was also proved that such hash functions are good extractors (by the leftover hash lemma) and hardcore functions (based on previous works [HMS04, BIO14]).

In the construction of [AIK06], for a (Boolean) function computable by a parity branching program of size S , its randomized encoding needs at least $\Omega(S^2)$ additional random input bits. Even worse, if such a function has m output bits, the randomized encoding requires $\Omega(mS^2)$ random input bits. However, to prove Theorem 5.3, we need to preserve *exponential* hardness of our one-way function, which means our extractors and hardcore functions can only have $O(n)$ random input bits. This is exactly what [App17] does. In particular, for a “skew” circuit C of size S and possibly many outputs, the randomized encoding of C in [App17] only requires $O(S)$ additional random inputs. Such circuits of linear size can already compute many powerful objects, e.g. universal hash functions [IKOS08].

5.4.1 Randomized Encodings for Skew Circuits

We introduce the randomized encodings in [App17] in more detail.

We consider circuits that consist of AND and XOR gates of fan-in 2 and has multiple output gates. Let C be such a circuit and X be a subset of input variables. (For example, let $C : \{0, 1\}^n \times \{0, 1\}^d \rightarrow \{0, 1\}^m$, we may think of X as the last d input variables.) We say C is *skew* with respect to X , if every AND gate in C has at least one child labeled by a constant or a variable in X . In particular, this implies that if we substitute the variables in X by (arbitrary) constants, the function that C computes is a *linear* function on variables not in X — each output bit is simply the XOR of a subset of these variables.

Let $C : \{0, 1\}^n \times \{0, 1\}^d \rightarrow \{0, 1\}^{d+m}$ be a skew circuit w.r.t. the last d inputs such that the first d outputs of C is always equal to the last d inputs of C .¹⁸ Let s be the number of internal (i.e. non-input, non-output) gates of C . The randomized encoding of C , denoted by $\tilde{C}$, is a function $\tilde{C} : \{0, 1\}^n \times \{0, 1\}^{d+s} \rightarrow \{0, 1\}^{d+m+s}$ defined as follows:

- The inputs of $\tilde{C}$ are $x \in \{0, 1\}^n$, $w \in \{0, 1\}^d$, and $r \in \{0, 1\}^s$.
- For each (input, internal, or output) gate $g \in C$, we associate a bit $r(g)$ with it. Each input gate is associated with its input value (i.e. $r(g) = x_i$ or w_i), the i -th internal gate is associated with $r(g) = r_i$, and every output gate is associated with $r(g) = 0$.
- The first d outputs of $\tilde{C}$ are simply w . The remaining $m + s$ outputs correspond to the internal gates and output gates of C . Let the i -th such gate be $g_i = g_j \nabla g_k$ (where $\nabla \in \{\text{AND}, \text{XOR}\}$), then the i -th output is $r(g_i) \text{ XOR } (r(g_j) \nabla r(g_k))$.

It was shown in [App17] that $\tilde{C}$ is a randomized encoding of C in the following sense. The space of encodings $\{0, 1\}^{d+m+s}$ can be partitioned into $\{D_y\}_{y \in \{0, 1\}^{d+m}}$ so that for every fixed $x \in \{0, 1\}^n$ and $w \in \{0, 1\}^d$, letting $y := C(x, w)$, the mapping $\tilde{C}(x, w, -) : \{0, 1\}^s \rightarrow \{0, 1\}^{d+m+s}$ is a bijection between $\{0, 1\}^s$ and D_y . Let $\text{Sim}(y)$ denote the uniform distribution over D_y (Sim stands for “simulator”), then Sim can be computed (i.e., sampled) in probabilistic polynomial time, and $\text{Sim}(y)$ is the same distribution as $\tilde{C}(x, w, \mathcal{U}_s)$.

Theorem 5.12. *Let $m := n + \gamma \log n$, $C : \{0, 1\}^{n'} \times \{0, 1\}^d \rightarrow \{0, 1\}^{d+m}$ be a circuit with s internal gates that is skew w.r.t. its second argument, and $g : \{0, 1\}^n \rightarrow \{0, 1\}^{n'}$ be an arbitrary function. Suppose that the function $G : \{0, 1\}^n \times \{0, 1\}^d \rightarrow \{0, 1\}^{d+m}$ defined as*

$$G(x, w) := C(g(x), w)$$

¹⁸That is, we pad the last d inputs at the beginning of our outputs, and the remaining m output bits are the “real” outputs of C . This is a technical restriction on C to ensure its randomized encoding exists.

is a condEP-PRG with stretch $\gamma \log n$ and security ϵ . Let $\tilde{C} : \{0, 1\}^{n'} \times \{0, 1\}^{d+s} \rightarrow \{0, 1\}^{d+m+s}$ be the randomized encoding of C . Then the function $\tilde{G}(x, w, r) := \tilde{C}(g(x), w, r)$ is a condEP-PRG with stretch $\gamma \log n$ and security ϵ .

Proof. The stretch of $\tilde{G}$ is easy to verify.

Let $\mathcal{E} \subseteq \{0, 1\}^n$ be the “event” such that (Pseudorandomness) and (Entropy-Preservation) in Definition 3.13 hold. Let $\tilde{\mathcal{E}} := \mathcal{E} \times \{0, 1\}^s$. We first observe that $\tilde{G}(\tilde{\mathcal{E}})$ is the same distribution as $\text{Sim}(G(\mathcal{E}))$. In fact, $\tilde{G}(\tilde{\mathcal{E}})$ is the distribution where we choose $(\mathbf{x}, \mathbf{w}) \leftarrow \mathcal{E}$, $\mathbf{r} \leftarrow \mathcal{U}_s$, and output $\tilde{C}(g(\mathbf{x}), \mathbf{w}; \mathbf{r})$; this is the same distribution as $\text{Sim}(C(g(\mathbf{x}), \mathbf{w})) = \text{Sim}(G(\mathcal{E}))$.

Now we verify that $\tilde{G}$ and $\tilde{\mathcal{E}}$ satisfies Definition 3.13.

Pseudorandomness. Suppose, for the sake of contradiction, that there is a PPT adversary $\mathcal{A}'$ such that

$$\Pr[\mathcal{A}'(\tilde{G}(\tilde{\mathcal{E}}))] - \Pr[\mathcal{A}'(\mathcal{U}_{d+m+s})] \geq \epsilon.$$

Consider an adversary $\mathcal{A}$ that on input y , samples $\tilde{\mathbf{y}} \leftarrow \text{Sim}(y)$ and returns $\mathcal{A}'(\tilde{\mathbf{y}})$. If $\mathbf{y} \leftarrow G(\mathcal{E})$, then $\tilde{\mathbf{y}}$ is distributed according to $\tilde{G}(\tilde{\mathcal{E}})$. On the other hand, if $\mathbf{y} \leftarrow \mathcal{U}_{d+m}$, since $\{D_y\}$ is a partition of $\{0, 1\}^{d+m+s}$, we have that $\tilde{\mathbf{y}}$ is distributed according to $\mathcal{U}_{d+m+s}$. Hence, $\mathcal{A}$ distinguishes $G(\mathcal{E})$ from the uniform distribution with advantage $\geq \epsilon$ as well, contradicting the security of G .

Entropy-preservation. Note that $H(\tilde{G}(\tilde{\mathcal{E}})) = H(\text{Sim}(G(\mathcal{E}))) = H(G(\mathcal{E})) + s$. Hence if $H(G(\mathcal{E})) \geq n + d - \alpha$ for some $\alpha \geq 1$, then $H(\tilde{G}(\tilde{\mathcal{E}})) \geq n + d + s - \alpha$ as well. $\square$

5.4.2 Highly-Uniform Linear-Size Hash Functions

As we are dealing with KT complexity, we will need the randomized encoding to be computable in DLOGTIME. Therefore, our skew circuits need to be *very uniform*. We state our definition of *uniform skew circuits* as follows; it is easy to see that if a family of skew circuits $\{C_n\}$ is uniform, then their randomized encodings can indeed be computed in DLOGTIME.

Definition 5.13 (Uniform Skew Circuits). Let $C = \{C_n : \{0, 1\}^n \times \{0, 1\}^{d(n)} \rightarrow \{0, 1\}^{s(n)}\}$ be a family of skew circuits, where $d(n)$ and $s(n)$ are computable in time $O(\log n)$. Moreover, assume that the fan-out of every gate is at most 2, and the last $s(n)$ gates (i.e., gates with the largest indices) are output gates.

We say that C is a *uniform* family of skew circuits, if there is an algorithm $\mathcal{A}$ with time complexity linear in its input length that, on inputs n, i (in binary), outputs the information about the i -th gate in C_n . This includes the gate type (input, AND, or XOR), indices of its input gates (if they exist), and indices of the (at most 2) gates it feeds to.

Remark 5.14. It may seem strange that we need to output not only predecessors but also successors of each gate. The reason is that in [IKOS08], we will need to *reverse* each wire when we transform an encoding circuit to an “exposure resilient function”. In particular, after that construction, the predecessors of each gate will become their previous successors. See Appendix B.4 for details.

We need a family of universal hash functions $\mathcal{H} = \{h_{n,m} : \{0, 1\}^n \times \{0, 1\}^k \rightarrow \{0, 1\}^m\}$ in [IKOS08], where $k = O(n + m)$. This family has the following important property: $\mathcal{H}$ can be computed by a family of linear-size uniform circuits that are skew w.r.t. the second argument (i.e. the last k bits).

Theorem 5.15. *For every integer n, m where $m = O(n)$, there exists an integer $k = O(n)$ and a family of universal hash functions $\{h_{n,m} : \{0, 1\}^n \times \{0, 1\}^k \rightarrow \{0, 1\}^m\}$ such that $h_{n,m}$ can be computed by a uniform family of linear-size circuits that are skew w.r.t. the second argument.*

In [IKOS08], the authors showed that $\mathcal{H}$ can be computed by a family of linear-size skew circuits, but they did not show that the circuits are uniform. Therefore, we include a proof sketch of Theorem 5.15 in Appendix B, with an emphasis on the uniformity of these circuits.

By the leftover hash lemma of [HILL99], $\{h_{n,m}\}$ is a strong (k, ϵ) -extractor whenever $m = k - 2\log(1/\epsilon)$. It was proved by [BIO14] (based on [HMS04]) that $\{h_{n,m}\}$ are good hardcore functions:

Lemma 5.16. *For every $\epsilon > 0$, $h_{n,m}$ is a hardcore function with distinguishing probability ϵ and a reconstruction algorithm of $\text{poly}(2^m \cdot n/\epsilon)$ time. (As a result, the list size is also $\text{poly}(2^m \cdot n/\epsilon)$.)*

5.4.3 Proof of Theorem 5.3

Theorem 5.3. *Suppose there is a weak one-way function f with security $2^{\Omega(n)}$ computable in DLOGTIME. Then there is a polynomial p such that KT requires $2^{\Omega(n)}$ size to compute on a $1 - 1/p(n)$ fraction of inputs.*

Proof Sketch. Let $\delta = 1/\text{poly}(n)$ such that f is hard to invert on a $(1 - \delta)$ fraction of inputs. We plug the hash functions h (which are also extractors and hardcore functions) into Construction 5.8 and build a condEP-PRG $G : \{0, 1\}^{n_1} \rightarrow \{0, 1\}^{n_1 + \alpha}$ with stretch $\alpha := O(\log n)$ and security $4\epsilon \leq 1/n^{10}$. Here, since the seed length of h is $O(n)$, we have $n_1 = O(n)$. Moreover, by Theorem 5.9, the condEP-PRG is 4ϵ -indistinguishable from the uniform distribution by $2^{\Omega(n)}$ -size adversaries.

As the hash functions admit a uniform family of skew circuits, the following is true: There is a (uniform) circuit C such that $G(x, z) = C(x, f(x), z)$, and C is a size- $O(n)$ skew circuit w.r.t. the z argument. Let $\tilde{C}$ be the randomized encoding of C and define $\tilde{G}(x, z, r) = \tilde{C}(x, f(x), z, r)$ where $r \in \{0, 1\}^{|C|}$. Then by Theorem 5.12, the function $\tilde{G} : \{0, 1\}^{n_2} \rightarrow \{0, 1\}^{n_2 + \alpha}$ is also a condEP-PRG. Here, since the size of C is $O(n)$, we have $n_2 = O(n)$. Further note that $\tilde{G}$ is computable in DLOGTIME, hence every output of $\tilde{G}$ has non-trivial KT complexity. Since $\tilde{G}$ is 4ϵ -indistinguishable from the uniform distribution by $2^{\Omega(n)}$ -size adversaries, we can see that KT is hard on average. $\square$

5.5 The Perebor Hypotheses

We mention some Perebor hypotheses as further research directions. Each hypothesis states that to some extent, “Perebor,” or brute-force search, is unavoidable to solve a certain meta-complexity problem. In this paper, we only consider the (bounded-error) average-case complexity of these problems, but similar hypotheses for the worst-case or zero-error average-case complexity can also be made. We only state these hypotheses against (uniform) randomized algorithms; the corresponding hypotheses against non-uniform algorithms (i.e., circuits) will be called “non-uniform Perebor hypotheses” accordingly.

These hypotheses are inspired by, and parallel to, the “exponential time hypotheses” for satisfiability [IP01, IPZ01, CIP09]. The *exponential time hypothesis* (ETH) asserts that 3-SAT requires $2^{\epsilon n}$ time to solve, where $\epsilon > 0$ is some absolute constant and n is the number of variables. The *strong exponential time hypothesis* (SETH) asserts that for *any* constant $\epsilon > 0$, CNF-SAT requires $2^{(1-\epsilon)n}$ time to solve. There is a large body of work on these two hypotheses and their variants; in particular, SETH has been a central hypothesis in fine-grained complexity [Wil18].

We believe that the future study of these Perebor hypotheses will bring us more insights into complexity theory, similar to what the study of ETH and SETH has brought us.

The weak Perebor hypotheses. We introduce the following two hypotheses for K^t and KT:

Hypothesis 5.17 (Weak Perebor Hypothesis for K^t). *There are a polynomial $t(n) \geq 2n$ and an absolute constant $c \geq 1$ such that the following holds. Every randomized algorithm that runs in $2^{n/c}$ time and attempts to solve K^t fails w.p. at least $1/n^c$ over a uniformly random input.*

Hypothesis 5.18 (Weak Perebor Hypothesis for KT). *There is an absolute constant $c \geq 1$ such that the following holds. Every randomized algorithm that runs in $2^{n/c}$ time and attempts to solve KT fails w.p. at least $1/n^c$ over a uniformly random input.*

Theorem 5.2 shows that the non-uniform version of Hypothesis 5.17 is equivalent to the existence of exponentially-hard weak one-way functions (against non-uniform adversaries). Theorem 5.3 shows that the non-uniform version of Hypothesis 5.18 is implied by the existence of exponentially-hard weak one-way function computable in DLOGTIME (also against non-uniform adversaries).

The strong Perebor hypotheses. We start with the following hypothesis:

Hypothesis 5.19 (Strong Perebor Hypothesis for K^t). *There are polynomials $t(n) \geq 2n$ and $p(n)$ such that for every constant $\epsilon > 0$, every probabilistic algorithm that runs in $2^{(1-\epsilon)n}$ time and attempts to solve K^t fails w.p. at least $1/p(n)$ over a uniformly random input.*

By Theorem 5.2, the non-uniform version of Hypothesis 5.19 is equivalent to the existence of weak one-way functions with hardness $2^{(1-o(1))n}$ (against non-uniform adversaries).

However, Building on Hellman [Hel80], Fiat and Naor [FN99] showed that no such one-way function exists in the *non-uniform RAM* model. In particular, for *any* function $f : \{0, 1\}^n \rightarrow \{0, 1\}^n$, there is an algorithm that runs in $2^{3n/4}$ time, uses $2^{3n/4}$ bits of advice, and inverts f at *any* point. It is conceivable that a similar attack could also be implemented in circuits, i.e. every function f could be inverted by a circuit of size $2^{99n/100}$ in the worst-case. This gives strong evidence that the non-uniform version of Hypothesis 5.19 is false. To the best of our knowledge, (the uniform version of) Hypothesis 5.19 seems secure.

Following [BGI08, Section 1.1], if we still want (non-uniform) maximum hardness, we can consider *collections* of one-way functions, which corresponds to the *conditional* (time-bounded) Kolmogorov complexity. We believe that this is the correct way to formulate the non-uniform version of strong Perebor hypothesis.

Fix a universal Turing machine U , let x, y be two strings and t be a time bound. Define $\text{cK}^t(x \mid y)$ as the length of the smallest description d such that for every $1 \leq i \leq |x| + 1$ and $b \in \{0, 1, \star\}$, $U^{d,y}(i, b)$ accepts in time t if and only if $x_i = b$. Note that the universal Turing machine is given random access to y (for free), hence d is a description of x *conditioned on* y . We assume that the default input distribution of cK^t consists of a random string x and a random string y , both of input length n . Hence the hypothesis below actually states that no non-uniform algorithm of $2^{(1-\epsilon)n}$ size can solve cK^t on input length $2n$.

Hypothesis 5.20 (Strong Perebor Hypothesis for cK^t ; Non-uniform Version). *There are polynomials $t(n) \geq 2n$ and $p(n)$ such that for every constant $\epsilon > 0$, every non-uniform algorithm of $2^{(1-\epsilon)n}$ size that attempts to solve cK^t fails on a $1/p(n)$ fraction of inputs.*

6 MCSP-Related Results

In this section, we generalize Theorem 1.1 to the case of MCSP. Throughout this section, we maintain the convention that our input is the truth table of a function $f : \{0, 1\}^n \rightarrow \{0, 1\}$, and $N = 2^n$ is the input length. We use tt to denote an input truth table. Recall that $\text{Size}(tt)$ is the circuit complexity of tt . The size of a circuit is always measured in *gates*. We consider circuits over the B_2 basis, i.e., a gate can compute any function over its 2 inputs.¹⁹

Theorem 1.3 (Informal). *The following are true:*

¹⁹Looking ahead, this is because we need Uhlig's theorem (Theorem 6.5) which is only known to hold for the B_2 basis.

- If MCSP is exponentially hard on average, then there is a (super-polynomially hard) one-way function.
- If there is an exponentially hard weak one-way function in NC^0 , then MCSP is (exponentially) hard on average.

Ideally, we would like to prove that MCSP is bounded-error hard on average if and only if there is a one-way function in DLOGTIME . However, we could only prove weaker results, since we do not have good understandings of the circuit complexity of a random Boolean function.

For KT complexity, we know that a random string x of length N is likely to satisfy that $\text{KT}(x) \in [N - O(\log N), N + O(\log N)]$. That is, N is a good estimate of the KT complexity of a random string, within additive error $\eta := O(\log N)$. It turns out that the overhead of [LP20] is $2^{O(\eta)}$, which is polynomial in N .

What about MCSP? For the maximum circuit complexity function, we only know that:

Theorem 6.1 ([FM05]). *There is a constant c such that the following is true. Let $C(n)$ be the maximum circuit complexity of any function $f : \{0, 1\}^n \rightarrow \{0, 1\}$, then $C_{\text{lb}}(n) \leq C(n) \leq C_{\text{ub}}(n)$, where*

$$C_{\text{lb}}(n) = \frac{2^n}{n} \left(1 + \frac{\log n}{n} - \frac{c}{n} \right), \text{ and } C_{\text{ub}}(n) = \frac{2^n}{n} \left(1 + \frac{3 \log n}{n} + \frac{c}{n} \right).$$

Therefore, given a random truth table tt of length $N = 2^n$, we could use any value between $C_{\text{lb}}(n)$ and $C_{\text{ub}}(n)$ as an estimate of $\text{Size}(tt)$. However, we could only prove that our additive error is $\eta := (C_{\text{ub}}(n) - C_{\text{lb}}(n)) \cdot O(n) = O(2^n \log n / n)$.²⁰ The overhead in [LP20] would be

$$2^{O(\eta)} = 2^{O\left(N \frac{\log \log N}{\log N}\right)}.$$

Nevertheless, as $\eta = o(N)$ is non-trivial, we can still achieve non-trivial results for MCSP.

Remark 6.2. Ilango encountered a similar issue in his search-to-decision reduction for MFSP (Minimum Formula Size Problem) [Ila20a]. The additive error for formula complexity is $\eta := O(N / \log \log N)$, thus Ilango only managed to show an (average-case) reduction with time complexity $2^{O(\eta)}$ unconditionally.

Comparing [Ila20a] and our work, the $2^{O(\eta)}$ factor comes from different reasons. Ilango’s algorithm runs in time $\text{poly}(t)$ where t is the number of “near-optimal” formulas for the input truth table; the current best upper bound of t for a random truth table is $2^{O(\eta)}$. In our case, we need to sample a *uniformly random circuit* (w.r.t. some encoding); let p be the probability that the truth table of a sampled circuit is equal to a given one, then the current best lower bound of p is $2^{-N - O(\eta)}$. (See Section 6.2.) It is an interesting open problem to improve either estimate.

6.1 Preliminaries

6.1.1 Extreme Hardness Amplification for One-Way Functions

We will construct a one-way function f_{MCSP} based on the assumption that MCSP is *exponentially* hard on average. However, we are only able to prove that f_{MCSP} is hard to invert on an inverse-sub-exponential fraction ($2^{-o(N)}$) of inputs. We will need the following variant of Theorem 3.12 that constructs a strong one-way function (of super-polynomial hardness) from such a one-way function that is “exponentially hard” but also “(sub-)exponentially weak”.

Theorem 6.3. *Let $p(n) = 2^{o(n)}$ and f be a polynomial-time computable length-preserving function that is exponentially hard to invert on a $1/p(n)$ fraction of inputs. In other words, there is*

²⁰The extra $O(n)$ factor is because we measure η by *bit-complexity* instead of *gate-complexity*, and every gate in the (maximum) circuit needs $O(n)$ bits to describe.

a constant $\epsilon > 0$ such that for every integer n and every randomized algorithm $\mathcal{A}$ that runs in $2^{\epsilon n}$ time,

$$\Pr_{\mathbf{x} \leftarrow \mathcal{U}_n} [\mathcal{A}(f(\mathbf{x})) \in f^{-1}(f(\mathbf{x}))] \leq 1 - 1/p(n).$$

Then there exists a one-way function.

Proof Sketch. We verify that the standard proof for Theorem 3.12 also works in our setting. We use notations in [Gol01, Theorem 2.3.2]. Let f be a candidate weak one-way function and $m(n) := n^2 \cdot p(n) < 2^{o(n)}$. By [Gol01, Theorem 2.3.2], we can construct a function g on $m(n)$ inputs bits such that the following holds. Given any adversary B that inverts g w.p. $1/q(m)$, we can construct an adversary that makes $a(n) := 2n^2 p(n) q(m(n))$ calls to B on input length $m(n)$ and inverts f w.p. $1 - 1/p(n)$.

Suppose that g is not a one-way function. Then there is a polynomial q and an adversary B that runs in $q(m)$ time and inverts g w.p. $1/q(m)$. We can invert f w.p. $1 - 1/p(n)$ by an adversary of time complexity

$$O(a(n) \cdot q(m(n))) < 2^{o(n)},$$

contradicting the hardness of f . □

6.1.2 Maximum Circuit Complexity

It will be convenient to fix an encoding of circuits into binary strings so that we can sample a uniformly random circuit with a certain description length. Fortunately, such an encoding naturally occurs in the lower bound proofs for the maximum circuit complexity, which usually use a counting argument [Sha49, FM05]: If every circuit of size $\text{LB}(n)$ can be encoded as a string of length $2^n - 1$, then there must exist an n -bit Boolean function without size- $\text{LB}(n)$ circuits.

In particular, in the lower bound proof of [FM05], the authors represented a circuit as a *stack program*. For a detailed description of stack programs, the reader is referred to [FM05]. We only need the following property of them:

Theorem 6.4. *There is a constant c such that every size- s circuit on n inputs can be encoded into a stack program of bit-length $(s + 1)(c + \log(n + s))$.*

We also need the fact that given the description of a stack program, we can compute its truth table (the truth table of the circuit corresponding to it) in polynomial time.

We define

$$C'_{\text{ub}}(n) := (C_{\text{ub}}(n) + 1)(\log(n + C_{\text{ub}}(n)) + O(1)) \leq 2^n \left(1 + \frac{2 \log n}{n} + \frac{O(1)}{n} \right).$$

By Theorem 6.4, every Boolean function over n inputs has a stack program of bit-length $C'_{\text{ub}}(n)$.

We also need the following theorem, which says that for any Boolean function f on n input bits, there is a circuit of size roughly $2^n/n$ that computes f *simultaneously on multiple inputs*.

Theorem 6.5 ([Uhl74, Uhl84]; see also [Weg87, p. 304]). *Let $f : \{0, 1\}^n \rightarrow \{0, 1\}$ be any Boolean function, r be a constant. There is a circuit C of size at most $(1 + o(1))2^n/n$ such that for every $x_1, x_2, \dots, x_r \in \{0, 1\}^n$, $C(x_1, x_2, \dots, x_r) = f(x_1) \circ f(x_2) \circ \dots \circ f(x_r)$.*

6.2 One-Way Functions from Hardness of MCSP

In this section, we construct a one-way function assuming MCSP is (exponentially) hard on average.

Theorem 6.6. Suppose that MCSP is exponentially hard on average. In particular, there is a constant $\epsilon > 0$ and a function $q(N) = 2^{o(N)}$ such that for every randomized algorithm $\mathcal{A}$ running in $2^{\epsilon N}$ time,

$$\Pr_{tt \leftarrow \mathcal{U}_N} [\mathcal{A}(tt) = \text{Size}(tt)] \leq 1 - 1/q(N).$$

Then there exists a one-way function.

Proof. By Theorem 6.3, it suffices to construct a length-preserving function f that satisfies the following one-wayness property: There is a function $p(\tilde{N}) = 2^{o(\tilde{N})}$ such that for every integer $\tilde{N}$ and every randomized algorithm $\mathcal{A}$ that runs in $2^{\epsilon \tilde{N}/10}$ time,

$$\Pr_{\mathbf{x} \leftarrow \mathcal{U}_{\tilde{N}}} [\mathcal{A}(f(\mathbf{x})) \in f^{-1}(f(\mathbf{x}))] \leq 1 - 1/p(\tilde{N}). \quad (3)$$

We first describe the construction of f . Let $\tilde{N}$ be the input length of f and n be the largest integer such that $n + C'_{\text{ub}}(n) \leq \tilde{N}$. (Recall that every Boolean function over n inputs can be represented by a circuit, or stack program, of bit-length $C'_{\text{ub}}(n)$.) The first n bits of the input denote an integer $s \leq C_{\text{ub}}(n)$, and the next $C'_{\text{ub}}(n)$ bits denote a circuit C of size at most s . If the input is invalid (e.g., if $s > C_{\text{ub}}(n)$ or the size of C is strictly larger than s), our function outputs $\perp$. Otherwise it outputs s and $tt(C)$, where $tt(C)$ is the length- 2^n truth table of C . Then we pad the output to make f length-preserving.

In other words, our weak one-way function is defined as follows:

$$f(s, C) = s \circ tt(C).$$

Let $\mathcal{A}_{\text{owf}}$ be any candidate adversary that tries to invert f . We will construct an algorithm $\mathcal{A}_{\text{MCSP}}$ based on $\mathcal{A}_{\text{owf}}$ as in Algorithm 3. In particular, $\mathcal{A}_{\text{MCSP}}$ attempts to solve MCSP on truth tables of length $N := 2^n$, using $\mathcal{A}_{\text{owf}}$ that attempts to invert f on input length $\tilde{N}$. Note that $\tilde{N} \leq 2N$ for large enough n . Hence, if $\mathcal{A}_{\text{owf}}$ runs in $2^{\epsilon \tilde{N}/10}$ time, then $\mathcal{A}_{\text{MCSP}}$ runs in $2^{\epsilon N}$ time. By the hardness of MCSP, $\mathcal{A}_{\text{MCSP}}$ does not compute the circuit complexity correctly on a significant fraction of truth tables. Based on that, we can show that $\mathcal{A}_{\text{owf}}$ satisfies Eq. (3).

Algorithm 3 Bounded-Error Heuristic $\mathcal{A}_{\text{MCSP}}$ for MCSP from Inverter $\mathcal{A}_{\text{owf}}$ for f

```

1: function  $\mathcal{A}_{\text{MCSP}}(tt)$ 
2:    $opt \leftarrow +\infty$ 
3:   for  $s \in [C_{\text{ub}}(n)]$  do
4:      $(s', C) \leftarrow \mathcal{A}_{\text{owf}}(s, tt)$ 
5:     if  $tt(C) = tt$  then
6:        $opt \leftarrow \min\{opt, |C|\}$ 
7:   return  $opt$ 
```

Let Err be the set of truth tables $tt \in \{0, 1\}^N$ on which $\mathcal{A}_{\text{MCSP}}$ fails to output the correct answer w.p. $\geq 1/2q(N)$, where the probability is over the internal randomness of $\mathcal{A}_{\text{MCSP}}$ (which depends on the internal randomness of $\mathcal{A}_{\text{owf}}$). By the hardness of MCSP and a Markov bound, we have

$$|\text{Err}|/2^N \geq 1 - \frac{1 - 1/q(N)}{1 - 1/2q(N)} \geq \frac{1}{2q(N) - 1}.$$

We can see that $\mathcal{A}_{\text{owf}}$ fails on every input of the form $(\text{Size}(tt), tt)$ where $tt \in \text{Err}$ also w.p. $\geq 1/2q(N)$. Every such input is generated in the OWF experiment w.p. at least $1/2^{C'_{\text{ub}}(n)+n}$. That is:

$$\Pr[f(\mathcal{U}_{\tilde{N}}) = (\text{Size}(tt), tt)] \geq 1/2^{C'_{\text{ub}}(n)+n}.$$

If, in the OWF experiment, such an input (of the form $(\text{Size}(tt), tt)$ where $tt \in \text{Err}$) is generated, then $\mathcal{A}_{\text{owf}}$ fails w.p. at least $1/2q(N)$. It follows that

$$\begin{aligned} \Pr_{\mathbf{x} \leftarrow \mathcal{U}_{\tilde{N}}} [\mathcal{A}_{\text{owf}}(f(\mathbf{x})) \notin f^{-1}(f(\mathbf{x}))] &\geq (|\text{Err}|/2^{C'_{\text{ub}}(n)+n}) \cdot (1/2q(N)) \\ &\geq (|\text{Err}|/2^{N+O(\frac{N \log \log N}{\log N})}) \cdot (1/2q(N)) \\ &\geq \frac{1}{(2q(N) - 1)2q(N)2^{O(\frac{N \log \log N}{\log N})}}. \end{aligned}$$

Let $p(\tilde{N}) := (2q(N) - 1)2q(N)2^{O(\frac{N \log \log N}{\log N})}$. Since $N = 2^n \geq \Omega(\tilde{N})$ and $q(N) = 2^{o(N)}$, it is indeed the case that $p(\tilde{N}) = 2^{o(\tilde{N})}$. We can see that every adversary $\mathcal{A}_{\text{owf}}$ that runs in $2^{\epsilon \tilde{N}/10}$ time fails to invert a random output of f w.p. $\geq p(\tilde{N})$. $\square$

6.3 Hardness of MCSP from DLOGTIME One-Way Functions

We establish a weak converse of Theorem 6.6, namely that if there is a weak one-way function in DLOGTIME with exponential security, then MCSP is (exponentially) hard on average. The first step is to build a condEP-PRG from the one-way function.

Lemma 6.7. *Suppose that there is a weak one-way function f computable in DLOGTIME with security $2^{\Omega(N)}$. Then for every small enough constant $\kappa > 0$, the following holds.*

Let n be an integer and $N := 2^n$. There is a DLOGTIME-computable condEP-PRG $\tilde{G}$ mapping $(1 - \kappa)N$ bits to N bits such that for some distribution $\mathcal{E}$:

- *no adversary of size $2^{o(N)}$ could $1/N^{10}$ -distinguish $\tilde{G}(\mathcal{E})$ from the uniform distribution;*
- *the Shannon entropy of $\tilde{G}(\mathcal{E})$ is at least $(1 - \kappa - o(1))N$;*
- *the outputs of $\tilde{G}$, when viewed as truth tables, have circuit complexity $\leq (1 - \kappa + o(1))N/n$.*

Proof. We start by building a condEP-PRG G with linear-size skew circuits. Let K be a large enough universal constant that we fix later, $\kappa_1 := \frac{K\kappa}{1-\kappa}$. Let $M := \lfloor \frac{N}{K+\kappa_1} \rfloor$ be an input length (of f). Let $\delta := 1/\text{poly}(M)$ and $\kappa_2 > 0$ be a constant such that every adversary of size $2^{\kappa_2 M}$ fails to invert f on a $1 - \delta$ fraction of length- M inputs. Let $\alpha := \kappa_1 M$ be the stretch of G . For $\epsilon := 1/4N^{10}$, the outputs of G should be 4ϵ -indistinguishable from uniformly random strings. We use the hash functions in Section 5.4 as the extractors and hardcore functions. Plugging in Construction 5.8, for some absolute constant $K_1 > 0$, we obtain a condEP-PRG $G : \{0, 1\}^M \times \{0, 1\}^{K_1 M} \rightarrow \{0, 1\}^{(1+K_1+\kappa_1)M}$ with the following properties.

- For some absolute constant $K_2 > 0$, G admits a circuit of size $K_2 M$ that is skew w.r.t. the last $K_1 M$ inputs. Moreover, the first $K_1 M$ outputs of G is equal to the last $K_1 M$ inputs of G . (We can achieve this property by rearranging the output bits of G in Construction 5.8.)
- No adversary of size $2^{\Omega(M)}$ could 4ϵ -distinguish the outputs of G from the uniform distribution. (Note that the list size of the hardcore function is $2^{O(\alpha)} = 2^{O(\kappa_1 M)}$. So this is true as long as $\kappa_2 > O(\kappa_1) \geq O(\kappa)$.)
- The entropy of the outputs of G is at least $M - \Omega(\log M)$.

Let $\tilde{G}$ denote the randomized encoding of G (as in Section 5.4.1). Then $\tilde{G}$ is a DLOGTIME-computable condEP-PRG that maps KM input bits to $(K + \kappa_1)M$ input bits. We pad $N - (K + \kappa_1)M$ “useless” random bits to both the inputs and the outputs of $\tilde{G}$, so its output length is indeed N . The entropy of the outputs of $\tilde{G}$ is at least $N - \kappa_1 M - O(\log M) \geq (1 - \kappa - o(1))N$. Again, no adversary of size $2^{o(M)}$ could 4ϵ -distinguish the outputs of $\tilde{G}$ from random strings.

Next, we prove that the outputs of $\tilde{G}$, when viewed as truth tables, have non-trivial circuit complexity. Let $tt^{\text{in}} \in \{0, 1\}^{(1-\kappa)N}$ be an input, $tt^{\text{out}} := \tilde{G}(tt^{\text{in}})$. Let r be a constant such that $\tilde{G}$ is a non-adaptive function that makes r queries to its input. That is, on input i , $\tilde{G}(tt^{\text{in}})$ computes the indices $q(i, 1), q(i, 2), \dots, q(i, r)$, queries $(tt^{\text{in}})_{q(i, j)}$ for each $1 \leq j \leq r$, and computes $(tt^{\text{out}})_i$ based on these answers. Note that every DLOGTIME machine making r *adaptive* queries is equivalent to a DLOGTIME machine making 2^r *non-adaptive* queries, thus it is without loss of generality to assume $\tilde{G}$ is non-adaptive.

By Theorem 6.5, there is a circuit C of size $(1 - \kappa + o(1))N/n$ that on input $(x_1, x_2, \dots, x_r)$, outputs the concatenation of $(tt^{\text{in}})_{x_1}, (tt^{\text{in}})_{x_2}, \dots, (tt^{\text{in}})_{x_r}$. We design a circuit for tt^{out} as follows.

- On input i , simulate $\tilde{G}(i)$ to obtain the indices $q(i, j)$ for every $1 \leq j \leq r$. This step takes $O(n)$ time and thus can be implemented in size $\text{poly}(n)$.
- Use the circuit C of size $(1 - \kappa + o(1))N/n$ to obtain $(tt^{\text{in}})_{q(i, j)}$ for every $1 \leq j \leq r$.
- Finally, we can simulate $\tilde{G}(i)$ to obtain $(tt^{\text{out}})_i$. Again, this step can be implemented in size $\text{poly}(n)$.

It follows that the circuit complexity of tt^{out} is at most $(1 - \kappa + o(1))N/n$. $\square$

Now we prove the hardness of MCSP using the condEP-PRG constructed above.

Theorem 6.8. *Suppose that there is a weak one-way function f computable in DLOGTIME with security $2^{\Omega(N)}$. (See Definition 5.1.) Then, no non-uniform algorithm of size $2^{o(N)}$ can solve MCSP on a $1 - 2^{-o(N)}$ fraction of inputs.*

Proof. Suppose there is a constant $\epsilon > 0$ and a non-uniform algorithm $\mathcal{A}_{\text{MCSP}}$ that solves MCSP on a $1 - 2^{-\epsilon N}$ fraction of inputs. Let $\tilde{G}$ be the condEP-PRG in the conclusion of Lemma 6.7 with parameter $\kappa := \epsilon/2$ and $\mathcal{E}$ be the corresponding distribution of inputs. We will use $\mathcal{A}_{\text{MCSP}}$ to construct a distinguisher $\mathcal{A}$ for $\tilde{G}(\mathcal{E})$ as follows: On input truth table tt , if $\mathcal{A}_{\text{MCSP}}(tt) \geq (1 - \kappa/2)N/n$, then $\mathcal{A}(tt) = 1$; otherwise $\mathcal{A}(tt) = 0$.

For a uniformly random truth table $\mathbf{tt} \leftarrow \mathcal{U}_N$, we have that

$$\begin{aligned} & \Pr_{\mathbf{tt} \leftarrow \mathcal{U}_N} [\mathcal{A}(\mathbf{tt}) = 1] \\ & \geq 1 - 2^{-\epsilon N} - \Pr_{\mathbf{tt} \leftarrow \mathcal{U}_N} [\text{Size}(\mathbf{tt}) < (1 - \kappa/2)N/n] \\ & \geq 1 - 2^{-\epsilon N} - 2^{(s+1)(O(1)+\log(n+s))}/2^N \text{ for } s = (1 - \kappa/2)N/n \quad \text{By Theorem 6.4} \\ & \geq 1 - 2^{-\epsilon N} - 2^{-(\kappa/2 - o(1))N}. \end{aligned}$$

On the other hand, consider a pseudorandom truth table $\mathbf{tt} \leftarrow \tilde{G}(\mathcal{E})$. Let $H := H(\tilde{G}(\mathcal{E}))$, Good be the set of outputs of $\tilde{G}$ that are outputted w.p. at most 2^{1-H} . Then

$$\Pr_{\mathbf{tt} \leftarrow \tilde{G}(\mathcal{E})} [\mathbf{tt} \in \text{Good}] \geq \frac{1}{(1 - \kappa)N - H + 1} \geq \frac{1}{o(N)}. \text{ (See also the proof of Lemma 4.7.)}$$

Let Err be the subset of Good for which $\mathcal{A}_{\text{MCSP}}$ outputs a wrong answer, then $|\text{Err}| \leq 2^{(1-\epsilon)N}$, which means

$$\Pr_{\mathbf{tt} \leftarrow \tilde{G}(\mathcal{E})} [\mathbf{tt} \in \text{Err}] \leq 2^{1-H} \cdot |\text{Err}| \leq 2^{(\kappa - \epsilon + o(1))N}.$$

It follows that

$$\Pr_{\mathbf{tt} \leftarrow \tilde{G}(\mathcal{E})} [\mathcal{A}(\mathbf{tt}) = 1] \leq 1 - \frac{1}{o(N)} + 2^{(\kappa - \epsilon + o(1))N}.$$

Therefore, $\mathcal{A}$ is a non-uniform algorithm of size $2^{o(N)}$ such that

$$\left| \Pr_{\mathbf{tt} \leftarrow \mathcal{U}_N} [\mathcal{A}(\mathbf{tt}) = 1] - \Pr_{\mathbf{tt} \leftarrow \tilde{G}(\varepsilon)} [\mathcal{A}(\mathbf{tt}) = 1] \right| > 1/N^2,$$

contradicting the security of $\tilde{G}$. $\square$

Remark 6.9. Theorem 6.6 and 6.8 are not exact converses of each other, as there are two gaps. First, there is a loss of $2^{O\left(\frac{N \log \log N}{\log N}\right)}$. Second, Theorem 6.6 only produces a (polynomial-time computable) one-way function, but Theorem 6.8 requires a DLOGTIME-computable one-way function to start with.

The first gap seems unavoidable given current knowledge about the maximum circuit complexity. However, we believe that the second gap can be eliminated. In particular, exponential average-case hardness of MCSP should imply a one-way function in DLOGTIME.

If there is a $\oplus\text{L}$ heuristic algorithm for evaluating the truth table of a stack program, then it is indeed true that exponential hardness of MCSP implies a one-way function in DLOGTIME. Note that this heuristic only needs to succeed on *most* stack programs.²¹ For example, if the circuit that corresponds to a uniformly random length- ℓ description has depth at most $O(\log \ell)$ with high probability, then a $\oplus\text{L}$ heuristic can evaluate the circuit up to a particular depth, and still be correct on most inputs. We believe that a random stack program should represent a shallow circuit (w.h.p.), but we are unable to prove it.

Remark 6.10 (Results for MFSP). It is possible to extend Theorem 6.6 to the case of MFSP (Minimum Formula Size Problem). In particular, if MFSP is exponentially hard on average, then there is a (super-polynomially hard) one-way function. Moreover, we only need to compute truth tables of *formulas* to evaluate this one-way function, which is in ALOGTIME [Bus87]. Hence this one-way function is in ALOGTIME and we obtain DLOGTIME-computable one-way functions from Theorem 3.21. We omit the proof here, as it is essentially the same as Theorem 6.6 except that it uses the best bounds for maximum formula complexity (see [Juk12] and references therein). However, we are not aware of any “mass production theorem” (Theorem 6.5) for formulas. Therefore we are not able to prove an MFSP-version of Theorem 6.8.

7 The Average-Case Complexity of MKtP

7.1 Characterizing One-Way Functions Using MKtP

We recall the main result of [LP20] showing an equivalence between the average-case hardness of K^p for some polynomial p and the existence of one-way functions.

Theorem 7.1 ([LP20]). *The following are equivalent:*

1. *There is a polynomial p such that K^p is bounded-error hard on average.*
2. *One-way functions exist.*
3. *For every polynomial $p(n) \geq 2n$ and constant $\lambda > 0$, K^p is bounded-error hard on average to approximate within an additive factor of $\lambda \log n$.*

Somewhat counter-intuitively, we show a similar equivalence between the average-case hardness of Kt and the existence of one-way functions. (Note that Kt is known to be EXP-hard in the worst case under polynomial-size reductions [ABK⁺06].) The proof is closely analogous to the proofs in Section 4, exploiting the fact that “typical” strings of high Kt complexity can be generated from their optimal descriptions in polynomial time. Hence we just provide a sketch.

²¹If this heuristic is always true (i.e. it is a worst-case algorithm), then $\oplus\text{L} = \text{P}$.

Theorem 7.2. *The following are equivalent:*

1. *Kt is bounded-error hard on average.*
2. *One-way functions exist.*
3. *For every constant $\lambda > 0$, Kt is bounded-error hard on average to approximate within an additive factor of $\lambda \log n$.*

Proof Sketch. (3) $\implies$ (1) is trivial.

(1) $\implies$ (2): the proof closely follows the proof of Theorem 4.1.

Suppose that there is a constant c such that every PPT algorithm computes Kt complexity correctly on at most a $1 - 1/n^c$ fraction of inputs. We observe that for all but a $1/n^{2c}$ fraction of inputs $x \in \{0,1\}^n$, optimal pairs (d,t) such that $d + \log t = \text{Kt}(x)$ have the property that $t \leq O(n^{2c+1})$. Actually, for all but a $1/n^{2c}$ fraction of inputs, $\text{K}(x) \geq n - 2c \log n - 1$, while for all inputs x we have $\text{Kt}(x) \leq n + \log n + O(1)$. Hence for all strings x with $\text{K}(x) \geq n - 2c \log n - 1$, x can be generated from its optimal Kt description in time $O(n^{2c+1})$.

We define a weak one-way function f as follows. It takes as input a triple (ℓ, k, M) , where $\ell \in [n + \log n]$, $k \in [(2c + 1) \log n]$, and $M \in \{0,1\}^{n+\log n}$, and outputs (ℓ, k, out) . Here out is the result of running $U^{M'}$ for at most 2^k steps, where M' is the ℓ -bit prefix of M . Just as in Claim 4.3, the output distribution of f on a uniformly chosen input “almost dominates” the uniform distribution. Assume there is an inverter for f , a heuristic algorithm for the search version of $\text{Kt}(x)$ can cycle over all possible ℓ and k , and find the optimal description of x . As Kt is bounded-error hard on average, our candidate one-way function f is secure.

(2) $\implies$ (3): We use Theorem 3.14 to construct a condEP-PRG G with stretch $\gamma \log n$ and security $1/n^\gamma$ from the presumed one-way function. Here, let c be a constant such that G is computable in time n^c , we choose $\gamma = \lambda + c + 2$.

We use an argument closely analogous to that of Lemma 4.7 to show that Kt is bounded-error hard on average to approximate within an additive factor of $\lambda \log n$. The idea is simple: every output of the condEP-PRG has Kt complexity at most $n + c \log n + O(1)$, while a random string of length $n + \gamma \log n$ is likely to have Kt complexity close to $n + \gamma \log n$. Hence, for our choice of parameters, an efficient heuristic algorithm that approximates Kt complexity within an additive factor of $\lambda \log n$ can distinguish the outputs of G from random. $\square$

Theorem 7.1 and Theorem 7.2 yield the following corollary.

Corollary 7.3. *Kt is bounded-error hard on average iff there is a polynomial p such that KP^p is bounded-error hard on average.*

Corollary 7.3 gives a new non-trivial connection between meta-complexity problems that seems hard to argue without using one-way functions as an intermediate notion.

7.2 A Complexity Theoretic Analogue

Theorem 7.2 shows that the weak average-case hardness of Kt is equivalent to the existence of cryptographic pseudo-random generators. We next show that for a slightly different setting of parameters, the average-case hardness of Kt is equivalent to the existence of complexity-theoretic pseudo-random generators against non-uniform adversaries. Thus average-case complexity of a single natural problem, namely Kt, can be used to characterize both cryptographic pseudorandomness and complexity-theoretic pseudorandomness.

Recall that cryptographic PRGs are required to be computable in fixed polynomial time but secure against adversaries that can run within any polynomial time bound. In contrast, complexity-theoretic PRGs are allowed to use more resources than the adversary.

Definition 7.4. Given functions $t : \mathbb{N} \rightarrow \mathbb{N}$, $\ell : \mathbb{N} \rightarrow \mathbb{N}$ (satisfying $\ell(n) \leq n$ for each n) and $s : \mathbb{N} \rightarrow \mathbb{N}$, we say that a family of functions $\{G_n\}$, where $G_n : \{0,1\}^{\ell(n)} \rightarrow \{0,1\}^n$ is a time t PRG (resp. i.o.PRG) with seed length ℓ against size s if $G(z)$ is computable in time $t(|z|)$ and for each n (resp. for infinitely many n), $G_n(\mathcal{U}_{\ell(n)})$ is $1/s(n)$ -indistinguishable from $\mathcal{U}_n$ by size $s(n)$ circuits. The PRG is said to be seed-extending if z is a prefix of $G(z)$ for each seed z .

Nisan and Wigderson [NW94, BFNW93] showed how to base seed-extending complexity-theoretic PRGs on the hardness of E (exponential-time). The parameters in the following theorem statement are implicit in their main result.

Theorem 7.5 ([NW94, BFNW93]). *If $\text{DTIME}[2^n \text{poly}(n)] \not\subseteq P_{/\text{poly}}$, then for each ℓ such that $\ell(n) = n^{\Omega(1)}$, there is a seed-extending time $2^\ell \text{poly}(\ell)$ i.o.PRG with seed length ℓ against polynomial size.*

We use Theorem 7.5 to derive an equivalence between the worst-case hardness of Kt, the existence of complexity-theoretic PRGs with non-trivial seed length, and very mild average-case hardness of Kt, where the hardness is against non-uniform adversaries. The idea of the proof is similar to that of [ABK⁺06], who showed that computing Kt complexity is hard for exponential-time under polynomial-size reductions.

Theorem 7.6. *The following are equivalent:*

1. $\text{EXP} \not\subseteq P_{/\text{poly}}$.
2. For each $\epsilon > 0$, there is a time $2^\ell \text{poly}(\ell)$ i.o.PRG with seed length n^ϵ against polynomial size.
3. There are no polynomial size circuits for Kt.
4. For each $\epsilon > 0$, there is a seed-extending time $2^\ell \text{poly}(\ell)$ i.o.PRG with seed length n^ϵ against polynomial size.
5. For any constant $\delta > 1/2$, there are no polynomial size circuits computing Kt on at least a $1 - 1/2^{\delta n}$ fraction of inputs of length n for infinitely many n .

Proof. (1) $\iff$ (3) is shown in [ABK⁺06].

(1) $\iff$ (2) is shown in [NW94, BFNW93].

(5) $\implies$ (3) is trivial.

(3) $\implies$ (4): We use Theorem 7.5. Kt can be computed in time $O(2^n \text{poly}(n))$, and we can define a decision version of Kt that is equivalent to the search version and computable in time $2^n \text{poly}(n)$ as follows: For $x \in \{0,1\}^n$ and $k \in [n + \log n]$, (x, k) is a Yes instance of the decision version of Kt iff $\text{Kt}(x) \leq k$. By Theorem 7.5, the hardness of the decision version implies that for each $\epsilon > 0$, there is a seed-extending time $2^m \text{poly}(m)$ i.o.PRG with seed length n^ϵ against polynomial size.

(4) $\implies$ (5): Consider a seed-extending $2^m \text{poly}(m)$ time i.o.PRG $G = \{G_n\}$ with seed length γn , where $1/2 > \gamma > 1 - \delta$. Such an i.o.PRG is implied by an i.o.PRG with smaller seed length, simply by ignoring all but the first n^ϵ bits of the seed when computing the output. Since the seed length is γn and the i.o.PRG is computable in time $2^{\gamma n} \text{poly}(n)$, we have that each output of the i.o.PRG has Kt complexity at most $2\gamma n + O(\log n) < n - \log n$. On the other hand, a uniformly chosen input of length n has Kt complexity very close to n , with high probability.

Suppose that there are polynomial size circuits $\{C_n\}$ computing Kt on a $1 - 1/2^{\delta n}$ fraction of inputs for all n . By our choice of δ , this means that they are correct on at least a $2/3$ fraction of strings $G_n(z)$ for seed z of length γn . Now we can define a family of distinguishers $\{D_n\}$ as follows: D_n computes $C_n(x)$ and accepts iff $C_n(x) \leq n - \log(n)$. D_n accepts with probability

$2/3$ on $G_n(z)$ for uniformly chosen z , but with probability at most $1/3$ on x for uniformly chosen x of length n , since all but a $o(1)$ fraction of strings have $\text{Kt}(x) > n - \log(n)$ and C_n answers correctly on all but a $o(1)$ fraction of these strings with high Kt complexity. Therefore D_n is a distinguisher of polynomial size for all n , contradicting the assumption that G is an i.o.PRG. $\square$

8 Open Problems

We conclude this paper with a few open questions.

Perebor hypotheses. How plausible are the Perebor hypotheses in Section 5.5? Recently, Mazon and Pass [MP24] and, independently, Hirahara, Ilango, and Williams [HIW24], refuted the non-uniform version of Hypothesis 5.19, by giving an efficient circuit implementation of the inverter in [FN99].

It would be exciting to refute the other Strong Perebor Hypotheses. Let $t(n)$ be a polynomial (say $t(n) = 10n$ for simplicity). Is there a (probabilistic) algorithm running in $2^n/n^{\omega(1)}$ time that computes Kt^t in the worst case? What about the average case? Does such an algorithm imply new circuit lower bounds, as in the case of SAT algorithms [Wil13] and learning algorithms [OS17]? Is there a circuit family of $2^n/n^{\omega(1)}$ size that computes cK^t (on input length $2n = n+n$)?

The Strong Exponential Time Hypothesis is used extensively in *fine-grained complexity*. Conditioning on SETH, we can prove many polynomial lower bounds for problems in P (e.g. the Orthogonal Vectors problem requires $n^{2-o(1)}$ time [Wil05]). Do the Strong Perebor Hypotheses imply non-trivial conditional lower bounds for natural problems in P?

Random circuits. Due to our limited knowledge about circuit complexity, the relations presented in Section 6 are not tight. We point out a few questions whose resolution would tighten the relationship between MCSP and one-way functions.

First, is there an efficiently samplable distribution over circuits such that for most truth table $tt \in \{0,1\}^N$, the probability that the optimal circuit for tt is sampled is at least $2^{-N}/\text{poly}(N)$? Such a distribution would imply a one-way function from super-polynomial hardness of MCSP. The trivial solution as presented in Section 6.2 is to sample a uniformly random circuit according to some encoding. The probability that the optimal circuit is sampled is $2^{-N}/2^{O\left(N \frac{\log \log N}{\log N}\right)}$.

Second, is there a $\oplus\text{L}$ heuristic algorithm for evaluating a random circuit that succeeds on *most* circuits? Of course, this depends on the exact definition of “random” circuits. Such a heuristic implies a DLOGTIME -computable one-way function from the hardness of MCSP, establishing a tighter converse of Theorem 6.8.

Last, does the existence of DLOGTIME -computable one-way functions imply the hardness of MFSP? The main technical difficulty is that we do not have a formula version of Theorem 6.5.

Other cryptographic primitives? Meta-complexity can characterize the existence of one-way functions [LP20] and one-way functions in NC^0 (this paper). Is there a similar characterization for other cryptographic primitives, such as public-key encryption [RSA78, DH76] or indistinguishability obfuscation [BGI⁺12]? Recently, Ball, Liu, Mazon, and Pass [BLMP23] introduced the notion of “interactive Kolmogorov complexity” IK^t , and used the hardness of estimating IK^t to characterize key agreement. This was followed by Liu, Mazon, and Pass [LMP25] who characterized public-key encryption by the hardness of a certain learning problem.

Is there a meta-complexity characterization of exponentially-hard *strong* one-way functions? This would bring new insights to the old question of hardness amplification for one-way functions that preserve exponential security [GIL⁺90].

Errorless vs. error-prone heuristics. The results of Liu and Pass [LP20,LP21], along with our findings on the average-case hardness of meta-complexity problems, suggest a potentially significant conceptual gap between the *errorless* and *error-prone* average-case complexity of such problems. Specifically, if MKtP admits an efficient errorless heuristic, then $\text{EXP} = \text{BPP}$ [ABK⁺06,LP21]. In contrast, the existence of efficient error-prone heuristics for MKtP only implies the non-existence of one-way functions, which is a far weaker consequence (compared with $\text{EXP} = \text{BPP}$).

This raises important questions: Which problems admit efficient errorless-to-error-prone average-case reductions? For which problems are such reductions difficult to obtain? Partly inspired by the above characterizations in meta-complexity, Hirahara and Santhanam [HS22] initiated a broader study of errorless vs. error-prone average-case complexity. However, these questions remain largely open and far from understood.

Acknowledgments

The first author is grateful to Lijie Chen, Mahdi Cheraghchi, and Yanyi Liu for helpful discussions. The second author thanks Yuval Ishai for a useful e-mail discussion. We thank the anonymous CCC and ToC reviewers for helpful comments that improve the presentation of this paper.

References

- [AB09] Sanjeev Arora and Boaz Barak. *Computational Complexity: A Modern Approach*. Cambridge University Press, 2009. (cit. on p. 25)
- [ABK⁺06] Eric Allender, Harry Buhrman, Michal Koucký, Dieter van Melkebeek, and Detlef Ronneburger. Power from random strings. *SIAM Journal of Computing*, 35(6):1467–1493, 2006. doi:10.1137/050628994. (cit. on p. 1, 2, 3, 4, 8, 12, 14, 41, 43, 45)
- [ABN⁺92] Noga Alon, Jehoshua Bruck, Joseph Naor, Moni Naor, and Ron M. Roth. Construction of asymptotically good low-rate error-correcting codes through pseudo-random graphs. *IEEE Transactions on Information Theory*, 38(2):509–516, 1992. doi:10.1109/18.119713. (cit. on p. 58)
- [ACM⁺21] Eric Allender, Mahdi Cheraghchi, Dimitrios Myrisiotis, Harsha Tirumala, and Ilya Volkovich. One-way functions and a conditional variant of MKTP. In *Proc. 41st Annual Conference on Foundations of Software Technology and Theoretical Computer Science (FSTTCS)*, volume 213 of *LIPIcs*, pages 7:1–7:19, 2021. doi:10.4230/LIPIcs.FSTTCS.2021.7. (cit. on p. 6, 7)
- [AGGM06] Adi Akavia, Oded Goldreich, Shafi Goldwasser, and Dana Moshkovitz. On basing one-way functions on NP-hardness. In *Proc. 38th Annual ACM Symposium on Theory of Computing (STOC)*, pages 701–710, 2006. doi:10.1145/1132516.1132614. (cit. on p. 2)
- [AH19] Eric Allender and Shuichi Hirahara. New insights on the (non-)hardness of circuit minimization and related problems. *ACM Transactions on Computation Theory*, 11(4):27:1–27:27, 2019. doi:10.1145/3349616. (cit. on p. 1)
- [AIK06] Benny Applebaum, Yuval Ishai, and Eyal Kushilevitz. Cryptography in NC^0 . *SIAM Journal of Computing*, 36(4):845–888, 2006. doi:10.1137/S0097539705446950. (cit. on p. 2, 3, 4, 5, 6, 8, 9, 17, 18, 21, 25, 32)
- [Ajt96] Miklós Ajtai. Generating hard instances of lattice problems (extended abstract). In *Proc. 28th Annual ACM Symposium on Theory of Computing (STOC)*, pages 99–108, 1996. doi:10.1145/237814.237838. (cit. on p. 1)
- [All01] Eric Allender. When worlds collide: Derandomization, lower bounds, and Kolmogorov complexity. In *Proc. 21st Foundations of Software Technology and Theoretical Computer Science (FSTTCS)*, volume 2245 of *Lecture Notes in Computer Science*, pages 1–15, 2001. doi:10.1007/3-540-45294-X\1. (cit. on p. 1, 2, 3, 13)

- [App14] Benny Applebaum. *Cryptography in Constant Parallel Time*. Information Security and Cryptography. Springer, 2014. doi:10.1007/978-3-642-17367-7. (cit. on p. 2, 6, 8)
- [App16] Benny Applebaum. Cryptographic hardness of random local functions - survey. *Computational Complexity*, 25(3):667–722, 2016. doi:10.1007/s00037-015-0121-8. (cit. on p. 6)
- [App17] Benny Applebaum. Exponentially-hard Gap-CSP and local PRG via local hardcore functions. In *Proc. 58th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 836–847, 2017. doi:10.1109/FOCS.2017.82. (cit. on p. 11, 32)
- [BFKL93] Avrim Blum, Merrick L. Furst, Michael J. Kearns, and Richard J. Lipton. Cryptographic primitives based on hard learning problems. In *Advances in Cryptology - CRYPTO '93, 13th Annual International Cryptology Conference, Santa Barbara, California, USA, August 22-26, 1993, Proceedings*, volume 773 of *Lecture Notes in Computer Science*, pages 278–291. Springer, 1993. doi:10.1007/3-540-48329-2_24. (cit. on p. 6)
- [BFNW93] László Babai, Lance Fortnow, Noam Nisan, and Avi Wigderson. BPP has subexponential time simulations unless EXPTIME has publishable proofs. *Computational Complexity*, 3:307–318, 1993. doi:10.1007/BF01275486. (cit. on p. 43)
- [BGI08] Eli Biham, Yaron J. Goren, and Yuval Ishai. Basing weak public-key cryptography on strong one-way functions. In *Proc. 5th Theory of Cryptography Conference (TCC)*, volume 4948 of *Lecture Notes in Computer Science*, pages 55–72, 2008. doi:10.1007/978-3-540-78524-8_4. (cit. on p. 35)
- [BGI⁺12] Boaz Barak, Oded Goldreich, Russell Impagliazzo, Steven Rudich, Amit Sahai, Salil P. Vadhan, and Ke Yang. On the (im)possibility of obfuscating programs. *Journal of the ACM*, 59(2):6:1–6:48, 2012. doi:10.1145/2160158.2160159. (cit. on p. 44)
- [BIO14] Joshua Baron, Yuval Ishai, and Rafail Ostrovsky. On linear-size pseudorandom generators and hardcore functions. *Theoretical Computer Science*, 554:50–63, 2014. doi:10.1016/j.tcs.2014.06.013. (cit. on p. 11, 32, 34)
- [BLMP23] Marshall Ball, Yanyi Liu, Noam Mazon, and Rafael Pass. Kolmogorov comes to cryptomania: On interactive Kolmogorov complexity and key-agreement. In *FOCS*, pages 458–483. IEEE, 2023. doi:10.1109/FOCS57990.2023.00034. (cit. on p. 44)
- [Bor57] J. L. Bordewijk. Inter-reciprocity applied to electrical networks. *Applied Scientific Research, Section A*, pages 1–74, 1957. doi:10.1007/BF02410413. (cit. on p. 58)
- [BT06] Andrej Bogdanov and Luca Trevisan. On worst-case to average-case reductions for NP problems. *SIAM Journal of Computing*, 36(4):1119–1159, 2006. doi:10.1137/S0097539705446974. (cit. on p. 2)
- [Bus87] Samuel R. Buss. The Boolean formula value problem is in ALOGTIME. In *Proc. 19th Annual ACM Symposium on Theory of Computing (STOC)*, pages 123–131, 1987. doi:10.1145/28395.28409. (cit. on p. 41)
- [CDH⁺00] Ran Canetti, Yevgeniy Dodis, Shai Halevi, Eyal Kushilevitz, and Amit Sahai. Exposure-resilient functions and all-or-nothing transforms. In *Advances in Cryptology - EUROCRYPT 2000, International Conference on the Theory and Application of Cryptographic Techniques*, volume 1807 of *Lecture Notes in Computer Science*, pages 453–469, 2000. doi:10.1007/3-540-45539-6_33. (cit. on p. 58)
- [CGH⁺85] Benny Chor, Oded Goldreich, Johan Håstad, Joel Friedman, Steven Rudich, and Roman Smolensky. The bit extraction problem or t -resilient functions (preliminary version). In *Proc. 26th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 396–407, 1985. doi:10.1109/SFCS.1985.55. (cit. on p. 58)
- [CHI⁺21] Marco Carmosino, Kenneth Hoover, Russell Impagliazzo, Valentine Kabanets, and Antonina Kolokolova. Lifting for constant-depth circuits and applications to MCSP. In *ICALP*, volume 198 of *LIPICs*, pages 44:1–44:20. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2021. doi:10.4230/LIPICs.ICALP.2021.44. (cit. on p. 7)

- [CIP09] Chris Calabro, Russell Impagliazzo, and Ramamohan Paturi. The complexity of satisfiability of small depth circuits. In *Parameterized and Exact Computation, 4th International Workshop, (IWPEC) 2009*, volume 5917 of *Lecture Notes in Computer Science*, pages 75–85. Springer, 2009. doi:10.1007/978-3-642-11269-0_6. (cit. on p. 5, 34)
- [CJW19] Lijie Chen, Ce Jin, and R. Ryan Williams. Hardness magnification for all sparse NP languages. In *Proc. 60th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 1240–1255, 2019. doi:10.1109/FOCS.2019.00077. (cit. on p. 1)
- [CR20] Lijie Chen and Hanlin Ren. Strong average-case lower bounds from non-trivial derandomization. In *Proc. 52nd Annual ACM Symposium on Theory of Computing (STOC)*, pages 1327–1334, 2020. doi:10.1145/3357713.3384279. (cit. on p. 17, 18)
- [DH76] Whitfield Diffie and Martin E. Hellman. New directions in cryptography. *IEEE Transactions on Information Theory*, 22(6):644–654, 1976. doi:10.1109/TIT.1976.1055638. (cit. on p. 1, 44)
- [FM05] Gudmund Skovbjerg Frandsen and Peter Bro Miltersen. Reviewing bounds on the circuit size of the hardest functions. *Information Processing Letters*, 95(2):354–357, 2005. doi:10.1016/j.ipl.2005.03.009. (cit. on p. 11, 36, 37)
- [FN99] Amos Fiat and Moni Naor. Rigorous time/space trade-offs for inverting functions. *SIAM Journal of Computing*, 29(3):790–803, 1999. doi:10.1137/S0097539795280512. (cit. on p. 35, 44)
- [FSUV13] Bill Fefferman, Ronen Shaltiel, Christopher Umans, and Emanuele Viola. On beating the hybrid argument. *Theory of Computing*, 9:809–843, 2013. doi:10.4086/toc.2013.v009a026. (cit. on p. 9, 17)
- [Für82] Martin Fürer. The tight deterministic time hierarchy. In *Proc. 14th Annual ACM Symposium on Theory of Computing (STOC)*, pages 8–16, 1982. doi:10.1145/800070.802172. (cit. on p. 14)
- [GG81] Ofer Gabber and Zvi Galil. Explicit constructions of linear-sized superconcentrators. *Journal of Computer and System Sciences*, 22(3):407–420, 1981. doi:10.1016/0022-0000(81)90040-4. (cit. on p. 56)
- [GGH⁺07] Shafi Goldwasser, Dan Gutfreund, Alexander Healy, Tali Kaufman, and Guy N. Rothblum. Verifying and decoding in constant depth. In *Proc. 39th Annual ACM Symposium on Theory of Computing (STOC)*, pages 440–449, 2007. doi:10.1145/1250790.1250855. (cit. on p. 17)
- [GI01] Venkatesan Guruswami and Piotr Indyk. Expander-based constructions of efficiently decodable codes. In *Proc. 42nd Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 658–667, 2001. doi:10.1109/SFCS.2001.959942. (cit. on p. 58)
- [GII⁺19] Alexander Golovnev, Rahul Ilango, Russell Impagliazzo, Valentine Kabanets, Antonina Kolokolova, and Avishay Tal. $AC^0[p]$ lower bounds against MCSP via the coin problem. In *Proc. 46th International Colloquium on Automata, Languages and Programming (ICALP)*, volume 132 of *LIPIcs*, pages 66:1–66:15, 2019. doi:10.4230/LIPIcs.ICALP.2019.66. (cit. on p. 1)
- [GIL⁺90] Oded Goldreich, Russell Impagliazzo, Leonid A. Levin, Ramarathnam Venkatesan, and David Zuckerman. Security preserving amplification of hardness. In *Proc. 31st Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 318–326, 1990. doi:10.1109/FOCS.1990.89550. (cit. on p. 28, 44)
- [GL89] Oded Goldreich and Leonid A. Levin. A hard-core predicate for all one-way functions. In *Proc. 21st Annual ACM Symposium on Theory of Computing (STOC)*, pages 25–32, 1989. doi:10.1145/73007.73010. (cit. on p. 10, 28, 30)
- [Gol01] Oded Goldreich. *The Foundations of Cryptography - Volume 1: Basic Techniques*. Cambridge University Press, 2001. doi:10.1017/CBO9780511546891. (cit. on p. 3, 15, 16, 37)
- [GUV09] Venkatesan Guruswami, Christopher Umans, and Salil P. Vadhan. Unbalanced expanders and randomness extractors from Parvaresh–Vardy codes. *Journal of the ACM*, 56(4):20:1–20:34, 2009. doi:10.1145/1538902.1538904. (cit. on p. 10, 28, 31)

- [Hel80] Martin E. Hellman. A cryptanalytic time-memory trade-off. *IEEE Transactions on Information Theory*, 26(4):401–406, 1980. doi:[10.1109/TIT.1980.1056220](https://doi.org/10.1109/TIT.1980.1056220). (cit. on p. 35)
- [HILL99] Johan Håstad, Russell Impagliazzo, Leonid A. Levin, and Michael Luby. A pseudorandom generator from any one-way function. *SIAM Journal of Computing*, 28(4):1364–1396, 1999. doi:[10.1137/S0097539793244708](https://doi.org/10.1137/S0097539793244708). (cit. on p. 4, 8, 30, 34)
- [Hir18] Shuichi Hirahara. Non-black-box worst-case to average-case reductions within NP. In *Proc. 59th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 247–258, 2018. doi:[10.1109/FOCS.2018.00032](https://doi.org/10.1109/FOCS.2018.00032). (cit. on p. 1)
- [Hir20a] Shuichi Hirahara. Characterizing average-case complexity of PH by worst-case meta-complexity. In *Proc. 61st Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 50–60, 2020. doi:[10.1109/FOCS46700.2020.00014](https://doi.org/10.1109/FOCS46700.2020.00014). (cit. on p. 10, 31)
- [Hir20b] Shuichi Hirahara. Unexpected hardness results for Kolmogorov complexity under uniform reductions. In *Proc. 52nd Annual ACM Symposium on Theory of Computing (STOC)*, pages 1038–1051, 2020. doi:[10.1145/3357713.3384251](https://doi.org/10.1145/3357713.3384251). (cit. on p. 10, 31)
- [HIW24] Shuichi Hirahara, Rahul Ilango, and R. Ryan Williams. Beating brute force for compression problems. In *STOC*, pages 659–670. ACM, 2024. doi:[10.1145/3618260.3649778](https://doi.org/10.1145/3618260.3649778). (cit. on p. 7, 44)
- [HLW06] Shlomo Hoory, Nathan Linial, and Avi Wigderson. Expander graphs and their applications. *Bulletin of the American Mathematical Society*, pages 439–561, 2006. doi:[10.1090/S0273-0979-06-01126-8](https://doi.org/10.1090/S0273-0979-06-01126-8). (cit. on p. 56)
- [HMS04] Thomas Holenstein, Ueli M. Maurer, and Johan Sjödin. Complete classification of bilinear hard-core functions. In *Proc. 24th Annual International Cryptology Conference (CRYPTO)*, volume 3152 of *Lecture Notes in Computer Science*, pages 73–91. Springer, 2004. doi:[10.1007/978-3-540-28628-8_5](https://doi.org/10.1007/978-3-540-28628-8_5). (cit. on p. 11, 32, 34)
- [HRV13] Iftach Haitner, Omer Reingold, and Salil P. Vadhan. Efficiency improvements in constructing pseudorandom generators from one-way functions. *SIAM Journal of Computing*, 42(3):1405–1430, 2013. doi:[10.1137/100814421](https://doi.org/10.1137/100814421). (cit. on p. 8)
- [HS17] Shuichi Hirahara and Rahul Santhanam. On the average-case complexity of MCSP and its variants. In *Proc. 32nd Computational Complexity Conference (CCC)*, volume 79 of *LIPICs*, pages 7:1–7:20, 2017. doi:[10.4230/LIPICs.CCC.2017.7](https://doi.org/10.4230/LIPICs.CCC.2017.7). (cit. on p. 1, 6, 17, 25)
- [HS22] Shuichi Hirahara and Rahul Santhanam. Errorless versus error-prone average-case complexity. In *ITCS*, volume 215 of *LIPICs*, pages 84:1–84:23. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2022. doi:[10.4230/LIPICs.ITCS.2022.84](https://doi.org/10.4230/LIPICs.ITCS.2022.84). (cit. on p. 45)
- [HS24] Shuichi Hirahara and Nobutaka Shimizu. Planted clique conjectures are equivalent. In *STOC*, pages 358–366. ACM, 2024. doi:[10.1145/3618260.3649751](https://doi.org/10.1145/3618260.3649751). (cit. on p. 2)
- [IK00] Yuval Ishai and Eyal Kushilevitz. Randomizing polynomials: A new representation with applications to round-efficient secure computation. In *Proc. 41st Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 294–304, 2000. doi:[10.1109/SFCS.2000.892118](https://doi.org/10.1109/SFCS.2000.892118). (cit. on p. 17, 18)
- [IK02] Yuval Ishai and Eyal Kushilevitz. Perfect constant-round secure computation via perfect randomizing polynomials. In *Proc. 29th International Colloquium on Automata, Languages and Programming (ICALP)*, pages 244–256, 2002. doi:[10.1007/3-540-45465-9_22](https://doi.org/10.1007/3-540-45465-9_22). (cit. on p. 17, 18)
- [IKOS08] Yuval Ishai, Eyal Kushilevitz, Rafail Ostrovsky, and Amit Sahai. Cryptography with constant computational overhead. In *Proc. 40th Annual ACM Symposium on Theory of Computing (STOC)*, pages 433–442, 2008. doi:[10.1145/1374376.1374438](https://doi.org/10.1145/1374376.1374438). (cit. on p. 32, 33, 34, 55, 56, 58, 59)
- [IL90] Russell Impagliazzo and Leonid A. Levin. No better ways to generate hard NP instances than picking uniformly at random. In *Proc. 31st Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 812–821, 1990. doi:[10.1109/FSCS.1990.89604](https://doi.org/10.1109/FSCS.1990.89604). (cit. on p. 6)

- [Ila20a] Rahul Ilango. Connecting perebor conjectures: Towards a search to decision reduction for minimizing formulas. In *Proc. 35th Computational Complexity Conference (CCC)*, volume 169 of *LIPICs*, pages 31:1–31:35, 2020. doi:[10.4230/LIPICs.CCC.2020.31](https://doi.org/10.4230/LIPICs.CCC.2020.31). (cit. on p. [1](#), [36](#))
- [Ila20b] Rahul Ilango. Constant depth formula and partial function versions of MCSP are hard. In *Proc. 61st Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 424–433, 2020. doi:[10.1109/FOCS46700.2020.00047](https://doi.org/10.1109/FOCS46700.2020.00047). (cit. on p. [1](#))
- [ILO20] Rahul Ilango, Bruno Loff, and Igor Carboni Oliveira. NP-hardness of circuit minimization for multi-output functions. In *Proc. 35th Computational Complexity Conference (CCC)*, volume 169 of *LIPICs*, pages 22:1–22:36, 2020. doi:[10.4230/LIPICs.CCC.2020.22](https://doi.org/10.4230/LIPICs.CCC.2020.22). (cit. on p. [1](#))
- [Imp95a] Russell Impagliazzo. Hard-core distributions for somewhat hard problems. In *Proc. 36th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 538–545, 1995. doi:[10.1109/SFCS.1995.492584](https://doi.org/10.1109/SFCS.1995.492584). (cit. on p. [10](#), [30](#), [52](#))
- [Imp95b] Russell Impagliazzo. A personal view of average-case complexity. In *Proc. 10th Annual Structure in Complexity Theory Conference*, pages 134–147, 1995. doi:[10.1109/SCT.1995.514853](https://doi.org/10.1109/SCT.1995.514853). (cit. on p. [2](#))
- [IP01] Russell Impagliazzo and Ramamohan Paturi. On the complexity of k -SAT. *Journal of Computer and System Sciences*, 62(2):367–375, 2001. doi:[10.1006/jcss.2000.1727](https://doi.org/10.1006/jcss.2000.1727). (cit. on p. [5](#), [34](#))
- [IPZ01] Russell Impagliazzo, Ramamohan Paturi, and Francis Zane. Which problems have strongly exponential complexity? *Journal of Computer and System Sciences*, 63(4):512–530, 2001. doi:[10.1006/jcss.2001.1774](https://doi.org/10.1006/jcss.2001.1774). (cit. on p. [5](#), [34](#))
- [Juk12] Stasys Jukna. *Boolean Function Complexity - Advances and Frontiers*, volume 27 of *Algorithms and combinatorics*. Springer, 2012. doi:[10.1007/978-3-642-24508-4](https://doi.org/10.1007/978-3-642-24508-4). (cit. on p. [41](#))
- [KC00] Valentine Kabanets and Jin-Yi Cai. Circuit minimization problem. In *Proc. 32nd Annual ACM Symposium on Theory of Computing (STOC)*, pages 73–79, 2000. doi:[10.1145/335305.335314](https://doi.org/10.1145/335305.335314). (cit. on p. [1](#), [8](#))
- [Ko91] Ker-I Ko. On the complexity of learning minimum time-bounded Turing machines. *SIAM Journal of Computing*, 20(5):962–986, 1991. doi:[10.1137/0220059](https://doi.org/10.1137/0220059). (cit. on p. [13](#))
- [Lev84] Leonid A. Levin. Randomness conservation inequalities; information and independence in mathematical theories. *Information and Control*, 61(1):15–37, 1984. doi:[10.1016/S0019-9958\(84\)80060-1](https://doi.org/10.1016/S0019-9958(84)80060-1). (cit. on p. [2](#), [13](#))
- [Lev03] Leonid A. Levin. The tale of one-way functions. *Problems of Information Transmission*, 39(1):92–103, 2003. doi:[10.1023/A%3A1023634616182](https://doi.org/10.1023/A%3A1023634616182). (cit. on p. [3](#))
- [LMP25] Yanyi Liu, Noam Mazon, and Rafael Pass. On white-box learning and public-key encryption. In *ITCS*, volume 325 of *LIPICs*, pages 73:1–73:22. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2025. doi:[10.4230/LIPICs.ITCS.2025.73](https://doi.org/10.4230/LIPICs.ITCS.2025.73). (cit. on p. [44](#))
- [LP20] Yanyi Liu and Rafael Pass. On one-way functions and Kolmogorov complexity. In *Proc. 61st Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 1243–1254, 2020. doi:[10.1109/FOCS46700.2020.00118](https://doi.org/10.1109/FOCS46700.2020.00118). (cit. on p. [2](#), [3](#), [6](#), [7](#), [8](#), [9](#), [10](#), [16](#), [17](#), [20](#), [24](#), [27](#), [28](#), [29](#), [30](#), [31](#), [36](#), [41](#), [44](#), [45](#), [55](#))
- [LP21] Yanyi Liu and Rafael Pass. On the possibility of basing cryptography on $\text{EXP} \neq \text{BPP}$. In *Proc. 41st Annual International Cryptology Conference (CRYPTO)*, volume 12825 of *Lecture Notes in Computer Science*, pages 11–40. Springer, 2021. doi:[10.1007/978-3-030-84242-0_2](https://doi.org/10.1007/978-3-030-84242-0_2). (cit. on p. [6](#), [7](#), [45](#))
- [Mar73] G. A. Margulis. Explicit constructions of concentrators. *Probl. Peredachi Inf.*, pages 71–80, 1973. (cit. on p. [56](#))
- [MMW19] Dylan M. McKay, Cody D. Murray, and R. Ryan Williams. Weak lower bounds on resource-bounded compression imply strong separations of complexity classes. In *Proc. 51st Annual ACM Symposium on Theory of Computing (STOC)*, pages 1215–1225, 2019. doi:[10.1145/3313276.3316396](https://doi.org/10.1145/3313276.3316396). (cit. on p. [1](#))

- [MP24] Noam Mazon and Rafael Pass. The non-uniform Peregbor conjecture for time-bounded Kolmogorov complexity is false. In *ITCS*, volume 287 of *LIPIcs*, pages 80:1–80:20. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2024. doi:10.4230/LIPIcs.ITCS.2024.80. (cit. on p. 7, 44)
- [MW17] Cody D. Murray and R. Ryan Williams. On the (non) NP-hardness of computing circuit complexity. *Theory of Computing*, 13(1):1–22, 2017. doi:10.4086/toc.2017.v013a004. (cit. on p. 1)
- [Nan21] Mikito Nanashima. On basing auxiliary-input cryptography on NP-hardness via nonadaptive black-box reductions. In *Proc. 12th Conference on Innovations in Theoretical Computer Science (ITCS)*, volume 185 of *LIPIcs*, pages 29:1–29:15, 2021. doi:10.4230/LIPIcs.ITCS.2021.29. (cit. on p. 2)
- [Nis96] Noam Nisan. Extracting randomness: How and why. A survey. In *Proc. 11th Annual IEEE Conference on Computational Complexity (CCC)*, pages 44–58, 1996. doi:10.1109/CCC.1996.507667. (cit. on p. 10)
- [NW94] Noam Nisan and Avi Wigderson. Hardness vs randomness. *Journal of Computer and System Sciences*, 49(2):149–167, 1994. doi:10.1016/S0022-0000(05)80043-1. (cit. on p. 4, 12, 43)
- [NZ93] Noam Nisan and David Zuckerman. More deterministic simulation in logspace. In *Proc. 25th Annual ACM Symposium on Theory of Computing (STOC)*, pages 235–244, 1993. doi:10.1145/167088.167162. (cit. on p. 10)
- [OPS21] Igor C. Oliveira, Ján Pich, and Rahul Santhanam. Hardness magnification near state-of-the-art lower bounds. *Theory of Computing*, 17:1–38, 2021. doi:10.4086/toc.2021.v017a011. (cit. on p. 1)
- [OS17] Igor Carboni Oliveira and Rahul Santhanam. Conspiracies between learning algorithms, circuit lower bounds, and pseudorandomness. In *Proc. 32nd Computational Complexity Conference (CCC)*, volume 79 of *LIPIcs*, pages 18:1–18:49, 2017. doi:10.4230/LIPIcs.CCC.2017.18. (cit. on p. 1, 6, 44)
- [OS18] Igor Carboni Oliveira and Rahul Santhanam. Hardness magnification for natural problems. In *Proc. 59th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 65–76, 2018. doi:10.1109/FOCS.2018.00016. (cit. on p. 1)
- [RR97] Alexander A. Razborov and Steven Rudich. Natural proofs. *Journal of Computer and System Sciences*, 55(1):24–35, 1997. doi:10.1006/jcss.1997.1494. (cit. on p. 6, 8)
- [RSA78] Ronald L. Rivest, Adi Shamir, and Leonard M. Adleman. A method for obtaining digital signatures and public-key cryptosystems. *Communications of the ACM*, 21(2):120–126, 1978. doi:10.1145/359340.359342. (cit. on p. 1, 44)
- [San20] Rahul Santhanam. Pseudorandomness and the minimum circuit size problem. In *Proc. 11th Conference on Innovations in Theoretical Computer Science (ITCS)*, volume 151 of *LIPIcs*, pages 68:1–68:26, 2020. doi:10.4230/LIPIcs.ITCS.2020.68. (cit. on p. 6)
- [Sha49] Claude E. Shannon. The synthesis of two-terminal switching circuits. *Bell System technical journal*, 28(1):59–98, 1949. doi:10.1002/j.1538-7305.1949.tb03624.x. (cit. on p. 37)
- [Spi96] Daniel A. Spielman. Linear-time encodable and decodable error-correcting codes. *IEEE Transactions on Information Theory*, 42(6):1723–1731, 1996. doi:10.1109/18.556668. (cit. on p. 56, 57)
- [Tel18] Roei Tell. Quantified derandomization of linear threshold circuits. In *Proc. 50th Annual ACM Symposium on Theory of Computing (STOC)*, pages 855–865, 2018. doi:10.1145/3188745.3188822. (cit. on p. 58)
- [Tra84] Boris A. Trakhtenbrot. A survey of Russian approaches to perebor (brute-force searches) algorithms. *IEEE Annals of the History of Computing*, 6(4):384–400, 1984. doi:10.1109/MAHC.1984.10036. (cit. on p. 1, 3)
- [Tre01] Luca Trevisan. Extractors and pseudorandom generators. *Journal of the ACM*, 48(4):860–879, 2001. doi:10.1145/502090.502099. (cit. on p. 10)

- [TZ04] Amnon Ta-Shma and David Zuckerman. Extractor codes. *IEEE Transactions on Information Theory*, 50(12):3015–3025, 2004. doi:10.1109/TIT.2004.838377. (cit. on p. 10)
- [Uhl74] D. Uhlig. On the synthesis of self-correcting schemes from functional elements with a small number of reliable elements. *Mathematical notes of the Academy of Sciences of the USSR*, 15:558–562, 1974. doi:10.1007/BF01152835. (cit. on p. 11, 37)
- [Uhl84] D. Uhlig. Zur parallelberechnung boolescher funktionen. *TR Ing.hochsch. Mittweida*, 1984. (cit. on p. 11, 37)
- [Vad12] Salil P. Vadhan. Pseudorandomness. *Foundations and Trends in Theoretical Computer Science*, 7(1-3):1–336, 2012. doi:10.1561/0400000010. (cit. on p. 30)
- [VZ13] Salil P. Vadhan and Colin Jia Zheng. A uniform min-max theorem with applications in cryptography. In *Proc. 33rd Annual International Cryptology Conference (CRYPTO)*, volume 8042 of *Lecture Notes in Computer Science*, pages 93–110, 2013. doi:10.1007/978-3-642-40041-4_6. (cit. on p. 27)
- [Wee06] Hoeteck Wee. Finding Pessiland. In *Proc. 3rd Theory of Cryptography Conference (TCC)*, volume 3876 of *Lecture Notes in Computer Science*, pages 429–442, 2006. doi:10.1007/11681878_22. (cit. on p. 2)
- [Weg87] Ingo Wegener. *The complexity of Boolean functions*. Wiley-Teubner, 1987. URL: <http://ls2-www.cs.uni-dortmund.de/monographs/bluebook/>. (cit. on p. 37)
- [Wil05] Ryan Williams. A new algorithm for optimal 2-constraint satisfaction and its implications. *Theoretical Computer Science*, 348(2-3):357–365, 2005. doi:10.1016/j.tcs.2005.09.023. (cit. on p. 44)
- [Wil13] Ryan Williams. Improving exhaustive search implies superpolynomial lower bounds. *SIAM Journal of Computing*, 42(3):1218–1244, 2013. doi:10.1137/10080703X. (cit. on p. 44)
- [Wil18] Virginia Vassilevska Williams. On some fine-grained questions in algorithms and complexity. In *Proc. of the ICM*, volume 3, pages 3431–3472, 2018. (cit. on p. 34)
- [Yao82] Andrew Chi-Chih Yao. Theory and applications of trapdoor functions (extended abstract). In *Proc. 23rd Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 80–91, 1982. doi:10.1109/SFCS.1982.45. (cit. on p. 10, 16)
- [YLW15] Yu Yu, Xiangxue Li, and Jian Weng. Pseudorandom generators from regular one-way functions: New constructions with improved parameters. *Theoretical Computer Science*, 569:58–69, 2015. doi:10.1016/j.tcs.2014.12.013. (cit. on p. 28, 29, 30)

A Proof of Theorem 5.9

Theorem 5.9. *Let $\epsilon, \delta, \alpha, f$ be defined as in Construction 5.8. If $\epsilon \geq 1/\text{poly}(n)$ and $L \leq \text{poly}(n)$, then there is a function $r : \mathbb{N} \rightarrow \mathbb{N}$ such that $G = \{G_{n,r(n)}\}_{n \in \mathbb{N}}$ is a condEP-PRG with stretch α and security 4ϵ .*

More precisely, let $\tilde{n} = n + 2d + d'$. Suppose that for every subset $\mathcal{D} \subseteq \{0,1\}^{\tilde{n}}$ such that $H(G(\mathcal{D})) \geq \tilde{n} - \Omega(\log(\frac{n}{\delta\epsilon}))$ and every k , there is an adversary of size s that 4ϵ -distinguishes $G_{n,k}(\mathcal{D})$ from the uniform random distribution. Then there is an adversary of size $s \cdot \text{poly}(nL/\epsilon)$ that inverts f on a $1 - \delta$ fraction of inputs.

For convenience, we only consider non-uniform adversaries in this section. (See also Remark 5.4.) Recall that we sometimes use a (multi-)set $\mathcal{S}$ to represent the uniform distribution over $\mathcal{S}$ and we assume that every one-way function is length-preserving.

A.1 Impagliazzo's Hardcore Lemma

Lemma A.1 ([Imp95a]). *Let f be a (candidate weak one-way) function computable in size $s_f(n)$ and $\epsilon, \delta > 0$. Suppose for every $\mathcal{E} \subseteq \{0, 1\}^n$ with $|\mathcal{E}| \geq \delta \cdot 2^n$, there is a circuit C of size $s(n)$ such that*

$$\Pr_{\mathbf{x} \leftarrow \mathcal{E}}[C(f(\mathbf{x})) \in f^{-1}(f(\mathbf{x}))] \geq \epsilon.$$

Then there is a circuit of size $O((s(n) + s_f(n)) \cdot n\epsilon^{-1})$ that inverts f on a $1 - \delta$ fraction of inputs.

Proof Sketch. In what follows, for a circuit C and an input $x \in \{0, 1\}^n$, we say that C *inverts* $f(x)$ if $C(f(x)) \in f^{-1}(f(x))$. Consider a zero-sum game between two players called Inverter and Adversary, where Inverter chooses a subset $\mathcal{E}$ of size at least $\delta \cdot 2^n$, Adversary chooses a circuit C of size $s(n)$, and the amount of money that Inverter pays to Adversary is equal to $\Pr_{\mathbf{x} \leftarrow \mathcal{E}}[C \text{ inverts } \mathbf{x}]$. Our assumption implies that Adversary can always guarantee a payoff of at least ϵ . By the Minimax theorem, there is a distribution $\mathcal{C}$ of size- $s(n)$ circuits such that no matter which subset $\mathcal{E}$ Inverter chooses, we always have

$$\Pr_{\mathbf{C} \leftarrow \mathcal{C}, \mathbf{x} \leftarrow \mathcal{E}}[\mathbf{C} \text{ inverts } \mathbf{x}] \geq \epsilon. \quad (4)$$

Let **Bad** denote the set of $x \in \{0, 1\}^n$ such that $\Pr_{\mathbf{C} \leftarrow \mathcal{C}}[\mathbf{C} \text{ inverts } x] < \epsilon$. We have $|\mathbf{Bad}| < \delta \cdot 2^n$; otherwise Inverter can take $\mathcal{E}$ to be the uniform distribution over **Bad** and contradict (4).

Consider the following distribution $\mathcal{A}$ of adversaries. Independently sample $t := O(n/\epsilon)$ circuits $A_1, A_2, \dots, A_t$ from $\mathcal{C}$. Given $y = f(x)$ as the input where $x \in \{0, 1\}^n$, $\mathcal{A}$ uses each A_i to invert y and outputs a preimage of y if any of A_i finds one. For every fixed $x \notin \mathbf{Bad}$, the probability that $\mathcal{A}$ fails to invert x is at most $(1 - \epsilon)^t \leq 4^{-n}$, hence a union bound over every $x \notin \mathbf{Bad}$ shows that with high probability, a random circuit in $\mathcal{A}$ inverts f on every $x \notin \mathbf{Bad}$, which is a $1 - \delta$ fraction of inputs. Every circuit in the support of $\mathcal{A}$ is of size $O(s(n) + s_f(n)) \cdot t \leq O((s(n) + s_f(n))n\epsilon^{-1})$, which completes the proof. $\square$

A.2 Step I: Making f Strong and Regular

Let f be a weak one-way function. The first step is to transform f into a strong and regular one-way function, but only under a certain input distribution. In particular, we will define a sequence of subsets $\mathcal{X} = \{\mathcal{X}_n\}$ (that is not necessarily easy to sample) such that on the uniform distribution over $\mathcal{X}_n$, f is both *strong* and *regular*. Here:

- We say f is *strong* on $\mathcal{X}$, if every polynomial-size adversary $\mathcal{A}$ fails to invert $f(\mathcal{X})$ except with negligible probability. (For comparison, we are only given that f is a *weak* one-way function on a uniform random input: No PPT adversary inverts f on a $(1 - 1/n^c)$ fraction of inputs, for some fixed constant $c > 0$.)
- For a function $r : \mathbb{N} \rightarrow \mathbb{N}$, we say f is *r -regular* on $\mathcal{X}$, if for every $n \in \mathbb{N}$ and every $y \in f_n(\mathcal{X}_n)$, we have $|f_{\mathcal{X}}^{-1}(y)| \in [2^{r(n)-1}, 2^{r(n)}]$. Here, $f_{\mathcal{X}}^{-1}(y) = \{x \in \mathcal{X} : f(x) = y\}$, and $|f_{\mathcal{X}}^{-1}(y)|$ denotes the size of the above set.

As discussed in Section 5.2, we use the hardcore lemma to find a subset of inputs on which f is strong. In particular, applying Lemma A.1, we have:

Claim A.2. *There is a sequence of subsets $\mathcal{X}' = \{\mathcal{X}'_n \subseteq \{0, 1\}^n\}$ with $|\mathcal{X}'_n| \geq 2^n/\text{poly}(n)$ such that for every polynomial-size adversary $\mathcal{A}$,*

$$\Pr_{\mathbf{x} \leftarrow \mathcal{X}'_n}[\mathcal{A}(f(\mathbf{x})) \in f^{-1}(f(\mathbf{x}))] < \text{negl}(n).$$

More precisely, suppose that for every subset $\mathcal{X}'_n \subseteq \{0, 1\}^n$ with $|\mathcal{X}'_n| \geq \frac{\delta}{2} \cdot 2^n$, there is an adversary of size s that inverts $f_n(\mathcal{X}'_n)$ w.p. at least ϵ . Then there is an adversary of size $O(s \cdot n\epsilon^{-2})$ that inverts f on a $1 - \delta$ fraction of inputs. $\square$

Now, for every string $y \in \{0, 1\}^n$, let $|f_{\mathcal{X}'}^{-1}(y)|$ denote the number of inputs $x \in \mathcal{X}'_n$ such that $f(x) = y$. For $r \in [1, n]$, let W_r be the number of strings $x \in \mathcal{X}'_n$ such that $|f_{\mathcal{X}'}^{-1}(f(x))| \in [2^{r-1}, 2^r]$. Then we have $\sum_{r=1}^n W_r \geq |\mathcal{X}'_n|$. By averaging, there is an integer $r \in [1, n]$ such that $W_r \geq |\mathcal{X}'_n|/n$. We define $r(n)$ to be this integer r , and

$$\mathcal{X}_n := \{x \in \mathcal{X}'_n : |f_{\mathcal{X}'}^{-1}(f(x))| \in [2^{r-1}, 2^r]\}.$$

By definition, f is r -regular on $\mathcal{X} := \{\mathcal{X}_n\}$. Since $|\mathcal{X}_n| \geq |\mathcal{X}'_n|/n$, any adversary that inverts f on an ϵ fraction of $\mathcal{X}_n$ also inverts f on an ϵ/n fraction of $\mathcal{X}'_n$. To summarize:

Claim A.3. *There is a function $r(n) \leq n$ and a sequence of subsets $\mathcal{X} = \{\mathcal{X}_n\}$ with $|\mathcal{X}_n| \geq 2^n/\text{poly}(n)$ such that f is r -regular on $\mathcal{X}$ and that for every polynomial-size adversary $\mathcal{A}$,*

$$\Pr_{\mathbf{x} \leftarrow \mathcal{X}_n} [\mathcal{A}(f(\mathbf{x})) \in f^{-1}(f(\mathbf{x}))] < \text{negl}(n).$$

More precisely, suppose that for every function $r : \mathbb{N} \rightarrow \mathbb{N}$ and sequence of subsets $\mathcal{X} = \{\mathcal{X}_n\}$ such that $|\mathcal{X}_n| \geq \frac{\delta}{2^n} \cdot 2^n$ and f is r -regular on $\mathcal{X}$, there is an adversary of size s that inverts $f_n(\mathcal{X}_n)$ w.p. at least ϵ . Then there is an adversary of size $s \cdot \text{poly}(n/\epsilon)$ that inverts f on a $1 - \delta$ fraction of inputs. $\square$

A.3 Step II: An Intermediate Function

We define another function ensemble $\tilde{f} = \{\tilde{f}_n\}_{n \in \mathbb{N}}$. Let $k_1 = r - 1$, $k_2 = \lfloor n - r - \log(2n/\delta) \rfloor$, and $d = d_{\text{Ext}}(n, \epsilon)$. We need the following two extractors:

- a strong (k_1, ϵ) -extractor $\text{Ext}_1 : \{0, 1\}^n \times \{0, 1\}^d \rightarrow \{0, 1\}^{m_1}$, where $m_1 := k_1 - 2 \log(1/\epsilon) - O(1)$;
- a strong (k_2, ϵ) -extractor $\text{Ext}_2 : \{0, 1\}^n \times \{0, 1\}^d \rightarrow \{0, 1\}^{m_2}$, where $m_2 := k_2 - 2 \log(1/\epsilon) - O(1)$.

The function $\tilde{f}_n : \{0, 1\}^n \times \{0, 1\}^{2d} \rightarrow \{0, 1\}^{m_1+m_2+2d}$ is defined as follows.

$$\tilde{f}_n(x, z_1, z_2) := z_1 \circ \text{Ext}_1(x, z_1) \circ z_2 \circ \text{Ext}_2(f(x), z_2).$$

Denote $\ell(n) := m_1 + m_2 + 2d$. The following lemma summarizes the properties of $\tilde{f}_n$ we need:

Lemma A.4. *For every integer n , the function $\tilde{f}_n$ satisfies the following properties:*

(Uniformity) For every integer n , $\text{SD}(\tilde{f}_n(\mathcal{X}_n, \mathcal{U}_{2d}), \mathcal{U}_{\ell(n)}) \leq 2\epsilon$.

(Hiding) For every polynomial-size adversary $\mathcal{A}$ and every integer n ,

$$\Pr_{\mathbf{x} \leftarrow \mathcal{X}_n} [\mathcal{A}(\tilde{f}_n(\mathbf{x}, \mathcal{U}_{2d})) = \mathbf{x}] \leq \text{negl}(n).$$

More precisely, if there is an adversary $\mathcal{A}$ of size s that on input $\tilde{f}_n(\mathcal{X}_n, \mathcal{U}_{2d})$ guesses $\mathcal{X}_n$ with success probability γ , then there is an adversary $\mathcal{A}'$ of size $O(s)$ that inverts $f_n(\mathcal{X}_n)$ w.p. at least $\Omega(\gamma/\epsilon^2)$.

Proof. (Uniformity) A sample from $\mathcal{X}_n$ can be obtained from two steps. First, we sample a string y_0 with probability $p(y_0) := \Pr_{\mathbf{x} \leftarrow \mathcal{X}_n} [f_n(\mathbf{x}) = y_0]$. Then we sample a string x_0 with probability $p(x_0 | y_0) := \Pr_{\mathbf{x} \leftarrow \mathcal{X}_n} [\mathbf{x} = x_0 | f_n(\mathbf{x}) = y_0]$.

Suppose y_0 is fixed. Since f is r -regular, we have $|f_{\mathcal{X}}^{-1}(y_0)| \geq 2^{r-1}$. Therefore, conditioned on y_0 , the min-entropy of the distribution of x_0 is at least $r - 1 \geq k_1$.

Let $\mathbf{x} \leftarrow \mathcal{X}_n$ and $\mathbf{z}_1 \leftarrow \mathcal{U}_d$. Since Ext_1 is a strong (k_1, ϵ) -extractor, we have

$$\text{SD}(\mathbf{z}_1 \circ \text{Ext}_1(\mathbf{x}, \mathbf{z}_1), \mathcal{U}_{d+m_1} \mid f(\mathbf{x})) \leq \epsilon.$$

(Here, the LHS denotes the statistical distance between $\mathbf{z}_1 \circ \text{Ext}_1(\mathbf{x}, \mathbf{z}_1)$ and $\mathcal{U}_{d+m_1}$ conditioned on the value $f(\mathbf{x})$.)

Now, for every $y_0 \in f_n(\mathcal{X}_n)$, since $|f_{\mathcal{X}}^{-1}(y_0)| \leq 2^r$, the probability that a sample of $f_n(\mathcal{X}_n)$ is equal to the particular y_0 is at most $2^r/|\mathcal{X}_n|$. It follows that the min-entropy of the distribution of y_0 is at least $\log(|\mathcal{X}_n|/2^r) \geq n - r + \log(\delta/2n) \geq k_2$. Since Ext_2 is a strong (k_2, ϵ) -extractor, we have

$$\text{SD}(\mathbf{z}_2 \circ \text{Ext}_2(f(\mathbf{x}), \mathbf{z}_2), \mathcal{U}_{d+m_2}) \leq \epsilon.$$

It follows that

$$\begin{aligned} & \text{SD}(\mathbf{z}_1 \circ \text{Ext}_1(\mathbf{x}, \mathbf{z}_1) \circ \mathbf{z}_2 \circ \text{Ext}_2(f(\mathbf{x}), \mathbf{z}_2), \mathcal{U}_{\ell(n)}) \\ & \leq \text{SD}(\mathbf{z}_1 \circ \text{Ext}_1(\mathbf{x}, \mathbf{z}_1) \circ \mathbf{z}_2 \circ \text{Ext}_2(f(\mathbf{x}), \mathbf{z}_2), \mathcal{U}_{d+m_1} \circ \mathbf{z}_2 \circ \text{Ext}_2(f(\mathbf{x}), \mathbf{z}_2)) \\ & \quad + \text{SD}(\mathcal{U}_{d+m_1} \circ \mathbf{z}_2 \circ \text{Ext}_2(f(\mathbf{x}), \mathbf{z}_2), \mathcal{U}_{\ell(n)}) \\ & \leq \epsilon + \epsilon = 2\epsilon. \end{aligned}$$

(Hiding) Let $\mathcal{A}$ be any adversary that violates the Hiding property. Suppose that

$$\Pr_{\mathbf{x} \leftarrow \mathcal{X}_n} [\mathcal{A}(\tilde{f}_n(\mathbf{x}, \mathcal{U}_{2d})) = \mathbf{x}] \geq \gamma.$$

We will use $\mathcal{A}$ to build an algorithm $\mathcal{A}'$ that inverts $f(\mathcal{X}_n)$ w.p. $O(\gamma/\epsilon^2)$.

Let $\mathbf{x} \leftarrow \mathcal{X}_n$ be a hidden string and $\mathbf{y} = f_n(\mathbf{x})$ be the input of $\mathcal{A}'$. We sample $\mathbf{z}_1, \mathbf{z}_2 \leftarrow \mathcal{U}_d$. We also “guess” a string $\mathbf{z} \leftarrow \mathcal{U}_{m_1}$, with the hope that $\text{Ext}_1(\mathbf{x}, \mathbf{z}_1) = \mathbf{z}$. Then we output $\mathcal{A}'(\mathbf{y}) := \mathcal{A}(\mathbf{z}_1 \circ \mathbf{z} \circ \mathbf{z}_2 \circ \text{Ext}_2(\mathbf{y}, \mathbf{z}_2))$.

Conditioned on $\text{Ext}_1(\mathbf{x}, \mathbf{z}_1) = \mathbf{z}$, the distribution of $\mathbf{z}_1 \circ \mathbf{z} \circ \mathbf{z}_2 \circ \text{Ext}_2(\mathbf{y}, \mathbf{z}_2)$ is exactly $\tilde{f}_n(\mathcal{X}_n, \mathcal{U}_{2d})$. Therefore,

$$\Pr[\mathcal{A}'(\mathbf{y}) = \mathbf{x}] \geq \gamma \cdot \Pr[\text{Ext}_1(\mathbf{x}, \mathbf{z}_1) = \mathbf{z}] = \gamma \cdot 2^{-m_1}.$$

Note that besides $\mathbf{y} = f(\mathbf{x})$, $\mathcal{A}'$ does not know any information about $\mathbf{x}$. Therefore, for every $x' \in f^{-1}(\mathbf{y})$, the probability that $\mathcal{A}'(\mathbf{y}) = x'$ should also be at least $\gamma \cdot 2^{-m_1}$. We have

$$\begin{aligned} \Pr_{\mathbf{y} \in f(\mathcal{X}_n)} [\mathcal{A}'(\mathbf{y}) \in f^{-1}(\mathbf{y})] & \geq \gamma \cdot 2^{-m_1} \cdot |f_{\mathcal{X}}^{-1}(\mathbf{y})| \\ & \geq \gamma \cdot 2^{r-1-m_1} = O(\gamma/\epsilon^2). \end{aligned} \quad \square$$

A.4 Step III: Appending a Hardcore Function

Note that the output length of $\tilde{f}_n$ is $\tau := \log(n/\delta) + 4 \log \frac{1}{\epsilon} + O(1)$ bits shorter than the input length of $\tilde{f}_n$. In this section, we append a hardcore function at the end of $\tilde{f}_n$, making it a pseudorandom generator with stretch $\alpha > 0$. In particular, we need:

- a hardcore function $\text{HC} : \{0, 1\}^n \times \{0, 1\}^{d'} \rightarrow \{0, 1\}^k$ with distinguishing probability ϵ , where $k := \tau + \alpha$, and $d' := d_{\text{HC}}(n, k, \epsilon)$. Let R be the reconstruction algorithm of this hardcore function, and $L := L(n, k, \epsilon)$ be the list size.

Let $\tilde{n} := n + 2d + d'$. Recall that $G_{n,r} : \{0, 1\}^{\tilde{n}} \rightarrow \{0, 1\}^{\tilde{n}+\alpha}$ is defined as

$$G_{n,r}(x, z_1, z_2, z_3) := z_1 \circ \text{Ext}_1(x, z_1) \circ z_2 \circ \text{Ext}_2(f(x), z_2) \circ z_3 \circ \text{HC}(x, z_3).$$

Let $\mathcal{E}_{\tilde{n}} := \mathcal{X}_n \times \{0, 1\}^{2d+d'}$. In other words, a uniform random string from $\mathcal{E}_{\tilde{n}}$ can be sampled as $\mathbf{x} \circ \mathbf{z}$ where $\mathbf{x} \leftarrow \mathcal{X}_n$ and $\mathbf{z} \leftarrow \mathcal{U}_{2d+d'}$. We will show that $G_{n,r}$ is a conDEP-PRG whose “condition” is $\mathcal{E}_{\tilde{n}}$. In particular, Lemma A.5 shows that $G_{n,r}(\mathcal{E}_{\tilde{n}})$ is pseudorandom, and Lemma A.6 shows that $G_{n,r}(\mathcal{E}_{\tilde{n}})$ is entropy-preserving.

Lemma A.5. *Every polynomial-size adversary $\mathcal{A}$ fails to 4ϵ -distinguish $G_{n,r}(\mathcal{E}_{\tilde{n}})$ from $\mathcal{U}_{\ell(n)+d'+k}$. More precisely, if there is an adversary $\mathcal{A}$ of size s that 4ϵ -distinguishes $G_{n,r}(\mathcal{E}_{\tilde{n}})$ from $\mathcal{U}_{\ell(n)+d'+k}$, then there is an adversary $\mathcal{A}'$ of size $s \cdot \text{poly}(n/\epsilon)$ that on input $\tilde{f}_n(\mathcal{X}_n, \mathcal{U}_{2d})$, guesses $\mathcal{X}_n$ with success probability $\epsilon/2L$.*

Proof. Suppose $\mathcal{A}$ is an adversary that 4ϵ -distinguishes $G_{n,r}(\mathcal{E}_{\tilde{n}})$ from $\mathcal{U}_{\ell(n)+d'+k}$.

Since $\text{SD}(\tilde{f}_n(\mathcal{X}_n, \mathcal{U}_{2d}), \mathcal{U}_{\ell(n)}) \leq 2\epsilon$, it must be the case that $\mathcal{A}$ could 2ϵ -distinguish $G_{n,r}(\mathcal{E}_{\tilde{n}})$ from $\tilde{f}_n(\mathcal{X}_n, \mathcal{U}_{2d}) \circ \mathcal{U}_{d'+k}$. Equivalently, let $\mathbf{x} \leftarrow \mathcal{X}_n$, then given the information of $\tilde{f}(\mathbf{x}, \mathcal{U}_{2d})$, $\mathcal{A}$ could 2ϵ -distinguish $\mathcal{U}_{d'} \circ \text{HC}(\mathbf{x}, \mathcal{U}_{d'})$ from $\mathcal{U}_{d'+k}$. We say a string $w := (x, z_1, z_2)$ is *good* if $\mathcal{A}$ could ϵ -distinguish $\tilde{f}_n(w) \circ \mathcal{U}_{d'} \circ \text{HC}(x, \mathcal{U}_{d'})$ from $\tilde{f}_n(w) \circ \mathcal{U}_{d'+k}$. Then by a Markov bound, a random $\mathbf{w} \leftarrow \mathcal{X}_n \circ \mathcal{U}_{2d}$ is good w.p. at least ϵ .

The adversary $\mathcal{A}'$ that violates the (Hiding) property of $\tilde{f}_n$ simply follows from the reconstruction algorithm R . In particular, on input $\tilde{f}_n(w) = \tilde{f}_n(x, z_1, z_2)$, $\mathcal{A}'$ constructs the following oracle:

$$\mathcal{O}(r) := \mathcal{A}(\tilde{f}_n(w) \circ r),$$

runs the algorithm $R^\mathcal{O}$ to obtain a list of size L , and outputs a random element in the list.

We analyze $\mathcal{A}'$. Suppose $\mathcal{A}'$ is given $\tilde{f}_n(w)$ for a good w , then $\mathcal{O}$ indeed ϵ -distinguishes $\mathcal{U}_{d'} \circ \text{HC}(x, \mathcal{U}_{d'})$ from $\mathcal{U}_{d'+k}$. Therefore, w.p. $\geq 1/2$, x is in the list outputted by $R^\mathcal{O}$. If this is the case, we will correctly output x w.p. $\geq 1/L$. It follows that on input $\tilde{f}_n(\mathbf{x}, \mathcal{U}_{2d})$ where $\mathbf{x} \leftarrow \mathcal{X}_n$, $\mathcal{A}'$ outputs $\mathbf{x}$ w.p. $\geq \epsilon/2L$. Finally, as R is a polynomial-size oracle circuit (actually a PPT oracle machine), the size of $\mathcal{A}'$ is $s(n) \cdot \text{poly}(n/\epsilon)$. $\square$

Lemma A.6. *Suppose that $\epsilon < \frac{1}{10n^2}$. Then $\text{H}(G_{n,r}(\mathcal{E}_{\tilde{n}})) \geq \tilde{n} - \tau - 2$.*

Proof. Since $\text{SD}(\tilde{f}_n(\mathcal{X}_n, \mathcal{U}_{2d}), \mathcal{U}_{\ell(n)}) \leq 2\epsilon < \frac{1}{\ell(n)^2}$, by [LP20, Lemma 2.2], we have $\text{H}(\tilde{f}_n(\mathcal{X}_n, \mathcal{U}_{2d})) \geq \ell(n) - 2$. It follows that $\text{H}(G_{n,r}(\mathcal{E}_{\tilde{n}})) \geq (\ell(n) - 2) + d' \geq \tilde{n} - \tau - 2$. $\square$

A.5 Putting It Together

Proof of Theorem 5.9. Suppose that for every $\mathcal{X} = \{\mathcal{X}_n\}$ that satisfies the premise of Claim A.3, and $\mathcal{E}_{\tilde{n}}$ defined above, there is an adversary of size $s(n)$ that 4ϵ -distinguishes $G_{n,r}(\mathcal{E}_{\tilde{n}})$ from the uniform distribution. Then:

- By Lemma A.5, there is an adversary of size $s(n) \cdot \text{poly}(n/\epsilon)$ that on input $\tilde{f}(\mathcal{X}_n, \mathcal{U}_{d_1+d_2})$, guesses $\mathcal{X}_n$ w.p. $\geq \epsilon/2L$.
- By Lemma A.4, there is an adversary of size $s(n) \cdot \text{poly}(n/\epsilon)$ that inverts $f_n(\mathcal{X}_n)$ w.p. $\geq \frac{1}{2\epsilon L}$.

It follows from Claim A.3 that there is an adversary of size $s \cdot \text{poly}(nL/\epsilon)$ that inverts f on a $1 - \delta$ fraction of inputs. $\square$

B Proof of Theorem 5.15

In this section, we briefly review the universal hash functions in [IKOS08] that are computable by linear-size circuits, with an emphasis on the uniformity of these circuits. Throughout this section, a circuit family is uniform if it satisfies Definition 5.13. An XOR-circuit is a (multi-output) circuit that only uses XOR gates of fan-in 2. To match Definition 5.13, we also require that every gate in an XOR-circuit has fan-out at most 2.

Outline. Our start point is the strongly explicit family of expanders by [Mar73, GG81]. Spielman [Spi96] showed that these expanders imply asymptotically optimal error-correcting codes (i.e., with constant rate and constant relative distance). Using an expander walk trick, for any constant $\epsilon > 0$, one could construct error-correcting codes with relative distance $1 - \epsilon$, constant rate, and constant alphabet size. By the construction of [IKOS08], such codes imply universal hash functions.

B.1 Strongly Explicit Expanders

We use the following construction due to [Mar73, GG81]. (See also [HLW06, Construction 8.1].) For every integer n , we have a graph $\mathcal{G}_n$ with n^2 vertices such that every vertex has degree 8. The vertex set of $\mathcal{G}_n$ is $\mathbb{Z}_n \times \mathbb{Z}_n$. Each vertex $v = (x, y)$ is adjacent to the following vertices

$$\gamma_1(v) = (x + 2y, y), \gamma_2(v) = (x + 2y + 1, y), \gamma_3(v) = (x, y + 2x), \gamma_4(v) = (x, y + 2x + 1).$$

Here the additions are modulo n . Note that $\gamma_1, \dots, \gamma_4$ are bijections and the other four neighbors of v are simply $\gamma_1^{-1}(v), \dots, \gamma_4^{-1}(v)$. The graph might contain self-loops or parallel edges.

Theorem B.1 ([GG81]). *For every integer $n \geq 1$, the second largest eigenvalue of the adjacency matrix of $\mathcal{G}_n$ is at most $5\sqrt{2} < 8$.*

In our construction, we need the degree of the expanders to be a large enough constant. We can simply pick a large enough constant k and take the k -th power of $\mathcal{G}_n$. Let $\mathcal{G}_n^k$ be the k -th power of $\mathcal{G}_n$, i.e., for every $u, v \in V(\mathcal{G}_n)$, the number of (parallel) edges between u and v in $\mathcal{G}_n^k$ is equal to the number of length- k paths between u and v in $\mathcal{G}_n$. Then the degree of $\mathcal{G}_n^k$ is $d := 8^k$, and the second largest eigenvalue of the adjacency matrix of $\mathcal{G}_n^k$ is at most $(5\sqrt{2})^k < d$.

Remark B.2. It will be convenient to define an explicit mapping (bijection) between $E(\mathcal{G}_n^k)$ and $[dn^2/2]$. Note that each edge $(u, v) \in \mathcal{G}_n^k$ can be represented by a start vertex u and a string $\sigma_1\sigma_2\dots\sigma_k$ where each $\sigma_i \in \Sigma := \{\gamma_1, \gamma_2, \gamma_3, \gamma_4, \gamma_1^{-1}, \gamma_2^{-1}, \gamma_3^{-1}, \gamma_4^{-1}\}$. The meaning of this representation is that $(\sigma_1 \circ \sigma_2 \circ \dots \circ \sigma_k)(u) = v$. Each edge has two representations: $(u, \sigma_1\sigma_2\dots\sigma_k)$ or $(v, \sigma_k^{-1}\sigma_{k-1}^{-1}\dots\sigma_1^{-1})$. We arbitrarily choose a size- $(d/2)$ subset S of Σ^k such that for each $\sigma_1, \sigma_2, \dots, \sigma_k \in \Sigma$, exactly one of $\sigma_1\sigma_2\dots\sigma_k$ and $\sigma_k^{-1}\sigma_{k-1}^{-1}\dots\sigma_1^{-1}$ is in S . We fix and hardcode a bijection between $[d/2]$ and S . Given an integer $i \in [dn^2/2]$, we interpret i as a pair of $v \in V(\mathcal{G}_n^k)$ and $\sigma_1\sigma_2\dots\sigma_k \in S$, and the edge corresponding to i is represented as $(v, \sigma_1\sigma_2\dots\sigma_k)$. This bijection and its inverse are computable in time $O(\log n)$.

B.2 Error-Reduction Codes

An intermediate step in [Spi96] is to construct *error-reduction codes*, which are weaker primitives compared to error-correcting codes.

Let $r, \delta, \epsilon > 0$ be constants. An error-reduction code of rate r , error reduction ϵ , and reducible distance δ is a function $\mathcal{C} : \{0, 1\}^{rn} \rightarrow \{0, 1\}^{(1-r)n}$ mapping rn “message” bits into $(1-r)n$ “check” bits such that the following holds. The codeword of a message x is $x \circ \mathcal{C}(x)$. For any message x , if we are given a corrupted codeword that differs from $x \circ \mathcal{C}(x)$ with at most $v \leq \delta n$ message bits and at most $t \leq \delta n$ check bits, then we can recover a codeword that differs from $x \circ \mathcal{C}(x)$ in at most ϵt message bits. (We will not be particularly interested in the complexity of recovery or decoding algorithms.)

We need a family of error-reduction codes $\mathcal{R}_n : \{0, 1\}^n \rightarrow \{0, 1\}^{\lfloor n/2 \rfloor}$ (that is, with rate $2/3$). The following construction appears in [Spi96, Section V] and we verify its uniformity:

Lemma B.3. *For some absolute constant $\delta > 0$, there is a family of error-reduction codes $\mathcal{R} = \{\mathcal{R}_n : \{0, 1\}^n \rightarrow \{0, 1\}^{\lfloor n/2 \rfloor}\}$ with error-reduction $1/2$ and reducible distance δ . Moreover, the sequence of functions $\{\mathcal{R}_n\}$ can be computed by a uniform family of linear-size XOR-circuits.*

Proof Sketch. Let ϵ and r be the constants fixed in the proof of [Spi96, Theorem 19], and $d := 8^k$ for some large enough constant k . Let m be the smallest integer such that $dm^2/2 \geq n$. Note that m can be computed in $O(\log n)$ time. Since $r > 4/5$, we have that $dm^2(1-r)/r \leq n/2$. It suffices to construct an error-reduction code with $dm^2/2$ message bits and $dm^2(1-r)/r$ check bits.

The error-reduction code will be of the form $\mathcal{R}(B, \mathcal{S})$ (as in [Spi96, Definition 16]) with B being the edge-vertex incidence graph of $\mathcal{G}_m^k$ and $\mathcal{S}$ being a good (linear) error-correcting code on d -bit messages that has rate r . (Since d is a constant, we can hardcode $\mathcal{S}$ in our algorithm. On the other hand, since d is large enough, $\mathcal{S}$ exists.) That is, we assign a message bit to every edge of $\mathcal{G}_m^k$. For each vertex $v \in V(\mathcal{G}_m^k)$, let $b_1 b_2 \dots b_d$ be the bits on the d incident edges of v . This vertex outputs $d(1-r)/r$ check bits which are the check bits of $\mathcal{S}$ on message $b_1 b_2 \dots b_d$. Concatenating the outputs of each vertex, we obtain an error-reduction code of $dm^2/2$ message bits and $dm^2(1-r)/r$ check bits. Inspecting the proof of [Spi96, Theorem 19], one can see that for some absolute constant $\delta > 0$, this error-reduction code has error-reduction $1/2$ and reducible distance δ .

Computing the i -th gate of the encoding circuit reduces to computing the indices of the incident edges of a vertex $v \in V(\mathcal{G}_m^k)$. By Remark B.2, this is computable in $O(\log n)$ time. $\square$

B.3 Error-Correcting Codes

The construction in [Spi96, Section II] transforms an error-reduction code into an error-correcting code. Here we only review its encoding algorithm and check that they can be implemented by uniform XOR circuits. The correctness of this error-correcting code is proved in [Spi96].

Lemma B.4. *There is a constant $n_0 > 1$ and a family $\mathcal{C} = \{\mathcal{C}_k : \{0, 1\}^{n_0 2^{k-2}} \rightarrow \{0, 1\}^{n_0 2^k}\}$ of error-correcting codes with constant relative distance. Moreover, $\mathcal{C}$ can be encoded by a uniform family of linear-size XOR circuits.*

Proof Sketch. We recursively define $\mathcal{C}_k$ as follows. First, $\mathcal{C}_0 : \{0, 1\}^{n_0/4} \rightarrow \{0, 1\}^{n_0}$ is any good enough error-correcting code. Since n_0 is a constant, our algorithm can hardcode $\mathcal{C}_0$.

Now, for $k \geq 1$, we define $\mathcal{C}_k$ as follows. Let $x \in \{0, 1\}^{n_0 2^{k-2}}$ be the inputs of $\mathcal{C}_k$.

- The first $n_0 2^{k-2}$ outputs of $\mathcal{C}_k$ will always be x itself.²² Note that we require the fan-out of gates to be at most 2. Therefore we need to make a copy of x . Similarly, we may need to copy the A_k, B_k, C_k defined below. The circuit size is still linear in 2^k .
- We pick an error-reduction code $\mathcal{R}_{k-2} : \{0, 1\}^{n_0 2^{k-2}} \rightarrow \{0, 1\}^{n_0 2^{k-3}}$, and output $A_k := \mathcal{R}_{k-2}(x)$.
- Let $\mathcal{C}_{k-1} : \{0, 1\}^{n_0 2^{k-3}} \rightarrow \{0, 1\}^{n_0 2^{k-1}}$ be the error-correcting code we recursively defined. Let $A_k \circ B_k := \mathcal{C}_{k-1}(A_k)$, and we output B_k . (Recall that the first $n_0 2^{k-3}$ outputs of $\mathcal{C}_{k-1}$ is equal to its inputs, i.e., A_k .)
- We pick an error-reduction code $\mathcal{R}_{k-1} : \{0, 1\}^{n_0 2^{k-1}} \rightarrow \{0, 1\}^{n_0 2^{k-2}}$ and output $C_k := \mathcal{R}_{k-1}(A_k \circ B_k)$.

The required error-reduction codes are constructed in Lemma B.3. The total number of output bits of $\mathcal{C}_k$ is $|x| + |A_k| + |B_k| + |C_k|$ which is indeed $n_0 2^k$.

The i -th gate of the encoding circuit of $\mathcal{C}_{k-1}$ can be computed as follows. Let $c 2^k$ be the circuit complexity of the first, second, and fourth bullet. (That is, circuit complexity of $\mathcal{C}_k$ not counting the recursive part for $\mathcal{C}_{k-1}$.) We may assume c is a power of 2. The encoding circuit for $\mathcal{C}_k$ has $|\mathcal{C}_0| + \sum_{i=1}^k c 2^i = |\mathcal{C}_0| + c(2^{k+1} - 1)$ gates. Taking the (base-2) logarithm of $(i - |\mathcal{C}_0|)/c$,

²²We can assume this is also true for $\mathcal{C}_0$.

we can find the “level of recursion” that the i -th gate is constructed. Then the problem reduces to computing the encoding circuit of $\mathcal{R}_j$ for some integer j , which is computable in $O(\log n)$ time. $\square$

For every constant $\epsilon > 0$, the construction of [IKOS08] needs a code with relative distance $1 - \epsilon$ and constant alphabet size. As in [ABN⁺92, GI01], we can “amplify” the code in Lemma B.4 by an expander:

Lemma B.5. *For every constant $\epsilon > 0$, there is a constant $D > 0$ and a family of error-correcting codes $\{\mathcal{C}'_k : \{0, 1\}^{n_0 2^{k-2}} \rightarrow [2^D]^{O(2^k)}\}$ that has relative distance $1 - \epsilon$. Moreover, if we interpret $[2^D]$ as length- D strings, then $\mathcal{C}'_k$ can be encoded by a uniform family of linear-size XOR circuits.*

Proof Sketch. Recall that $\{\mathcal{G}_n\}$ is the expander family constructed in Theorem B.1, and $\{\mathcal{C}_k : \{0, 1\}^{n_0 2^{k-2}} \rightarrow \{0, 1\}^{n_0 2^k}\}$ is the family of error-correcting codes constructed in Lemma B.4. Let m be the smallest integer such that $m^2 \geq n_0 2^k$. We pad zeros to the outputs of $\mathcal{C}_k$ so that $\mathcal{C}_k$ can be regarded as a code that outputs m^2 bits. We assign an output bit of $\mathcal{C}_k$ to each vertex in $\mathcal{G}_m$. The relative distance of $\mathcal{C}_k$ is still lower bounded by an absolute constant $\delta > 0$.

We will pick a large enough constant p such that $\mathcal{G}_m^p$ has good expansion property: Every subset of $V(\mathcal{G}_m^p)$ with size at least $\delta \cdot m^2$ has at least $(1 - \epsilon)m^2$ neighbors. (See e.g. [ABN⁺92, Corollary 1].) Let $D := 8^p$, then every vertex in $\mathcal{G}_m^p$ has degree D . On input $x \in \{0, 1\}^{n_0 2^{k-2}}$, recall that we assigned each vertex in $V(\mathcal{G}_m)$ a bit of the codeword $\mathcal{C}_k(x)$. For every $v \in V(\mathcal{G}_m^p)$, the vertex v will output the concatenation of the bits assigned to its neighbor, which can be interpreted as an element in $[2^D]$. The code $\mathcal{C}'_k(x)$ simply concatenates the outputs of each vertex $v \in V(\mathcal{G}_m^p)$ together.

Consider the encoding circuits of $\mathcal{C}'_k$. As we need each gate to have fanout at most 2, we make D copies of the encoding circuit of $\mathcal{C}_k$. For every $\sigma = \sigma_1 \sigma_2 \dots \sigma_k \in \Sigma^k$, we have a copy of $\mathcal{C}_k$ denoted as $\mathcal{C}_k^\sigma$. For each vertex v , and each $\sigma \in \Sigma^k$, let u be the σ -th neighbor of v . The σ -th bit of the output of v is the u -th output of the circuit $\mathcal{C}_k^\sigma$. We can see this encoding circuit is uniform. $\square$

Remark B.6. Lijie Chen (personal communication) suggested a similar approach based on expander random walks [Tel18, Proposition 6.6]. As the p -th power of an expander graph G consists of length- p walks in G , the two approaches are essentially the same.

B.4 Universal Hash Functions

Finally, we are ready to verify that the universal hash functions in [IKOS08] are uniform.

Theorem 5.15. *For every integer n, m where $m = O(n)$, there exists an integer $k = O(n)$ and a family of universal hash functions $\{h_{n,m} : \{0, 1\}^n \times \{0, 1\}^k \rightarrow \{0, 1\}^m\}$ such that $h_{n,m}$ can be computed by a uniform family of linear-size circuits that are skew w.r.t. the second argument.*

Proof Sketch. Let $n_1 := cn$ for a large enough constant c , ϵ be a small enough constant, and D be the constant in Lemma B.5 depending on ϵ . We need three ingredients:

- An ℓ -exposure resilient function (ERF) $\text{ERF} : \{0, 1\}^{n_1 D} \rightarrow \{0, 1\}^m$ [CDH⁺00]. It is shown in [CGH⁺85] that for any (linear) error-correcting code $\mathcal{C} : \{0, 1\}^m \rightarrow \{0, 1\}^n$ with generator matrix G and minimum distance d , the transpose matrix $G^\top$ mapping n input bits to m output bits is a perfect ℓ -ERF where $\ell := n - d + 1$.

For an XOR-circuit C that computes the linear transform G over $\text{GF}(2)$, we can obtain a circuit computing the linear transform $G^\top$ by exchanging the input gates and output gates and reversing the directions of every wire [Bor57, IKOS08]. In particular, every gate

$g \in C$ whose output feeds to the gates $g_1, g_2, \dots, g_k$ becomes, in the new circuit, an XOR gate g whose *inputs* are $g_1, g_2, \dots, g_k$.

Therefore, Lemma B.4 shows that an ℓ -ERF $\text{ERF} : \{0, 1\}^{n_1 D} \rightarrow \{0, 1\}^m$ is computable by a uniform family of linear-size XOR circuits.

- An error-correcting code $\mathcal{C}'_k : \{0, 1\}^n \rightarrow [2^D]^{n_1}$ with relative distance $1 - \epsilon$, as in Lemma B.5.
- A hash family $H : \{0, 1\}^D \times \{0, 1\}^{2D-1} \rightarrow \{0, 1\}^D$, computable by a skew circuit w.r.t. the second argument. As D is a constant, we can simply hardcode this hash family. (See e.g. Section 5.2.1 for an instantiation based on Toeplitz matrices.)

The construction of [IKOS08] goes as follows. On input $x \in \{0, 1\}^n$, we first compute $\mathcal{C}'(x) \in [2^{h+1}]^{n_1}$. Next, we receive n_1 keys $k_1, k_2, \dots, k_{n_1} \in \{0, 1\}^{2h+1}$ which are the keys for our hash function. Let $t \in [2^{h+1}]^{n_1}$ be the following message: $t_i := H(\mathcal{C}'(x)_i, k_i)$. We treat t as a string of length $m(h+1)$ and the output of our hash function is $\text{ERF}(t)$.

It is easy to see that this family is uniform. □